

COMPLETE STUDY PACK

Backend Interview Prep

Every claim on my resume, explained from zero — Java and OOP through Spring Boot, JPA/Hibernate, PostgreSQL, JWT security, React, machine learning, and three projects defended line by line.

How to use this pack

Work through the sections in order — they are sequenced so each one builds on the last. The callouts mean the same thing throughout:

SAY: an answer you can speak out loud • **TRAP:** where an interviewer will push • **SELF-TEST:** check yourself before moving on

Start with section 00 — it holds the study schedule, the self-introduction, and the ranked list of the riskiest claims on the resume.

Siva Charan Kollauram Gandla • Software Developer • October 2026
17 sections • written from the resume and the public project repositories

Contents

Sections are in study order. Every section and subsection is also a bookmark in the PDF sidebar.

- 00 How to use this pack 3
- 01 JAVA FROM ZERO + OOP 11
- 02 SPRING BOOT FROM ABSOLUTE ZERO 20
- 03 REST APIs, HTTP, JSON 27
- 04 DATABASE → ORM → JPA → HIBERNATE → SPRING DATA JPA 33
- 05 POSTGRESQL + SQL 42
- 06 SECURITY: SPRING SECURITY + JWT + BCrypt + ROLES 49
- 07 JAVASCRIPT + REACT (including the AlgoViz engine) 59
- 08 HTML, CSS, TAILWIND, VITE 69
- 09 NODE.JS, EXPRESS, FLASK, PYTHON 74
- 10 MACHINE LEARNING: SCIKIT-LEARN, RANDOM FOREST, PANDAS, NUMPY, PREPROCESSING 80
- 11 GIT / GITHUB + DSA 88
- 12 PROJECT DEFENSE (REVERTPAY · EDU2JOB · ALGOVIZ) 96
- 13 "SHOW ME" · "WHY?" · "WHAT IF?" 111
- 14 RESUME BULLET ATTACK + THE NO-BLUFF SYSTEM 119
- 15 THE FINAL INTERVIEW CHEAT SHEET + RAPID FIRE 126
- 16 MOCK INTERVIEW (RUN THIS WITH ME IN CHAT) 136

SECTION 00

HOW TO USE THIS PACK

For: Siva Charan Kollauram Gandla — interview preparation

Time budget: 3-4 hours. Start with this section.

HOW TO USE THIS PACK (read this in 3 minutes)

This pack has 17 sections. **You will not read all of them cover to cover.** Use the schedule below.

Everything is written in the order you should study it. Within each file:

- [FULL] = explanation you must understand
- SAY: = an answer you could literally say out loud (memorize the *shape*, not the words)
- NOTE: = trap / follow-up attack
- [TEST] = self-test question (cover the answer, answer out loud, then check)

Ground rule I am holding you to: you said "assume I know nothing." I am teaching everything from zero, but I am NOT adding technologies outside your resume. The only additions I make are the ones *required* to explain your own claims (Spring, ORM, HTTP, hashing, state machines, reducers, etc.).

PART 1 — RESUME DECOMPOSITION (every claim, ranked)

Resume Item	What an interviewer could ask	Priority	Why
RevertPay V1 (escrow workflow)	Walk me through funding → release. What's escrow? What's a state machine?	P0	Flagship project. 15-25 min of your interview lives here.
JWT authentication (RevertPay)	How does JWT work? Where do you store it? What if token is invalid?	P0	Security is the #1 bluff detector in Java interviews.
Role-based authorization	Difference between authentication and authorization? Show me the roles.	P0	Almost always asked immediately after JWT.
Spring Security	What does it actually do? What is the filter chain?	P0	You put it on the resume; it's the guard on every API.
Spring Boot (all projects)	Why Spring Boot? What is auto-configuration?	P0	Core skill for a Java backend role.
Spring Data JPA + JPA/Hibernate	What is ORM? JPA vs Hibernate? Entity relationships in RevertPay?	P0	You claim entity design — they will ask you to draw it.
PostgreSQL (all projects)	Why Postgres? What is a primary/foreign key? What is a transaction?	P0	Backing store for every claim you make.
SQL	Write a JOIN. Find duplicates. GROUP BY vs HAVING.	P0	They may ask you to write SQL live.
REST APIs	GET vs POST? Status codes? Path vs query param?	P0	The contract between all three projects.
Java	== vs equals, collections, exceptions, static/final.	P0	Base language of RevertPay + EDU2JOB backend.
OOP (4 pillars)	Explain encapsulation/inheritance/polymorphism/abstraction with examples	P0	Guaranteed first-round question.
Transaction handling / @Transactional	What if two requests arrive together? What is a DB transaction?	P0	RevertPay is a money project — atomicity WILL be asked.
EDU2JOB (Infosys internship)	How does Java talk to Python? What is a feature? What is a prediction?	P0	Your internship = real experience bullet, highest credibility.

Resume Item	What an interviewer could ask	Priority	Why
AlgoViz 2.0	How does visualization work? How does pause work?	P0	They will ask "why is this hard?" — you need the reducer/frames answer.
React.js	What is a component? Props vs state? useState/useEffect.	P0	Frontend half of RevertPay, all of AlgoViz.
JavaScript	let/const, map/filter, async/await, promises.	P0	Language under React.
Random Forest + scikit-learn	How does a random forest make a prediction? Why not one tree?	P1	EDU2JOB's brain. Medium-probability but high-damage.
Data preprocessing / encoding	Why can't ML read text categories?	P1	Directly in your resume wording ("Data Preprocessing").
Pandas / NumPy	What's a DataFrame? Why these libraries?	P1	Standard ML follow-up chain.
Python	Why Python for ML? List vs dict.	P1	Cited in internship project.
Flask	What is Flask? Why a separate service?	P1	You claim the Java→Python boundary in EDU2JOB.
React Router	Why route-level structure? What's a route?	P1	AlgoViz claim, easy to answer well.
React hooks (useState/useEffect/useReducer and useRef)	Which hooks did you use and why?	P1	"Hooks" is a standard React round.
Node.js	What is Node? How is it different from Spring Boot?	P1	On resume under Backend. Likely 1–2 questions.
Express.js	What is middleware?	P1	Same as above; be honest about depth.
Git / GitHub	Commit vs push? What's a pull request?	P1	Free marks. Also revision-control workflow questions.
Vite	What is Vite? Why is it fast?	P1	On every frontend project. 30-second answer.
Tailwind CSS	Utility-first CSS — what does that mean?	P1	AlgoViz claim; easy win.
DSA (500+ LeetCode)	Arrays/HashMap/Two pointers/Big-O — you claimed 500+	P1	You invited this claim. Expect one live coding question.

Resume Item	What an interviewer could ask	Priority	Why
HTML / CSS	Box model, flexbox, semantic HTML.	P2	Rarely deep, but free marks.
Deeper SQL (indexes, isolation)	Why is an index fast? What is ACID?	P2	Bonus points if you have time.
Advanced Java (generics, streams, collections internals)	HashMap internals, ArrayList vs LinkedList.	P2	Differentiator if you have spare minutes.
Certifications (CodeTantra, NPTEL)	What did you learn?	P3	One sentence each.
CGPA 8.51	Why not 9+? (rare)	P2	Answer calmly: consistency + projects + LeetCode.

Total claims extracted: 45. Nothing on your resume is left uncovered in this pack.

PART 2 — THE 3.5-HOUR SCHEDULE (follow the clock, not your feelings)

Time	Block	File	Output you must produce
0:00-0:10	Setup + self-intro script	00-START-HERE.md §3	Say your intro out loud 3×
0:10-0:45	Java + OOP	01-java-oops.md	Answer 6 self-tests
0:45-1:20	Spring Boot + REST	02-spring-boot.md, 03-rest-api.md	Draw CLIENT→CONTROLLER→SERVICE→REPOSITORY from memory
1:20-1:55	JPA/Hibernate + PostgreSQL + SQL	04-jpa-hibernate.md, 05-postgresql-sql.md	Draw RevertPay's 4 tables + relations
1:55-2:25	Auth: Spring Security + JWT	06-security-jwt.md	Recite login → token → protected call flow
2:25-2:50	React + JavaScript + AlgoViz engine	07-react-javascript.md	Explain pause/resume with state flow
2:50-3:20	PROJECT DEFENSE (highest value)	12-project-defense.md	Say the 30s/1min/3min versions of all 3 projects
3:20-3:35	ML + Flask + Node/Express (slim)	09-node-express-flask.md, 10-ml-pandas-numpy.md	5 rapid answers
3:35-3:50	Cheat sheet + rapid fire	15-cheatsheet-rapidfire.md	60 rapid-fire answers
3:50-4:00	Mock interview with me (chat)	16-mock-interview.md	Answer, get scored 0-5
last 15 min	Final revision	§"FINAL 15-MINUTE REVISION" in 15-...md	Nothing new — only recall

If a block runs over, **cut P2/P3 content, never cut Project Defense or JWT/Spring.**

Low-value files you may skip if time dies: 08-html-css-tailwind-vite.md (skim §Tailwind+Vite only), 11-git-dsa.md (skim).

§3 — THE SELF-INTRODUCTION (memorize this shape, 45-60 seconds)

"Hi, I'm Siva Charan Kollauram Gandla. I'm a final-year Computer Science student at G. Pullaiah College of Engineering, Kurnool, graduating in May 2027, with a CGPA of 8.51.

I mainly work on Java backend development with Spring Boot. My strongest project is RevertPay — an escrow and payment-reversal platform where I built the escrow workflow as a validated state machine, secured the APIs with JWT and role-based authorization, and persisted users, transactions, escrow records and transaction history in PostgreSQL using Spring Data JPA.

I also completed an Infosys Springboard internship, EDU2JOB, where I built a job-role recommendation system — a Spring Boot backend with REST APIs, a React frontend, PostgreSQL persistence, and a Python Random Forest model that predicts suitable job roles from a student's degree, CGPA and skills.

On the frontend side I built AlgoViz 2.0, a React and Tailwind algorithm visualization platform, where execution is recorded as frames and played back with pause, speed and step controls.

I've also solved 500+ DSA problems on LeetCode. I'm looking for a backend-focused role where I can deepen my understanding of Java, Spring and databases."

Why this works: it names exactly three projects, tells them the stack, and gives them the hooks they'll ask about. Never say "I know everything" or list 20 technologies.

TRAP: "You said final year 2027 — are you available full-time?" → "I'm graduating in May 2027, so I'm looking for an internship or a role that aligns with my final year, and I can be flexible on start date."

§4 — THE 10 MOST DANGEROUS CLAIMS ON YOUR RESUME (ranked)

These are the claims where an interviewer can ask "how?" and expose depth. **If you only fix 10 things tonight, fix these.**

Rank	Claim	Why it's dangerous	Minimum you must know	File
EXTREME 1	"JWT authentication + role-based authorization"	Easy to fake, easy to destroy. They will ask: where's the token stored, what's in it, what if expired.	Header/payload/signature Bearer header, filter validates signature + expiry, roles from token → endpoint rules	06
EXTREME 2	"Escrow workflow: funding → shipping → delivery → release → expiry → review"	Sounds impressive; if you can't define escrow or explain transaction validation, it collapses.	Escrow = third-party holds money until condition met. State machine = legal edges only.	12
EXTREME 3	"Designed entity relationships"	They ask you to draw ER diagram on the spot.	users 1—N transactions, transaction 1—1 escrow, transaction 1—N history	04, 12
EXTREME 4	"Spring Data JPA" + "JPA/Hibernate"	ORM is where beginners die.	ORM = objects ↔ tables. JPA = spec, Hibernate = implementation. Entity → table.	04
HIGH 5	"Random Forest"	They ask why not one tree; if you say "it's more accurate" with no reason, it's over.	Many trees on random subsets, average/vote → less variance/overfitting	10
HIGH 6	"Transactions" (money movement)	"What if two requests arrive together?"	DB transaction = all-or-nothing. @Transactional, rollback, optimistic locking	04, 12

Rank	Claim	Why it's dangerous	Minimum you must know	File
HIGH 7	"500+ LeetCode"	They can hand you a problem right now.	Be able to do an easy array/hashmap problem and talk Big-O	11
HIGH 8	"React + hooks + reusable components"	Fundamental React questions are binary — you either know props/state or you don't.	Component, props vs state, useState, useEffect, keys, lifting state	07
IMPORTANT 9	"Flask + scikit-learn integration"	"Why two runtimes?"	Java owns web/auth/DB; Python owns model. Browser never calls Flask directly.	09, 10
IMPORTANT 10	Portfolio-level claims: <i>double-entry ledger, idempotency key, optimistic locking, isotonic calibration, scheduled expiry</i>	These are in your public case studies but NOT in your resume bullets. If you mention them you own them.	Either learn the one-liners (they're in 12) or don't bring them up.	12, 13

Rule: you are allowed to say "*that was implemented with help of documentation and I can explain the idea, but I'd need to re-read the exact code*" — that's honest and safe. Never invent a class name you don't have.

§5 — THE HONESTY PROTOCOL (Part 34 of your brief)

When you don't know something, use this 3-part answer:

1. **What you do know** (the concept one level above).
2. **What you'd do to find out** (a real engineering action).
3. **Stop talking.** Do not guess.

SAY: Template:

"I haven't worked with that directly — what I do know is [related concept]. In RevertPay the closest thing I handled was [X]. If I needed that feature, I'd [read the docs / add a test / profile it]. I'd rather be honest than guess at the internals."

That answer scores **3-4 out of 5**. Bluffing scores 0 and poisons the rest of the interview.

Never claim: you deployed RevertPay to production, real money moved, you wrote a payment gateway, you have thousands of users, you know Kubernetes/Kafka/Redis/AWS microservices. None of that is on your resume, and none of it is needed.

§6 — WHAT THE COMPANY LIKELY CARES ABOUT

A product company in the subscription/media-tech space — subscription management and media distribution platforms for publishers. For a fresher/Java role, expect:

- Java + OOP fundamentals (heavy)
- Spring Boot + REST + JPA (their stack is Java-heavy)
- SQL / relational thinking (subscriptions, billing, entitlements = tables and joins)

- DSA basics + one coding problem
- Project deep-dive
- Behavioural: learning attitude, teamwork, internship experience

Positioning line to reuse: *"Most of my work is about state and money-like flows — who owns what, what state something is in, and what's allowed to change next. That's exactly the kind of problem subscriptions and billing have."*

That single sentence connects RevertPay to their business. Use it.

Next: open [01-java-oops.md](#).

SECTION 01

01 — JAVA FROM ZERO + OOP

Time: 25-35 min. This is P0. Do not skip.

PART A — WHAT IS JAVA?

What problem does it solve?

Different computers (Windows, Mac, Linux) speak different machine languages. Before Java, you compiled code separately for each one. Java's answer: **compile once into an in-between language (bytecode), and let each machine run it with its own translator (JVM).**

Simple explanation

Java is a programming language plus a runtime. You write `.java` files. A compiler turns them into `.class` files containing **bytecode**. A program called the **JVM** reads the bytecode and runs it. Because the JVM exists on Windows/Mac/Linux, the same `.class` file works everywhere → "Write Once, Run Anywhere."

Real-world analogy

Your speech (Java source) → translated into a universal script (bytecode) → different countries each have a trained interpreter (JVM) who can read the universal script. You write one script, every country understands it.

JDK vs JRE vs JVM (asked constantly)

Thing	What it is	Analogy
JVM	The engine that runs bytecode. Platform-specific.	The engine in a car
JRE	JVM + the standard libraries needed to <i>run</i> Java programs	Engine + wheels + fuel
JDK	JRE + tools to <i>develop</i> (javac compiler, jar, javadoc)	Engine + wheels + fuel + the mechanic's toolbox

SAY: "JDK is for developing, JRE is for running, JVM is the actual runtime engine. I have the JDK installed since I need `javac` and build tools."

Compilation flow

```
Hello.java --javac--> Hello.class (bytecode) --JVM--> runs on your OS
```

SAY: "Java is both compiled and interpreted: it's compiled to bytecode ahead of time, and the JVM interprets/JIT-compiles that bytecode at runtime, which is why it stays fast without being OS-specific."

PART B — CORE LANGUAGE (only what interviewers ask)

Class and object

- **Class** = a blueprint/template.
- **Object** = one real thing built from that blueprint (an "instance").

Analogy: `Car` is the class (design document). Your specific car with plate KA-01-AB-1234 is an object.

```
public class User {
    private String name;           // field / instance variable
    private int age;

    public User(String name, int age) { // constructor: builds the object
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; } // getter (encapsulation)
    public void birthday() { age++; }       // method (behaviour)
}
```

```
User u = new User("Siva", 21); // object created on the heap
u.birthday();
```

SAY: "A class is the type; an object is an instance of that type. `new` allocates it."

Constructor

Special method, same name as the class, no return type, runs once when you `new`. If you don't write one, Java gives a default no-arg constructor. If you write a parameterized one, the default disappears (a classic trick question).

`this`

Refers to the current object — used to disambiguate a field from a parameter with the same name (`this.name = name`).

Variables & data types

- **Primitives** (store the value directly): `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`
- **Reference types** (store an address to an object): `String`, arrays, any class.

TRAP: "`int` vs `Integer`?" → `int` is a primitive with a default 0; `Integer` is a wrapper **object** (needed for generics like `List<Integer>`) with a default `null`. Auto-boxing converts between them.

TRAP: "Size of `int`?" → 32-bit signed, range ± 2.1 billion. `long` is 64-bit. `double` is the default for decimals in Java (writing `3.14` makes a double).

Conditionals / loops (30-second recall, don't over-study)

```
if (x > 0) { } else if (x == 0) { } else { }
for (int i = 0; i < n; i++) { }           // known count
while (cond) { }                         // unknown count
for (String s : list) { }                // for-each, no index needed
switch (day) { case 1: ...; break; default: ... }
```

Arrays

Fixed size, same type, zero-indexed. `int[] a = new int[5];` → defaults to 0s. `a.length` is a **field**, `list.size()` is a **method** (a classic gotcha).

PART C — STRING (guaranteed interview territory)

String immutability

SAY: "In Java a `String` is immutable — once created it can't be changed. Any operation that looks like a change actually creates a new `String`."

Why did the designers do that?

1. **Security** — strings hold passwords, file paths, DB URLs. If another reference could mutate them, that's a vulnerability.
2. **Caching & sharing** — string literals live in a pool; identical literals can share one object.
3. **Safe as HashMap keys** — if a key's content changed, its `hashCode` would change and the map would lose the entry.
4. **Thread safety** — immutable objects can be shared without locks.

Analogy: a printed book vs a whiteboard. To change a printed page you print a new book.

```
String s = "Siva";
s.toUpperCase();    // s is STILL "Siva" – the new string was thrown away
s = s.toUpperCase(); // now s points to a NEW string "SIVA"
```

If you mutate a lot in a loop, use `StringBuilder` (mutable, avoids creating many objects).

`==` vs `.equals()` NOTE: (asked in almost every interview)

- `==` compares **references** for objects (are these the same object?) and **values** for primitives.
- `.equals()` compares **content** (if the class overrides it — `String` does).

```
String a = "hi";
String b = "hi";
String c = new String("hi");

a == b      // true  (both point to the same pooled literal)
a == c      // false (c is a separate object)
a.equals(c) // true  (same characters)
```

SAY: "`==` checks identity, `.equals()` checks logical equality. For strings I always use `.equals()` because I care about the text, not the memory address."

TRAP: "Then why does `a == b` return true?" → Because literal `"hi"` is interned in the **string constant pool**, so both variables point at the same object. `new String(...)` deliberately creates a fresh object outside the pool.

`hashCode()`

The contract: **if two objects are equal, they must have the same hashCode. The reverse is not required** (collisions are allowed).

Why it exists: `HashMap`/`HashSet` use `hashCode` to compute a bucket, then use `equals` to confirm the match inside the bucket. If you override `equals` but not `hashCode`, two "equal" objects can land in different buckets and `map.get(key)` fails.

SAY: "`equals` decides equality; `hashCode` is a fast bucket index. Break that contract and hash-based collections stop finding your keys."

PART D — EXCEPTION HANDLING

Simple explanation

When something goes wrong, Java **throws** an exception object instead of returning garbage. The exception travels up the call stack until somebody **catches** it. If nobody catches it, the program crashes with a stack trace.

Analogy: a parcel you can't deliver is returned to the sender, with a note saying what was wrong, instead of being silently thrown away.

```
Throwable
├── Error          (JVM-level, don't handle: OutOfMemoryError)
└── Exception
    ├── Checked    (compiler forces you to handle: IOException, SQLException)
    └── RuntimeException / Unchecked (optional to handle: NullPointerException,
                                     ArithmeticException, IllegalArgumentException)
```

```
try {
    int r = 10 / x;           // risky code
} catch (ArithmeticException e) {
    log.error("bad divisor", e); // handle it
} finally {
    conn.close();             // ALWAYS runs (cleanup)
}
```

SAY: "Checked exceptions must be declared or handled at compile time; unchecked ones are programming bugs that happen at runtime. `finally` always runs, which is why cleanup goes there — and try-with-resources is the modern way to close files and connections."

TRAP: "Can `finally` be skipped?" → Yes, if the JVM exits (`System.exit`) or the thread dies. Otherwise it always runs, even if the try block returns.

How you connect it to your projects

SAY: "In Spring Boot I don't catch everything in every method. I handle expected business errors by throwing a custom exception from the service layer and converting it to an HTTP response in a global `@RestControllerAdvice` handler — for example an invalid escrow state transition returns a 409 instead of a 500."

PART E — COLLECTIONS (P0 — you claim Java on 3 projects)

The problem collections solve

Arrays are fixed-size and clumsy. Collections are ready-made data structures with the operations you actually need (add, remove, search, sort).

```
Iterable
├── Collection
│   ├── List    → ordered, allows duplicates    → ArrayList, LinkedList
│   ├── Set     → no duplicates                  → HashSet, TreeSet, LinkedHashSet
│   └── Queue   → FIFO processing                → ArrayDeque, PriorityQueue
Map is separate → key → value pairs              → HashMap, TreeMap, LinkedHashMap
```

(**Map is not a Collection** — classic trick question. It's a separate interface.)

List vs Set vs Map

	List	Set	Map
Duplicates	Yes	No	Keys: no, Values: yes
Order	Insertion order	HashSet: none; LinkedHashSet: insertion; TreeSet: sorted	HashMap: none
Access	By index <code>get(0)</code>	No index	By key <code>get(k)</code>
Analogy	A queue of people	A guest list (no repeats)	A dictionary / phone book

ArrayList

Backed by a **resizable array**. Fast `get(index)` → **O(1)**. Appending is amortised O(1) (it doubles capacity when full). Inserting/removing in the middle is **O(n)** because everything shifts.

```
List<String> roles = new ArrayList<>();
roles.add("ADMIN");
roles.add("SELLER");
roles.get(0);           // "ADMIN"
roles.contains("BUYER"); // O(n)
roles.size();
```

```
List<String> immutable = List.of("BUYER", "SELLER"); // modern, unmodifiable
```

ArrayList vs LinkedList: "LinkedList is O(1) to insert at the head but has poor cache locality and O(n) indexing, so ArrayList is the default choice in practice." (Saying this alone signals you know Java.)

HashSet / HashMap (very high yield)

- **HashSet** = a **HashMap** where you only care about keys. Add is O(1) average, O(n) worst case if everything collides.
- **HashMap** = array of **buckets**; `hash(key)` picks the bucket; collisions become a linked list which converts to a **red-black tree** after 8 entries in one bucket (Java 8+); resizes when load factor > 0.75.
- **TreeMap** keeps keys sorted by natural order or a **Comparator** — O(log n).

SAY: "HashMap gives average O(1) lookup by hashing the key into a bucket. HashSet is basically the same structure where only the keys matter, used to track 'have I seen this before?'"

Generics

Problem: a list without generics accepts anything and hands back **Object**, forcing casts that can fail at runtime.

```
List list = new ArrayList();
list.add("text"); list.add(42);           // legal, dangerous
String s = (String) list.get(1);         // ClassCastException at RUNTIME

List<String> safe = new ArrayList<>();
safe.add(42);                           // compile-time ERROR ← the point
```

SAY: "Generics move type errors from runtime to compile time, and remove manual casting. `<T>` is a type parameter filled in when you use the class."

PART F — `static`, `final`, ACCESS MODIFIERS

`static`

Belongs to the **class**, not to any instance. One copy shared by everyone.

```
class Counter {
    static int total = 0;    // shared across all Counter objects
    int id;                 // per-object
    Counter() { total++; id = total; }
}
```

Use for: constants (`static final double PI`), utility methods (`Math.max`), factory methods, `main`. TRAP: "Can a static method use an instance variable?" → No. There's no object, so there's no `this`.

`final`

- `final variable` → can be assigned once.
- `final method` → cannot be overridden.
- `final class` → cannot be extended (e.g. `String`). TRAP: "Is a final list immutable?" → No. `final` freezes the *reference*, not the contents: you can still `add()`, you just can't reassign the variable.

Access modifiers

Modifier	Same class	Same package	Subclass	Everywhere
<code>private</code>	[YES]	[NO]	[NO]	[NO]
<i>(default)</i>	[YES]	[YES]	[NO]	[NO]
<code>protected</code>	[YES]	[YES]	[YES]	[NO]
<code>public</code>	[YES]	[YES]	[YES]	[YES]

SAY: "I default to `private` fields with public getters/setters — that's encapsulation. Only the API surface is public."

PART G — INTERFACE vs ABSTRACT CLASS

Interface

A **contract**: what a class can do, with no state. Methods are public abstract by default; fields are `public static final`.

```
public interface PaymentMethod {
    void pay(double amount);    // no body
    default String describe() { return "generic payment"; } // Java 8 default method
}
```

Abstract class

A partially built class: may hold state + constructor + some implemented methods, some abstract ones.

```
public abstract class LedgerEntry {
    protected Long id;          // shared state
    public abstract double signedAmount(); // subclasses must define
    public boolean isCredit() { return signedAmount() > 0; } // shared behaviour
}
```

```
}

```

	Interface	Abstract class
State / fields	Only constants	Yes, instance fields allowed
Constructor	No	Yes
Multiple inheritance	Yes — a class can implement many	No — only one parent
When to use	"Can do X" capability	"Is a kind of X" + shared code

SAY: "An interface says what something can do; an abstract class lets me share code and state with a family of related classes. Since Java allows a class only one parent, interfaces are how I compose capabilities."

Where you actually used this (say this if asked): "In Spring, `JpaRepository` is an interface — I extend it to get CRUD methods without writing any implementation. Spring generates the implementation at runtime. And my escrow service is written against an interface so the controller only depends on the contract."

PART H — THE FOUR PILLARS OF OOP

"What is OOP?"

SAY: "Object-Oriented Programming organises code around objects that hold both data and the behaviour that acts on that data, rather than around functions and global variables. It gives me four tools: encapsulation, abstraction, inheritance and polymorphism, and the goal is code that's easier to change without breaking."

Why it matters: software changes constantly. OOP limits the blast radius of a change. Change the internals of `EscrowService` and no controller needs to be touched.

1. Encapsulation [FULL]

Idea: hide the internals; expose a controlled interface. Fields `private`, access through methods that can validate.

Analogy: an ATM. You insert a card and get money; you can't reach into the cash drawer. The machine decides what operations are legal.

```
public class Escrow {
    private BigDecimal amount;
    private EscrowStatus status = EscrowStatus.CREATED;

    public void setStatus(EscrowStatus next) {
        if (!status.canMoveTo(next)) throw new InvalidEscrowTransition(...);
        this.status = next;
    }
    public BigDecimal getAmount() { return amount; }
}
```

Why useful: no other class can put the escrow in an impossible state. **This is literally your RevertPay design decision.**

TRAP: "Is encapsulation just getters and setters?" → No. Getters/setters without logic are *exposing* data, not hiding it. Real encapsulation is controlling which changes are legal — exactly the `setStatus` validation above, or a derived balance that has no setter at all.

2. Abstraction [FULL]

Idea: show what something does; hide how. You don't care how `save()` works, just that it persists.

Analogy: driving a car — you use the steering wheel and pedals, not the fuel-injection timing.

```
public interface JobPredictor {
    List<RolePrediction> predict(StudentProfile profile);
}
// Caller doesn't know if this is HTTP-to-Flask, a local model, or a stub in a test.
```

Encapsulation vs abstraction: encapsulation is about *restricting access to internals*; abstraction is about *hiding complexity behind a simple contract*. Encapsulation hides data; abstraction hides implementation.

3. Inheritance [FULL]

Idea: a child class reuses/extends a parent (`class Child extends Parent`). It represents "is-a".

Analogy: a biology taxonomy — a Sparrow *is a* Bird, so it inherits "has wings, lays eggs" and adds "sings".

```
public abstract class BaseEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Instant createdAt = Instant.now();
}
public class Transaction extends BaseEntity { ... } // gets id + createdAt free
```

Why useful: no duplicated common fields; one place to change auditing.

TRAP: "Any downside?" → Yes. Deep hierarchies are rigid: change the parent and you risk every child, and Java forces single inheritance. **Prefer composition over inheritance** — see below.

4. Polymorphism [FULL]

Idea: one name, many forms — the same call does the right thing for the actual object.

- **Compile-time (overloading):** same method name, different parameters.
- **Runtime (overriding):** the subclass's version runs, decided at runtime via dynamic dispatch.

Analogy: the command "play" — a phone plays audio, a video player plays video. Same word, correct behaviour for the object.

```
// Overloading (compile-time)
void pay(BigDecimal amount) {}
void pay(BigDecimal amount, String idempotencyKey) {}

// Overriding (runtime)
abstract class Report { abstract String render(); }
class PdfReport extends Report { String render() { return "PDF"; } }
class CsvReport extends Report { String render() { return "CSV"; } }

Report r = new PdfReport();
r.render(); // "PDF" — decided at runtime, not by the variable type
```

SAY: "Polymorphism means the caller writes one line against the parent type, and each subclass supplies its own behaviour. It's what lets me add a new report type without editing existing code."

TRAP: "Overloading vs overriding?" → Overloading: same class, same name, different signature, resolved at compile time. Overriding: subclass replaces the parent's method, same signature, resolved at runtime with `@Override`.

Inheritance vs composition NOTE: (advanced-sounding, easy to answer)

SAY: "Inheritance is an *is-a* relationship; composition is a *has-a*. I prefer composition when the relationship isn't truly a subtype, because it avoids fragile deep hierarchies and lets me change behaviour by swapping an object — which is also how Spring's dependency injection works: my service *has a* repository, it isn't a kind of repository."

PART I — HOW OOP APPEARS IN YOUR SPRING PROJECT (they WILL ask this)

SAY: **Say this, it's the highest-value OOP answer you can give:**

"Almost everything in Spring is OOP applied: - **Encapsulation** — my entities keep their fields private and I only allow legal state changes, e.g. my escrow status can only move along allowed transitions; the balance is derived from ledger entries rather than being a settable field. - **Abstraction** — the controller talks to a service interface; it doesn't know about SQL, JPA or transactions. - **Inheritance** — every repository extends `JpaRepository`, every entity extends a base class that supplies `id`, `createdAt`, and my custom exceptions extend `RuntimeException`. - **Polymorphism** — `JpaRepository` is an interface with many implementations, and Spring picks the right one at runtime; overriding `equals/hashCode` on entities is also polymorphism. So OOP is not just a theory I studied — it's the structure of the framework I build on."

SELF-TEST (answer out loud before reading on)

1. Difference between JDK, JRE and JVM?
2. Why is `String` immutable, and name two benefits.
3. What's the difference between `==` and `.equals()`?
4. If I override `equals()` but not `hashCode()`, what breaks and why?
5. `ArrayList` vs `HashSet` vs `HashMap` — one line each.
6. Explain the four pillars using something from `RevertPay`.
7. Interface vs abstract class — pick one and justify it.
8. What does `@Transactional`-style OOP thinking mean for an escrow status change?

Answer key (short)

1. JVM runs bytecode; JRE = JVM + libraries; JDK = JRE + dev tools (javac).
2. Security, string pooling/caching, safe map keys, thread safety.
3. `==` identity/reference; `.equals()` content/equality.
4. Hash-based collections (`HashMap/Set`) can't find equal keys because they hash into different buckets.
5. `ArrayList` = ordered indexable list ($O(1)$ get); `HashSet` = unique elements, $O(1)$ contains; `HashMap` = key→value, $O(1)$ average lookup.
6. Encapsulation = controlled escrow transitions; abstraction = service interface over JPA; inheritance = `JpaRepository` / base entity; polymorphism = repository interface with generated implementation.
7. Interface for capability + multiple contracts; abstract class when sharing state/code.
8. The status change is a method that validates the transition, so no code path can bypass the rules — and it happens inside a transaction with the ledger write.

If you got 5/8, move on. If fewer, reread Parts H and I — they carry the most interview weight.

Next: [02-spring-boot.md](#)

SECTION 02

02 — SPRING BOOT FROM ABSOLUTE ZERO

Time: 20 min. This is P0 — you claim it on 2 of 3 projects.

STEP 0 — THE WORDS PEOPLE USE WITHOUT EXPLAINING

What is a backend? What is a server?

- **Frontend** = what the user sees and clicks (React, HTML/CSS). Runs in the browser.
- **Backend** = the code that the user can't see: it holds the rules ("can this escrow be released?"), talks to the database, and decides what data is allowed out. It runs on a machine somewhere else.
- **Server** = a program that listens on a port, waits for requests, and sends back responses. (Also used to mean "the machine it runs on" — context decides.)
- **Client** = whoever makes the request (typically the browser / React app).

Analogy: a restaurant. The menu and dining room are the frontend. The kitchen is the backend. The waiter is the API — the agreed way to ask for things. You never walk into the kitchen yourself.

What is an API?

API = Application Programming Interface = an agreed contract for how one program asks another program to do something. It defines: the address, the inputs, the outputs, and the error cases. **A REST API is an API that follows REST conventions over HTTP.**

What is a framework vs a library?

- **Library:** you call it. You're in charge of the flow. (Pandas, Jackson)
- **Framework:** it calls you. It controls the flow and your code plugs into defined points (**inversion of control**).

Analogy: a library is a toolbox; a framework is a house frame with sockets where your code slots in. Spring Boot is a framework.

STEP 1 — WHAT IS SPRING?

The problem: building a Java web application by hand is painful. You'd have to create objects in the right order, wire everything together (`new UserService(new UserRepository(...))`), open and close connections, map URLs to code, parse JSON, manage transactions, and read config files. Thousands of lines of plumbing before your first business rule.

Spring's answer: *don't create and wire objects yourself — describe them, and let the container create and inject them.* That is **Inversion of Control**, and the mechanism is **Dependency Injection**.

SAY: "Spring is a Java framework whose core idea is that an object shouldn't build its own dependencies. You declare components, Spring's IoC container creates them and injects what they need, so your classes stay focused on business logic."

STEP 2 — WHAT IS SPRING BOOT?

SAY: **Say it like this (natural, not textbook):**

"Spring Boot is basically a way to build Java backend applications without manually configuring a huge amount of Spring infrastructure. It gives me automatic configuration and an embedded server — Tomcat is already inside the application — so I can run it as a normal Java program with `main()` and focus on my code instead of XML and deployment setup."

Boot adds over plain Spring:

1. **Auto-configuration** — sees `spring-boot-starter-data-jpa` on the classpath and configures a `DataSource`, an `EntityManagerFactory`, and transaction management for you.
2. **Starters** — one dependency pulls a consistent set of compatible libraries (`spring-boot-starter-web`, `spring-boot-starter-security`, `spring-boot-starter-data-jpa`).
3. **Embedded server** — Tomcat ships inside the jar. No external server to install. `java -jar app.jar` and it's running.
4. **Opinionated defaults + config** — `application.properties/application.yml` instead of reams of XML.
5. **Production features** — Actuator health checks, externalised config, profiles.

SAY: "Why Spring Boot over plain Spring/Java EE?"

"Boot removes the boilerplate. With plain Spring I'd configure the dispatcher, the data source and the transaction manager manually. Boot's convention-over-configuration means the defaults are already right, and I only override what I need in `application.properties`. For a solo project on a deadline, that's a big difference, and it's also the industry standard, so there's a large ecosystem and docs."

TRAP: "Is auto-configuration magic?" → "It's conditional: auto-configurations are applied only when certain classes or properties are present — that's `@ConditionalOnClass`, `@ConditionalOnMissingBean` and similar. If I define my own `DataSource`, Boot backs off and uses mine."

STEP 3 — IoC CONTAINER, DI, BEANS**Bean**

A **bean** is simply an object that the Spring container creates and manages (= a Spring-managed object).

IoC (Inversion of Control)

Normally **you** control object creation. With IoC the **container** controls it — it decides when to instantiate, what to inject, and when to destroy.

Dependency Injection

The container *provides* an object's dependencies from the outside, usually via the constructor.

```
// WITHOUT DI – tightly coupled, hard to test
public class EscrowService {
    private TransactionRepository repo = new TransactionRepositoryImpl(); // hard-coded
}

// WITH DI – the container hands it in
@Service
public class EscrowService {
    private final TransactionRepository repo;

    public EscrowService(TransactionRepository repo) { // constructor injection
        this.repo = repo;
    }
}
```

Why this is better:

- **Testable** — in a unit test I pass a fake/mock repository. No database needed.
- **Loose coupling** — the service depends on an interface, not a concrete class.
- **Single responsibility** — the service doesn't know how to build a repository.
- **Lifecycle managed** — one shared instance (singleton by default), not one per request.

SAY: "Why constructor injection over field injection (@Autowired on a field)?"

"Constructor injection makes dependencies explicit and mandatory, lets me mark the field `final` so it can't change, and makes the class testable with plain `new` — no Spring context required in unit tests. Field injection hides the dependencies and needs reflection to test."

Stereotype annotations (what they mean, not just what they do)

Annotation	Meaning	Layer
@Component	generic Spring-managed bean	any
@Service	business logic	service
@Repository	data access; also translates DB exceptions into Spring's <code>DataAccessException</code>	persistence
@Controller / @RestController	handles HTTP requests; <code>@RestController = @Controller + @ResponseBody</code> , so the returned object is serialised to JSON automatically	web
@Configuration + @Bean	"this method's return value is a bean I want managed" — for third-party classes you can't annotate	config

All of them are specialisations of `@Component`; they're semantically meaningful and enable extra behaviour. That's how component scanning finds them: `@SpringBootApplication` includes `@ComponentScan`, which scans the package of the main class and below.

Component scanning / classpath

- **Classpath** = the set of compiled classes and jars the JVM can load. Boot auto-configures based on what's on it.
- **Component scan** = Boot walks your packages, finds annotated classes, and registers them as beans.

@Autowired and how injection actually happens

1. `SpringApplication.run()` starts the container.
2. Component scan finds `@Service`, `@RestController`, `@Repository` classes.
3. For each, Spring finds a constructor and resolves each parameter by type from the container → dependency graph built.
4. Singletons are created once and cached.
5. The embedded Tomcat starts and the app listens on port 8080.

TRAP: "What if two beans are the same type?" → Spring throws `NoUniqueBeanDefinitionException`, unless you disambiguate with `@Primary` or `@Qualifier("name")`.

@SpringBootApplication and SpringApplication.run()

`@SpringBootApplication = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan`.

```
@SpringBootApplication
public class RevertPayApplication {
    public static void main(String[] args) {
        SpringApplication.run(RevertPayApplication.class, args);
    }
}
```

```
}
}
```

SAY: "main() is the entry point; `SpringApplication.run` boots the container, runs auto-configuration and starts the embedded server. I don't call the server — the framework does."

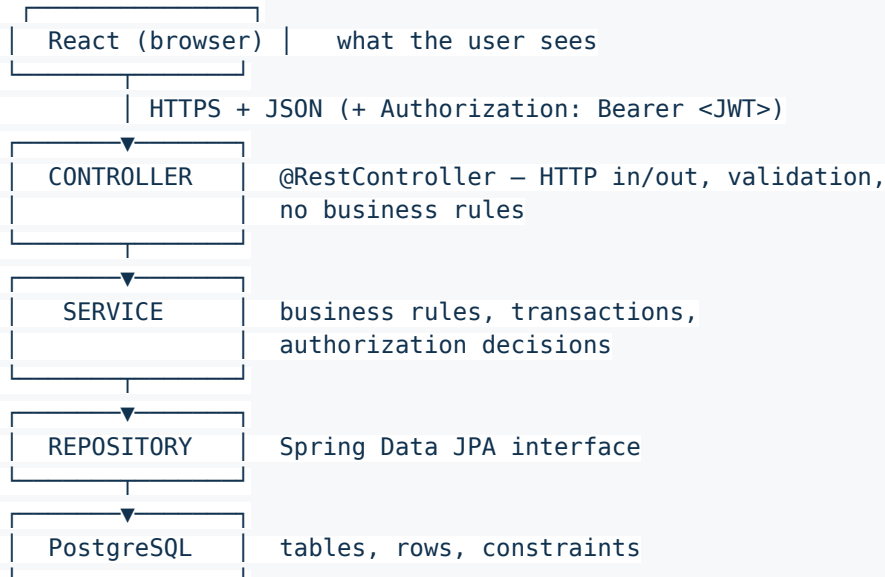
application.properties

```
server.port=8081
spring.datasource.url=jdbc:postgresql://localhost:5432/revertpay
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
jwt.secret=${JWT_SECRET}
jwt.expiration-ms=86400000
```

SAY: "This is externalised configuration — ports, datasource, JPA behaviour, JWT secret and expiry. I keep secrets in environment variables rather than committing them. Spring profiles let me have different values for dev and prod."

Bean scopes: default is **singleton** (one instance for the whole container). **prototype** = new instance per request. Web scopes: **request**, **session**. NOTE: Consequence: since services are singletons, they must be **stateless** — no per-request data in fields. That's why a service takes parameters instead of storing the current user in a field. (Excellent answer to give.)

STEP 4 — THE ARCHITECTURE (be able to draw this in 20 seconds)



Every box explained:

- **Controller** — the door. Maps HTTP method + path to a Java method, reads JSON into an object, validates input, returns a status code + JSON. Should contain **no** business logic.
- **Service** — the brain and the transaction boundary. Enforces the rules ("you can only release funds from DELIVERED"), checks ownership, calls the repository, decides what happens on failure.
- **Repository** — the persistence contract. An interface extending `JpaRepository`; Spring Data generates the implementation and the SQL.
- **Entity** — a Java class mapped to a table.
- **Database** — durable storage with constraints and transactions.

- **DTO** — the object shape that crosses the wire (request/response), kept separate from entities so I never leak password hashes or internal fields and so the API doesn't break when a column changes.

SAY: "Why layers, not one big class?" → "Separation of concerns. I can change the database implementation without touching business rules, test the service without HTTP, and change the API shape without changing the domain. It's also the standard Spring structure, so any Java developer can navigate my code."

STEP 5 — WHAT HAPPENS ON `POST /api/login` (walk through it; they love this)

1. **React** sends `fetch("POST /api/auth/login", body: {email, password})` with `Content-Type: application/json`.
2. **DispatcherServlet** (Spring MVC's front controller) receives it and matches it to a handler method using the method + path.
3. **Security filter chain** runs *before* the controller. For `/api/auth/login` we permit it without a token; for a protected path it would check the `Authorization: Bearer <jwt>` header and reject with 401/403 if invalid.
4. **Controller** method `login(@Valid @RequestBody LoginRequest req)` runs. Jackson converts the JSON into a Java object; `@Valid` runs bean validation (`@Email`, `@NotBlank`) → failure gives 400.
5. **Service** `authService.login(...)` loads the user through the repository.
6. **Repository** → Hibernate generates SQL → **PostgreSQL** returns the row.
7. **Password check:** BCrypt compares the submitted password's hash with the stored hash. Never decrypt — you compare hashes.
8. On success the service builds a **JWT** signed with our secret (subject = user id, roles claim, expiry).
9. **Response 200 OK** with `{ token, user: {id, name, roles} }` — serialised to JSON by Jackson automatically because of `@RestController`.
10. If credentials are wrong → **401 Unauthorized** with a generic message ("Invalid email or password") so we don't reveal which part was wrong.

Then the frontend stores the token (in this project: browser storage) and attaches it as `Authorization: Bearer <token>` on later requests.

TRAP: "Where's the user's password compared — in the DB?" → "No. BCrypt is one-way; the database only stores the hash and the salt. I hash the submitted password and compare with `passwordEncoder.matches(raw, stored)`."

STEP 6 — ANNOTATIONS YOU CLAIM (one line each)

Annotation	What it does
<code>@SpringBootApplication</code>	main entry: auto-config + component scan
<code>@RestController</code>	HTTP handler returning JSON bodies
<code>@RequestMapping("/api/transactions")</code>	base path for the class
<code>@GetMapping/@PostMapping/@PutMapping/@PatchMapping/@DeleteMapping</code>	bind HTTP method + path to a method
<code>@PathVariable</code>	value from the URL path, e.g. <code>/transactions/{id}</code>
<code>@RequestParam</code>	value from the query string, <code>?page=0&size=10</code>
<code>@RequestBody</code>	deserialise the JSON body into an object
<code>@Valid</code>	run Bean Validation on the argument
<code>@Service/@Repository/@Component</code>	mark a bean and its role
<code>@Autowired</code>	ask the container to inject (constructors don't even need it)
<code>@Configuration + @Bean</code>	declare beans for classes I don't own (e.g. <code>PasswordEncoder</code> , <code>SecurityFilterChain</code>)
<code>@Value("\${jwt.expiration-ms}")</code>	inject a config property
<code>@Transactional</code>	run this method in one DB transaction
<code>@ControllerAdvice/@RestControllerAdvice + @ExceptionHandler</code>	global exception → HTTP status mapping
<code>@Entity, @Table, @Id, @GeneratedValue, @Column</code>	JPA mapping (see file 04)

SAY: If asked "which annotations did you use most?" → "`@RestController` + `@RequestMapping` on the web layer, `@Service` with `@Transactional` on the business layer, `@Repository/JpaRepository` for persistence, and `@Entity` for the model. Plus `@Valid` for input validation."

STEP 7 — HOW YOU USED SPRING BOOT (say this, adapted per project)

RevertPay:

"RevertPay's backend is a Spring Boot application. Controllers expose the escrow endpoints; a transactional service layer holds the state machine and writes the money movements; repositories are Spring Data JPA interfaces over PostgreSQL; Spring Security with JWT guards every endpoint; and cross-cutting concerns like validation and exception-to-status mapping live in a global handler so controllers stay thin."

EDU2JOB:

"The Spring Boot application owns the API, authentication and persistence. When a prediction is requested, the service calls the Python Flask model service over HTTP, stores the inputs and the raw response as prediction history in PostgreSQL, and returns the ranked roles to React."

SELF-TEST

1. What is the difference between a library and a framework?
2. What is a bean? What is IoC?
3. Why use Spring Boot instead of plain Spring? (3 reasons)
4. Why constructor injection? (3 reasons)
5. Name the four layers and one responsibility each.
6. What does `@RestController` do that `@Controller` doesn't?
7. Why must a `@Service` be stateless?
8. What happens if two beans of the same type exist?

Key

1. Framework controls the flow and calls you (IoC); library is called by you.
2. A Spring-managed object; IoC = the container, not your code, controls creation and wiring.
3. Auto-configuration, starters, embedded server (+ externalised config, ecosystem).
4. Explicit + final + testable without Spring.
5. Controller = HTTP; Service = rules + transactions; Repository = persistence; DB = storage.
6. Serialises returned objects to the response body (JSON) automatically.
7. Beans are singletons, so request data in a field would leak across users / race conditions.
8. `NoUniqueBeanDefinitionException` unless `@Primary` or `@Qualifier`.

Next: `03-rest-api.md`

SECTION 03

03 — REST APIs, HTTP, JSON

Time: 15 min. This is P0 — it's the contract between all three of your projects.

1 — WHAT IS AN API? (again, because everyone asks)

API = Application Programming Interface — an agreed contract for how two programs talk. It defines *what you can ask for*, *what you must send*, and *what you'll get back*, without exposing how it's implemented.

Analogy: the waiter in a restaurant. You don't shout into the kitchen; you use the menu, the waiter takes your order in a known format, and you get food back. If the kitchen changes chefs, you still order the same way.

2 — WHAT IS REST?

SAY: "REST is a set of conventions for building APIs over HTTP. The main ideas: everything is a **resource** identified by a URL; you act on it with standard HTTP **methods**; each request is **stateless** — the server doesn't remember you between requests; and the response is a **representation** of the resource, usually JSON."

Six constraints (know the first three properly):

1. **Client-server** — UI and data are separate, so each can change independently.
2. **Stateless** — every request carries everything needed (including the token). No server-side session. That's exactly *why* JWT fits: the identity travels in the token instead of in a session map.
3. **Cacheable** — responses can declare themselves cacheable.
4. **Uniform interface** — same pattern for every resource.
5. **Layered system** — proxies/gateways can sit in between.
6. **Code on demand** (optional).

NOTE: "Is REST a standard/protocol?" → "No, it's an architectural **style**. Practically, most APIs — including mine — are REST-ish: resources + HTTP verbs + status codes + JSON, with proper URIs."

3 — HTTP IN ONE PAGE

HTTP = HyperText Transfer Protocol — the request/response protocol of the web.

Anatomy of a request

```
POST /api/transactions/42/release HTTP/1.1    ← method + path + version
Host: api.revertpay.local                     ← headers
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs... ← the JWT
Accept: application/json
                                             ← blank line
{ "note": "buyer confirmed delivery" }         ← body (optional)
```

Anatomy of a response

```
HTTP/1.1 200 OK                               ← status line
Content-Type: application/json
                                             ← blank line
{ "id": 42, "status": "RELEASED", "amount": 1500.00 }
```

Parts defined:

- **URL / URI** — the address of the resource. `https://host/api/transactions/42`
- **Endpoint** — a specific method + path combination that the server handles: `GET /api/transactions/{id}`.
- **Path variable** — identifies *which* resource: `/transactions/{id}` → `@PathVariable Long id`.
- **Query parameter** — filters/pagination/options, doesn't identify the resource: `/transactions?status=DELIVERED&page=0&size=20` → `@RequestParam`.
- **Headers** — metadata: `Authorization`, `Content-Type`, `Accept`, `Cache-Control`.
- **Body** — the payload, usually JSON.
- **JSON** — JavaScript Object Notation: `{"key": "value", "list": [1,2], "nested": {"a": true}}`. Human-readable, language-neutral. (Server side Jackson maps it to Java objects.)

SAY: "Path variable decides *which* thing; query parameter *shapes* the result."

The methods (and idempotency — say this word)

Method	Purpose	Safe?	Idempotent?
GET	read a resource (never changes state)	Yes	Yes
POST	create something / trigger an action	No	No
PUT	full replace of a resource	No	Yes
PATCH	partial update	No	Not necessarily
DELETE	remove a resource	No	Yes

Idempotent = doing it twice has the same effect as doing it once.

SAY: "GET must never change state. PUT is idempotent — sending the same full update twice leaves the same result. POST is not, which is exactly why money operations need an idempotency key: a retried POST could otherwise create two payments." (This sentence alone makes you sound senior. It's true of your RevertPay case study — see file 12.)

TRAP: "Why not use GET for deleting with `?delete=1`?" → "Because GET is supposed to be safe and cacheable; crawlers and prefetching could trigger it. The method should reflect intent."

Status codes (memorize the classes and these specifics)

Code	Meaning	When YOU return it
200	OK	successful GET / update with a body
201	Created	after a POST that creates a resource (add <code>Location</code> header)
204	No Content	successful DELETE or update with nothing to return
400	Bad Request	validation failure, malformed JSON
401	Unauthorized	not authenticated — missing/invalid/expired token
403	Forbidden	authenticated but not allowed (wrong role / not the owner)
404	Not Found	the resource doesn't exist
405	Method Not Allowed	path exists but not for this verb
409	Conflict	business-state clash: duplicate email, illegal state transition, optimistic-lock failure
422	Unprocessable Entity	request understood but semantically invalid (some teams use instead of 400)
500	Internal Server Error	unexpected server bug — never for a user's bad input
503	Service Unavailable	dependency down / overloaded (e.g. Flask model service unreachable)

Rule of thumb: 4xx = the client's problem, 5xx = the server's problem.

TRAP: "401 vs 403?" → "401 means *we don't know who you are* — authenticate. 403 means *we know who you are, and you still can't do this*." TRAP: "What's the difference between 400 and 409 for a duplicate email?" → "400 is a malformed/invalid request in general; 409 is specifically a conflict with the current state of the server's data. I'd use 409 for a duplicate registration or an invalid escrow transition."

4 — AUTHENTICATION vs AUTHORIZATION (they will ask; get the sentence exact)

SAY: "**Authentication** is *proving who you are* — that's login, and it's what the JWT does. **Authorization** is *what you're allowed to do* once known — that's roles and ownership checks. Authentication happens first; authorization happens on every protected request."

Analogy: the airport. Your passport authenticates you (you are who you say). Your boarding pass authorizes you (you may board *this* flight, not any flight).

Statelessness matters here: with classic sessions the server stores "user 42 is logged in" in memory and gives you a session id cookie. With JWT the server stores nothing and re-derives your identity from the signed token on every request. That scales horizontally and keeps the API stateless — which is a REST constraint.

TRAP: "If JWT is stateless, how do you log out?" → "Technically the token stays valid until it expires. Options are short-lived access tokens plus a refresh token, a server-side revocation/blacklist list, or a token version stored on the user. In RevertPay the pragmatic approach is short expiry; server-side revocation is a known limitation I'd add next."

5 — CONNECTING TO YOUR PROJECTS

EDU2JOB REST APIs (resume bullet 2)

"I built the Spring Boot backend and exposed REST endpoints for four things: authentication (register/login returning a JWT), profile management (read/update the student's education, CGPA and skills), job prediction (POST the profile data, get back the ranked roles), and prediction history (GET the user's previous predictions)."

Method + path (illustrative)	Purpose	Success
POST /api/auth/register	create a user	201
POST /api/auth/login	verify credentials, return JWT	200
GET /api/profile/me	read my profile	200
PUT /api/profile/me	update degree/major/CGPA/skills	200
POST /api/predictions	run the model for my profile	201 (stored in history)
GET /api/predictions	my prediction history (paged)	200

SAY: "The browser never calls the Python service directly — it only knows about Spring Boot. Spring owns the contract, the auth and the persistence."

RevertPay REST APIs (resume bullet 2)

Method + path	Purpose	Rules
POST /api/escrows	buyer creates an escrow and funds it	BUYER role, idempotency key
GET /api/transactions/{id}	view a transaction + escrow state	owner or admin
POST /api/escrows/{id}/ship	seller marks shipped	SELLER + owner + state = FUNDED
POST /api/escrows/{id}/deliver	buyer confirms delivery	BUYER + owner + state = SHIPPED
POST /api/escrows/{id}/release	release funds to seller	state = DELIVERED
POST /api/escrows/{id}/dispute	move to review	either party
POST /api/escrows/{id}/resolve	release or refund after review	ADMIN
GET /api/transactions/{id}/history	audit trail	owner or admin

SAY: "Actions are state transitions, so I modelled release/ship/deliver as POST endpoints on the escrow resource rather than arbitrary PUT updates on a status field — the server decides the next state, the client can't just post `status: RELEASED`."

TRAP: "Why POST and not PUT for release?" → "Because release isn't a full replacement of the resource and it isn't idempotent in the naive sense — it triggers an action. POST models commands; PUT models replacement. If I needed retry-safety I'd add an idempotency key."

SAY: "What REST APIs did YOU build?" — natural answer

"In RevertPay I built the escrow lifecycle endpoints — create and fund, ship, deliver, release, dispute and review — plus authentication, and a history endpoint that returns the audit trail for a transaction. Each one is a controller method that validates the request, delegates to a transactional service, and maps to a proper status code — 201 on creation, 409 for an illegal state transition or duplicate, 403 when you're logged in but it isn't your transaction. In EDU2JOB I built auth, profile, prediction, and prediction-history endpoints, where the prediction endpoint is the one that crosses into the Python service."

6 — ERROR HANDLING SHAPE (mention this; it signals real implementation)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(InvalidEscrowTransitionException.class)
    ResponseEntity<ApiError> handleInvalidTransition(InvalidEscrowTransitionException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT) // 409
            .body(new ApiError("INVALID_TRANSITION", ex.getMessage()));
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<ApiError> notFound(ResourceNotFoundException ex) { /* 404 */ ... }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ApiError> invalid(MethodArgumentNotValidException ex) { /* 400 */ ... }
}
```

SAY: "I return a consistent error body — a code, a message, and the timestamp — instead of leaking stack traces. Expected business failures are custom exceptions mapped in a `@RestControllerAdvice`; anything unexpected becomes a 500 with the details logged server-side but not returned to the client."

7 — NATURAL ANSWER BUNDLE (memorize the shape)

"What is REST?"

"It's a style for HTTP APIs: resources with URLs, standard methods, stateless requests, and JSON representations. I follow it in both my backend projects."

"GET vs POST?"

"GET reads and must not change anything; POST creates or triggers something and carries a body. GET is safe and repeatable, POST isn't, so POST is where I care about retries."

"What's statelessness?"

"The server keeps no memory of me between requests. Every request must carry what it needs — which is why the JWT travels in the Authorization header instead of the server holding a session."

"How does the frontend know your API shape?"

"We agree on the contract: method, path, request JSON and response JSON, plus status codes. In my projects the React side calls those endpoints with `fetch` and sends the token in the Authorization header."

SELF-TEST

1. Define API, REST, endpoint, path variable, query parameter.
2. Which methods are idempotent and what does that mean?
3. Give me the status codes for: created, validation error, expired token, wrong role, illegal state transition, unexpected bug.
4. Authentication vs authorization in one line each.
5. Why does JWT fit a stateless API?
6. "You say the escrow can move FUNDED → SHIPPED. Which HTTP method and why?"
7. "What does the browser send on the second request after login?"

Key

1. API = contract; REST = resource + verbs + stateless; endpoint = method+path; path variable identifies the resource; query param filters/shapes it.
2. GET, PUT, DELETE — repeating has the same effect.
3. 201 / 400 / 401 / 403 / 409 / 500.
4. AuthN = who you are; AuthZ = what you may do.
5. Identity travels in the signed token, so the server needs no session state.
6. POST — it's a command/action, not a replacement of the resource.
7. `Authorization: Bearer <jwt>` plus the request body; no cookies/session needed.

Next: `04-jpa-hibernate.md`

SECTION 04

04 — DATABASE → ORM → JPA → HIBERNATE → SPRING DATA JPA

Time: 20 min. This is P0 and one of the most dangerous claims on your resume.

STEP 0 — WHY DATABASES EXIST

Problem: if your Java objects only live in memory, they vanish when the program stops. You need durable storage that survives restarts, is queryable, and can be shared between users.

SAY: "A database is a program that stores data safely on disk and lets many clients read and write it concurrently with transactions, so data doesn't get corrupted."

- **Relational database (RDBMS):** data organised in **tables** (rows and columns) with **relationships** enforced by keys and constraints. You query with SQL. Examples: PostgreSQL, MySQL, Oracle. Strong consistency + ACID.
- **Non-relational (NoSQL):** documents, key-value, graphs. Flexible schema, easy horizontal scaling, but different consistency trade-offs. (PostgreSQL is what your resume uses — no need to go further.)

Analogy: a spreadsheet the whole company shares — but with rules: no duplicate IDs, no orphan rows, and changes that either fully happen or don't happen at all.

STEP 1 — THE "IMPEDANCE MISMATCH" (this is the *why* behind ORM)

In Java, you work with **objects**: `Transaction` has fields, references to other objects, and behaviour. Objects can point to each other in a graph, and changes are visible immediately in memory.

In a database, you work with **tables**: rows, columns, foreign keys. Nothing changes until you run a statement (INSERT/UPDATE) and commit.

They don't line up:

Java world	Database world
Class	Table
Object / instance	Row
Field	Column
Reference to another object	Foreign key
<code>new Transaction(...)</code>	<code>INSERT INTO transactions ...</code>
Mutating a field	<code>UPDATE transactions SET ...</code>
Collection of objects	<code>JOIN</code> / related table

This gap is called the **object-relational impedance mismatch**. Without help, you write hundreds of lines of `ResultSet` code by hand.

SAY: "Writing JDBC by hand means manually opening connections, writing SQL strings, mapping every column to a field, and closing everything in a finally block. It's error-prone and there's no compile-time safety. That's the problem ORM solves."

STEP 2 — WHAT IS ORM?

ORM = Object-Relational Mapping. It maps Java classes to tables and Java objects to rows, and generates the SQL for you.

SAY: "ORM is a layer that lets me work with objects and persists them to a relational database automatically. I declare the mapping once with annotations and the framework writes the SQL, manages the connection, tracks changes, and handles relationships."

Analogy: a translator between two people who don't share a language. You speak Java; the database speaks SQL; the ORM translates both directions. You should still *understand* SQL so you can check the translation.

Benefits: far less boilerplate, compile-time-safe queries (in the JPA sense), automatic dirty tracking, caching, relationship handling, portability across databases. **Costs:** generated SQL can be inefficient (N+1 queries, over-fetching), and it hides what's happening — so a developer who doesn't understand SQL gets into trouble. **Saying that trade-off out loud is a strong answer.**

STEP 3 — JPA vs HIBERNATE (guaranteed question)

SAY: **Exact wording:**

"**JPA is a specification** — a set of interfaces (`EntityManager`, `@Entity`, `@Id`, `@OneToMany`...) defined by Jakarta EE that says *how* object-relational mapping should work in Java. **Hibernate is an implementation** of that specification — it's the library that actually generates and runs the SQL. Spring Data JPA sits on top of Hibernate and removes most of the code I'd have to write: I declare an interface and it provides the implementation."

Analogy: JPA is the USB standard; Hibernate is a specific USB device. Spring Data JPA is the driver that makes it plug-and-play.

```
Your code
  ↓
Spring Data JPA  ← repository interfaces, derived queries, no boilerplate
  ↓
JPA (spec)      ← @Entity, EntityManager, JPQL, annotations
  ↓
Hibernate (impl) ← SQL generation, persistence context, dialect, caching
  ↓
JDBC + connection pool (HikariCP)
  ↓
PostgreSQL
```

TRAP: "So you're using Hibernate?" → "I use JPA as the API and Hibernate as the provider underneath, since Spring Boot's `spring-boot-starter-data-jpa` brings Hibernate in by default. I could swap in EclipseLink without changing my entities."

STEP 4 — YOUR ANNOTATIONS, ONE BY ONE

```

@Entity
@Table(name = "transactions",
       uniqueConstraints = @UniqueConstraint(columnNames = {"idempotency_key"}))
public class Transaction extends BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private TransactionStatus status;

    @Column(name = "idempotency_key", unique = true)
    private String idempotencyKey;

    @Version
    private Long version;                // optimistic locking

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "buyer_id", nullable = false)
    private User buyer;

    @OneToOne(mappedBy = "transaction", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private Escrow escrow;

    @OneToMany(mappedBy = "transaction", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("createdAt ASC")
    private List<TransactionHistory> history = new ArrayList<>();
}

```

Annotation	Meaning	Why it matters
@Entity	this class maps to a table	Hibernate manages instances of it
@Table(name=...)	the table name (+ indexes/constraints)	table is <code>transactions</code> , not <code>Transaction</code>
@Id	primary key field	identifies the row
@GeneratedValue(strategy = IDENTITY)	DB auto-generates the key	Postgres <code>BIGSERIAL</code> ; alternatives: <code>SEQUENCE</code> , <code>AUTO</code> , <code>UUID</code>
@Column	column name, nullability, length, precision	maps fields to columns; <code>BigDecimal</code> + <code>precision/scale</code> for money
@Enumerated(EnumType.STRING)	store the enum's <i>name</i>	far safer than <code>ORDINAL</code> (0,1,2), where reordering the enum silently corrupts data
@Version	optimistic locking column	concurrent update fails instead of silently overwriting

Annotation	Meaning	Why it matters
@ManyToOne + @JoinColumn	many rows here point to one row there	transactions.buyer_id → users.id
@OneToOne(mappedBy=...)	the inverse side; the other entity owns the FK	escrows.transaction_id holds the key
@OneToMany(mappedBy=...)	one parent, many children	transaction → history rows
cascade = ALL	operations propagate to children	saving the transaction saves the escrow/history
orphanRemoval = true	removing a child from the collection deletes its row	keeps history clean if a record is removed
fetch = LAZY	load only when accessed	avoids loading huge graphs on every query
@Temporal / Instant/LocalDateTime	date/time mapping	Instant → timestamp _{tz} , best for audit timestamps
@Transient	don't persist this field	e.g. a computed value kept only in memory

Where DTOs fit: @Entity classes are for persistence; the API sends **DTOs** (records/POJOs). This prevents mass-assignment (a client shouldn't be able to set `status` or `role` directly), avoids leaking password hashes, and stops lazy-loading errors from surfacing during JSON serialisation. *Saying this is a strong senior signal.*

STEP 5 — REPOSITORY / SPRING DATA JPA

```
public interface TransactionRepository extends JpaRepository<Transaction, Long> {
    List<Transaction> findByBuyerIdOrderByCreatedAtDesc(Long buyerId);

    Optional<Transaction> findByIdempotencyKey(String key);

    @Query("""
        SELECT t FROM Transaction t
        JOIN FETCH t.escrow
        WHERE t.buyer.id = :buyerId AND t.status = :status
        """)
    List<Transaction> findWithEscrow(@Param("buyerId") Long buyerId,
                                    @Param("status") TransactionStatus status);
}
```

What JpaRepository<Transaction, Long> gives you for free: `save`, `saveAll`, `findById`, `findAll`, `existsById`, `count`, `deleteById`, `deleteAll`, plus paging (`findAll(Pageable)`) and sorting. `<Transaction, Long>` = *entity type*, *primary key type*.

Derived queries: `findByBuyerIdAndStatus` — Spring parses the method name and generates the JPQL/SQL. And, Or, OrderBy, Containing, Between, LessThan, In, IgnoreCase are supported. `@Query` for anything complex — JPQL operates on **entities and fields**, not table names/columns. `@Modifying @Query` for bulk update/delete. `@Param` binds method arguments into the query.

SAY: "I define the repository as an interface; Spring Data creates the implementation at startup. Simple lookups come from the method name, and anything non-trivial I write as `@Query` in JPQL."

TRAP: "Is it a class or interface?" → "An interface. `JpaRepository` is an interface with implementations (`SimpleJpaRepository`); Spring generates a proxy for *my* interface at runtime."

STEP 6 — ENTITY RELATIONSHIPS (they will ask you to draw RevertPay)

In a relational database, a relationship is just a foreign key column. Object relationships are the *navigation* convenience on top.

One-to-many / many-to-one (the common case)

```
users —1—N—> transactions      transactions.buyer_id → users.id
                                transactions.seller_id → users.id
transactions —1—N—> transaction_history
```

```
// Child (owning side — holds the FK column)
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "buyer_id")
private User buyer;

// Parent (inverse side — mappedBy points at the field in the child)
@OneToMany(mappedBy = "transaction", cascade = CascadeType.ALL)
private List<TransactionHistory> history = new ArrayList<>();
```

SAY: "The `@ManyToOne` side owns the foreign key. The `@OneToMany` side is mapped by it — `mappedBy` is the name of the field in the child class, not a column."

One-to-one

```
transactions —1—1—> escrows    (escrows.transaction_id UNIQUE)
```

```
@OneToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "transaction_id", unique = true)
private Transaction transaction;           // escrow owns the FK

@OneToOne(mappedBy = "transaction", cascade = CascadeType.ALL)
private Escrow escrow;                    // transaction side
```

Many-to-many

Needs a **join table**: `users ↔ skills` via `user_skills(user_id, skill_id)`.

```
@ManyToMany
@JoinTable(name = "user_skills",
           joinColumns = @JoinColumn(name = "user_id"),
           inverseJoinColumns = @JoinColumn(name = "skill_id"))
private Set<Skill> skills = new HashSet<>();
```

SAY: "Use a `Set`, not a `List`, for many-to-many — it avoids duplicate rows and makes `remove` work sensibly."

NOTE: **The N+1 problem** — the most valuable thing you can volunteer:

"If I load 100 transactions and each lazily loads its buyer, that's 1 query plus 100 more — the N+1 problem. I fix it by fetching the association in the same query — a `JOIN FETCH` in JPQL or an `@EntityGraph` — or by using a DTO projection that selects exactly the columns I need." Demonstrating awareness of this marks you above the average fresher.

STEP 7 — PERSISTENCE CONTEXT, DIRTY CHECKING, LOADING

Persistence context

SAY: "The persistence context is Hibernate's first-level cache and unit of work — a set of managed entities for the current transaction. Within one transaction, loading the same row twice returns the same Java object rather than a second copy. It's the reason dirty checking can work."

Entity states:

- **Transient / new** — just created with `new`, not known to Hibernate.
- **Managed / persistent** — attached to the persistence context; changes are tracked.
- **Detached** — was managed, but the transaction/session ended.
- **Removed** — scheduled for deletion.

Dirty checking

SAY: "If an entity is managed and I change a field, Hibernate compares the object against the snapshot it took when the entity was loaded, and at flush time it emits the `UPDATE` automatically — I don't call `save()` for a managed entity inside a transaction."

```
@Transactional
public void ship(Long id, User actor) {
    Transaction t = repo.findById(id).orElseThrow(); // managed
    t.getEscrow().setStatus(SHIPPED);                // just mutate
    // no save() needed → UPDATE is generated at commit
}
```

TRAP: "So `save()` is useless?" → "No. `save()` is needed for new/transient entities and for detached ones. Inside a transaction with a managed entity, dirty checking already handles it."

LAZY vs EAGER

	LAZY	EAGER
Loads	on first access	immediately with the parent
Default for	<code>@OneToMany</code> , <code>@ManyToOne</code> , <code>@OneToOne</code> (* <code>@ManyToOne/@OneToOne</code> default to EAGER — always override to LAZY)	—
Good for	big graphs, lists of children	a small mandatory association
Risk	<code>LazyInitializationException</code> after the session closes	silently loading the whole object graph, N+1

TRAP: "What is `LazyInitializationException`?" → "Accessing a lazy association after the transaction/session has closed, so Hibernate can't run the query. I avoid it by fetching what I need inside the transaction — `JOIN FETCH` or an `@EntityGraph` — or by mapping to a DTO before returning." TRAP: "Can you access lazy fields in a controller?" → "Not safely — the transaction usually ends in the service. That's part of why I return DTOs."

Cascade & orphanRemoval

SAY: "Cascade propagates an operation from parent to child — `CascadeType.ALL` means if I persist the transaction, its escrow and history are persisted too. `orphanRemoval` deletes a child row when it's removed from the parent's collection."

NOTE: **Bidirectional consistency**: if you add a child to a parent's list, also set the child's parent reference — otherwise the in-memory object graph disagrees with the FK that will be written. Provide helper methods:

```
public void addHistory(TransactionHistory h) {
    history.add(h);
    h.setTransaction(this);
}
```

STEP 8 — TRANSACTIONS & `@Transactional` (critical for a money project)

SAY: "A database transaction groups several statements so they either all commit or all roll back — that's atomicity. In Spring, `@Transactional` on a service method wraps it in one transaction: the escrow status change, both ledger entries, and the audit row all succeed together or none of them happen."

ACID:

- **Atomicity** — all or nothing.
- **Consistency** — constraints and invariants hold before and after (no money created or destroyed).
- **Isolation** — concurrent transactions don't corrupt each other (levels: `READ_COMMITTED` default in Postgres, `REPEATABLE_READ`, `SERIALIZABLE`).
- **Durability** — after commit, it survives a crash.

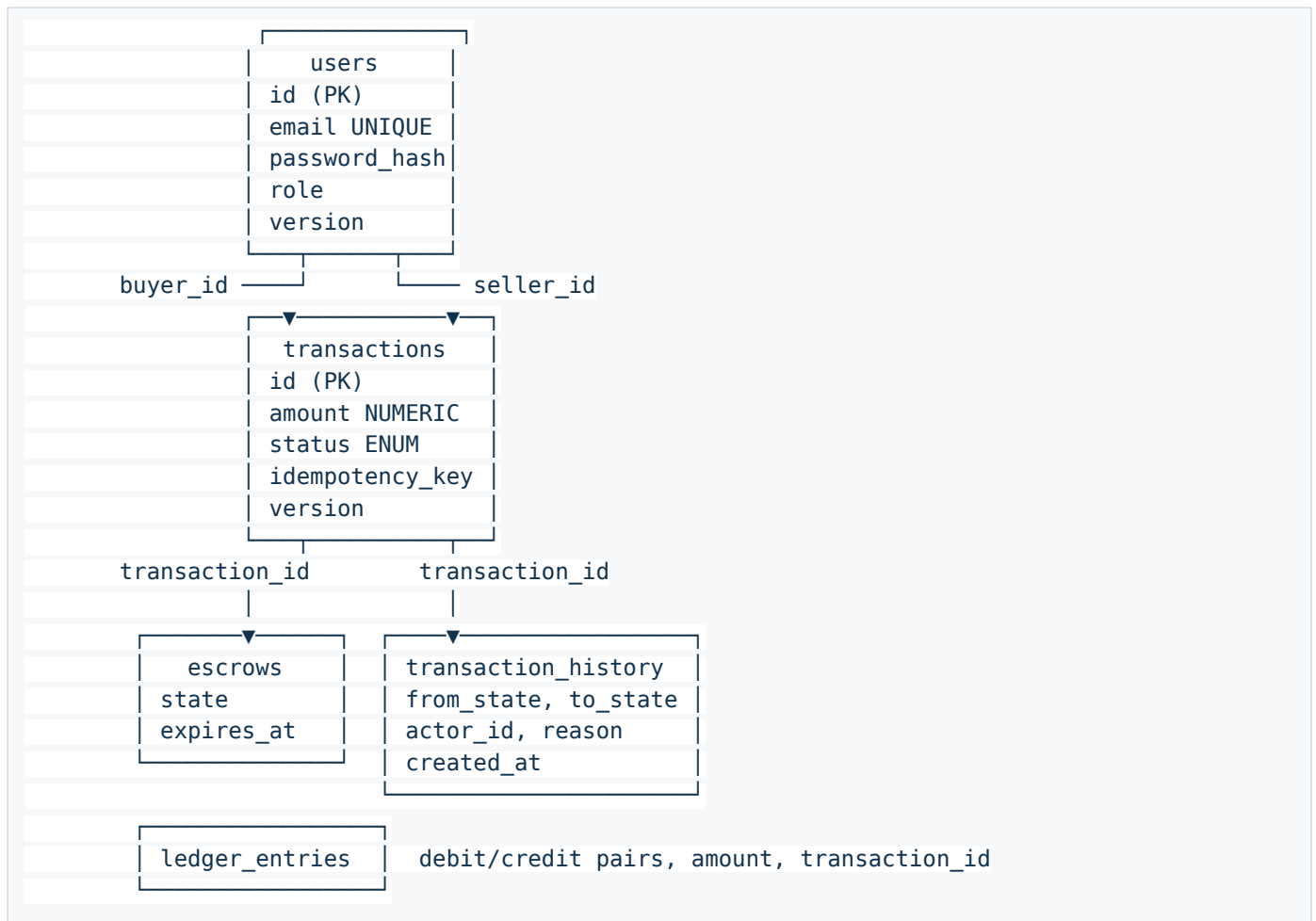
Key mechanics to say:

- Default **propagation** is `REQUIRED`: join an existing transaction, or start one.
- **Rollback rule**: by default Spring rolls back on **unchecked** (`RuntimeException`) exceptions only — not on checked ones. Use `@Transactional(rollbackFor = Exception.class)` if you need checked exceptions to roll back. *This is a classic trap.*
- `@Transactional(readOnly = true)` on read methods — a hint that lets Hibernate skip dirty-checking and lets the DB optimise.
- Where to put it: **service layer, public methods only**. It's proxy-based, so self-invocation (calling a `@Transactional` method from another method of the same class) bypasses the proxy and silently loses the transaction. *Another classic trap.*
- **Transaction boundary in RevertPay**: "the money legs and the state change live in the same transactional service method, so I can never end up with funds released but the escrow still showing DELIVERED-not-released."

NOTE: "What if two requests try to release the same escrow at the same time?" SAY: "Two protections. First, the status check and the update are inside one transaction, so one of them will see the wrong state and be rejected with a 409. Second, I use **optimistic locking** — an `@Version` column — so if two transactions both read version 3 and try to write, the second gets an `OptimisticLockException` instead of silently double-releasing. A stronger option is a pessimistic lock (`SELECT ... FOR UPDATE`) or a database-level unique constraint on the ledger entries." **Deliver that answer and you will impress them.** It's the highest-value paragraph in this whole pack for a payments-flavoured project.

SAY: "What about retries? The API takes an **idempotency key** on escrow creation with a unique constraint, so a retried POST can't create two escrows." (True of your case study — but read file 12 on how to be *honest* about the parts that were planned vs shipped.)

STEP 9 — "SHOW ME HOW YOUR ENTITIES ARE RELATED" (rehearse this)



SAY: "Users have many transactions in two roles, buyer and seller. Each transaction has one escrow holding its state and expiry, and many history rows that form the audit trail. Money movements are recorded as paired ledger entries referencing the transaction, and an account balance is derived as the sum of credits minus debits — I don't store a mutable balance column, because that can drift out of sync with the history."

SELF-TEST

1. What is ORM and what problem does it solve?
2. JPA vs Hibernate vs Spring Data JPA — one line each.
3. What does `<Transaction, Long>` mean in `JpaRepository<Transaction, Long>`?
4. `@ManyToOne` vs `@OneToMany` — which side owns the FK?
5. What's the persistence context and dirty checking?
6. LAZY vs EAGER, and what exception does LAZY cause if misused?
7. What does `@Transactional` guarantee, and which exceptions roll back by default?
8. Two concurrent release requests — walk me through what happens.

Key

1. Maps objects ↔ tables, removes JDBC boilerplate, handles relationships and change tracking.
2. Spec / implementation / repository abstraction that generates implementations.
3. Entity type and primary-key type.
4. `@ManyToOne` (child) owns the FK column; `@OneToMany` uses `mappedBy`.

5. A per-transaction first-level cache/unit of work; dirty checking auto-issues UPDATES for changed managed entities.
6. LAZY = on access, EAGER = immediately; `LazyInitializationException`.
7. All-or-nothing grouping; unchecked (`RuntimeException`) by default.
8. One sees an invalid state → 409; `@Version` optimistic locking makes the loser fail rather than double-release.

Next: [05-postgresql-sql.md](#)

SECTION 05

05 — POSTGRESQL + SQL

Time: 15 min (skim teaching, practice queries). P0 for SQL basics, P2 for deep internals.

PART A — POSTGRESQL

What is a relational database?

Data lives in **tables** made of **rows** (records) and **columns** (typed fields). Tables relate through **keys**. You query with **SQL**. The DB enforces the rules.

What is PostgreSQL and why use it?

SAY: "PostgreSQL is an open-source relational database. I use it because it's free and open source, it's fully ACID-compliant with strong transactional guarantees, it has a rich type system — proper `NUMERIC` for money, `timestampz`, UUIDs, JSONB — and it enforces constraints like unique and foreign keys at the database level, so bad data can't get in even if my application code has a bug."

Why Postgres over MySQL? → "Both work. Postgres has stricter standards compliance, richer types, better support for complex queries and window functions, and it's the common choice in Java/Spring shops. MySQL is often lighter and very common in shared hosting." **Why not NoSQL for RevertPay?** → "Money needs strong consistency and transactions across multiple rows — the escrow state, the ledger entries and the audit record must commit together. A relational database with ACID transactions is the right fit; eventual consistency would be dangerous here."

The vocabulary

Term	Meaning	Example
Table	A named collection of rows	<code>transactions</code>
Row / record	One entity instance	one escrow transaction
Column / field	One typed attribute	<code>amount NUMERIC(12,2)</code>
Primary key (PK)	Unique, non-null identifier for a row	<code>id BIGSERIAL PRIMARY KEY</code>
Foreign key (FK)	A column referencing another table's PK; enforces referential integrity	<code>buyer_id REFERENCES users(id)</code>
Unique	No duplicates allowed	<code>email UNIQUE</code>
NOT NULL	Value required	<code>status TEXT NOT NULL</code>
CHECK	Value must satisfy a condition	<code>CHECK (amount > 0)</code>
Index	Extra structure for fast lookups	<code>CREATE INDEX ON transactions(buyer_id)</code>
Constraint	A rule enforced by the DB	unique, FK, check, not null

SAY: "The database is the last line of defence. Even if a bug slips through the service layer, a foreign key or a unique constraint prevents corrupt data."

Relationships

- **1:N** — a foreign key lives in the "many" table (`transactions.buyer_id → users.id`).
- **1:1** — a foreign key plus a `UNIQUE` constraint (`escrows.transaction_id UNIQUE`).
- **N:N** — a **join table** holding two FKs (`user_skills(user_id, skill_id)`, primary key on both).

Indexes

SAY: "An index is a sorted structure — a B-tree by default in Postgres — that lets the database find rows without scanning the whole table. It turns a full-table scan into a lookup, roughly $O(n)$ into $O(\log n)$. The cost is extra storage and slower writes, because every insert has to update the index."

- **Primary keys are automatically indexed.** So are `UNIQUE` columns.
- **Foreign keys are NOT automatically indexed** in Postgres → index your FK columns, because joins and lookups filter on them. *This is a great detail to volunteer.*
- Composite index (`buyer_id, created_at`) serves queries filtering by buyer and sorting by date.
- Indexes help `WHERE`, `JOIN`, `ORDER BY`. They don't help if you wrap the column in a function (`WHERE LOWER(email) = ...` won't use a plain index on `email`).
- `EXPLAIN ANALYZE` shows whether an index was used — that's "how I would investigate a slow query".

Transactions & ACID

```
BEGIN;
UPDATE escrows SET state = 'RELEASED' WHERE transaction_id = 42 AND state = 'DELIVERED';
INSERT INTO ledger_entries (transaction_id, account_id, direction, amount) VALUES (42, 7, 'DEBIT', 1500.00);
INSERT INTO ledger_entries (transaction_id, account_id, direction, amount) VALUES (42, 9, 'CREDIT', 1500.00);
INSERT INTO transaction_history (transaction_id, from_state, to_state) VALUES (42, 'DELIVERED', 'RELEASED');
COMMIT;      -- or ROLLBACK;
```

SAY: "Postgres is ACID: atomic, consistent, isolated and durable. That's why the escrow state change and its ledger entries are in one transaction — a partial commit would mean money moved without the state changing, or vice versa."

Isolation levels (Postgres default = `READ COMMITTED`): `READ COMMITTED` sees only committed data; `REPEATABLE READ` sees a stable snapshot for the whole transaction and errors on write conflicts; `SERIALIZABLE` is as if transactions ran one at a time and may require retries.

NOTE: "The `UPDATE ... WHERE state = 'DELIVERED'` above is a **conditional update** — if zero rows change, someone already released it, and the service turns that into a 409. That's a database-level guard on top of the application check."

Normalisation (basic)

- **1NF**: atomic values — no comma-separated lists in a column (that's why skills are a join table).
- **2NF**: no partial dependency on part of a composite key.
- **3NF**: no transitive dependency — don't store `seller_email` in `transactions` when it lives in `users`. SAY: "I normalise to avoid duplicating data and creating update anomalies. Where I need speed for a read-heavy view, I'd denormalise deliberately and document it."

Derived balance (your RevertPay design decision — say it with confidence)

```
SELECT COALESCE(SUM(CASE WHEN direction = 'CREDIT' THEN amount ELSE -amount END), 0) AS balance
FROM ledger_entries
WHERE account_id = :accountId;
```

SAY: "I don't store a balance column. The ledger is append-only and the balance is derived from it, so the balance can never disagree with the history — an audit is just the same arithmetic re-run."

PART B — SQL (they may ask you to write it)

Retrieval — easy → medium → project-specific

Easy

```
SELECT id, amount, status FROM transactions WHERE status = 'DELIVERED';

SELECT * FROM users WHERE email = 'siva@example.com';

SELECT * FROM transactions ORDER BY created_at DESC LIMIT 10;

SELECT DISTINCT status FROM transactions;

SELECT COUNT(*) FROM transactions WHERE buyer_id = 7;
SELECT SUM(amount), AVG(amount), MIN(amount), MAX(amount) FROM transactions;
```

GROUP BY / HAVING (the classic distinction)

```
-- how many transactions per status, but only statuses with more than 5
SELECT status, COUNT(*) AS total
FROM transactions
GROUP BY status
HAVING COUNT(*) > 5
ORDER BY total DESC;
```

SAY: "WHERE filters **rows before** grouping. HAVING filters **groups after** aggregation. You can't use an aggregate in WHERE."

Aggregate with a join (per user, with names)

```
SELECT u.id, u.name, COUNT(t.id) AS tx_count, COALESCE(SUM(t.amount),0) AS volume
FROM users u
LEFT JOIN transactions t ON t.buyer_id = u.id
GROUP BY u.id, u.name
ORDER BY volume DESC;
```

SAY: "A LEFT JOIN from users keeps users with zero transactions; that's why I coalesce the sum to zero."

INNER vs LEFT JOIN

```
-- INNER: only rows that match on both sides
SELECT t.id, u.email FROM transactions t
JOIN users u ON u.id = t.buyer_id;

-- LEFT: all transactions, seller may be NULL
SELECT t.id, u.email AS seller_email FROM transactions t
LEFT JOIN users u ON u.id = t.seller_id;
```

Join	Returns
INNER JOIN	only matching rows from both tables
LEFT JOIN	all rows from the left + matches (NULLs where none)

Join	Returns
RIGHT JOIN	all rows from the right (rarely used — just flip the tables)
FULL OUTER	all rows from both (rare)
CROSS JOIN	cartesian product (rare, dangerous)
SELF JOIN	a table joined to itself (users buyer ↔ seller)

TRAP: "What happens if you **JOIN** on the wrong column?" → "You get a cartesian-style blow-up — the row count multiplies and the aggregates are wrong. That's why I check row counts on joined queries."

Subquery, IN, EXISTS

```
-- subquery
SELECT * FROM users
WHERE id IN (SELECT buyer_id FROM transactions WHERE status = 'UNDER_REVIEW');

-- EXISTS (usually better: stops at the first match)
SELECT u.* FROM users u
WHERE EXISTS (SELECT 1 FROM transactions t
              WHERE t.buyer_id = u.id AND t.status = 'UNDER_REVIEW');

-- correlated scalar subquery
SELECT t.id,
       (SELECT COUNT(*) FROM transaction_history h WHERE h.transaction_id = t.id) AS events
FROM transactions t;
```

SAY: "**IN** compares against a list of values; **EXISTS** asks whether any row satisfies the condition, and it can short-circuit. For anti-joins I'd use **NOT EXISTS** rather than **NOT IN**, because **NOT IN** behaves unexpectedly if the subquery returns a NULL."

Anti-join: users with no transactions

```
SELECT u.* FROM users u
LEFT JOIN transactions t ON t.buyer_id = u.id
WHERE t.id IS NULL;
```

Duplicates (very common interview question)

```
-- find duplicated emails
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- keep the smallest id, delete the rest
DELETE FROM users a USING users b
WHERE a.email = b.email AND a.id > b.id;
```

Money sanity check — do debits equal credits? (project-specific, shows you understand a ledger)

```
SELECT transaction_id,
       SUM(CASE WHEN direction='DEBIT' THEN amount ELSE -amount END) AS imbalance
FROM ledger_entries
GROUP BY transaction_id
HAVING SUM(CASE WHEN direction='DEBIT' THEN amount ELSE -amount END) <> 0;
```

SAY: "A double-entry ledger must balance to zero per transaction, so this query is the invariant check for RevertPay — and it runs in the audit view."

Window function (bonus, impresses in 10 seconds)

```
-- most recent transaction per buyer
SELECT * FROM (
  SELECT t.*, ROW_NUMBER() OVER (PARTITION BY buyer_id ORDER BY created_at DESC) AS rn
  FROM transactions t
) ranked
WHERE rn = 1;
```

Pagination (how a real API does it)

```
SELECT * FROM transactions WHERE buyer_id = 7 ORDER BY created_at DESC LIMIT 20 OFFSET 40;
```

DDL — CREATE TABLE (be able to write this for RevertPay)

```

CREATE TABLE users (
  id          BIGSERIAL PRIMARY KEY,
  name        VARCHAR(120) NOT NULL,
  email       VARCHAR(180) NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,
  role        VARCHAR(20) NOT NULL CHECK (role IN ('BUYER','SELLER','ADMIN')),
  version     BIGINT NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE transactions (
  id          BIGSERIAL PRIMARY KEY,
  buyer_id    BIGINT NOT NULL REFERENCES users(id),
  seller_id    BIGINT NOT NULL REFERENCES users(id),
  amount       NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  status       VARCHAR(20) NOT NULL,
  idempotency_key VARCHAR(80) UNIQUE,
  version     BIGINT NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_transactions_buyer ON transactions(buyer_id);
CREATE INDEX idx_transactions_seller ON transactions(seller_id);

CREATE TABLE escrows (
  id          BIGSERIAL PRIMARY KEY,
  transaction_id BIGINT NOT NULL UNIQUE REFERENCES transactions(id) ON DELETE CASCADE,
  state       VARCHAR(20) NOT NULL,
  expires_at  TIMESTAMPTZ
);

CREATE TABLE ledger_entries (
  id          BIGSERIAL PRIMARY KEY,
  transaction_id BIGINT NOT NULL REFERENCES transactions(id),
  account_id   BIGINT NOT NULL REFERENCES users(id),
  direction    VARCHAR(6) NOT NULL CHECK (direction IN ('DEBIT','CREDIT')),
  amount       NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE transaction_history (
  id          BIGSERIAL PRIMARY KEY,
  transaction_id BIGINT NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
  from_state   VARCHAR(20),
  to_state     VARCHAR(20) NOT NULL,
  actor_id     BIGINT REFERENCES users(id),
  reason       TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

SAY: "Points worth calling out: `NUMERIC(12,2)` not `FLOAT` for money — floats can't represent 0.1 exactly; `CHECK` constraints enforce valid states and positive amounts; foreign keys keep referential integrity; `UNIQUE` on `idempotency_key` makes retries safe; indexes on the FK columns I filter by; and `version` for optimistic locking."

INSERT / UPDATE / DELETE

```
INSERT INTO users (name, email, password_hash, role)
VALUES ('Siva', 'siva@example.com', '$2a$10$...', 'BUYER')
RETURNING id;

UPDATE escrows SET state = 'RELEASED' WHERE transaction_id = 42 AND state = 'DELIVERED';
-- check the row count: 1 = released, 0 = someone else already did it

DELETE FROM transactions WHERE id = 99;
-- prefer soft delete (deleted_at / status ARCHIVED) for financial data — you need the history
```

SAY: "For money data I would not hard-delete. A financial record should be append-only with an archived or reversed status, so the audit trail survives."

SELF-TEST (write these out — actually type them)

1. All transactions for buyer 7 with a status of DELIVERED, newest first.
2. Count transactions per status, only statuses with more than 5.
3. Users with zero transactions.
4. Total money released per seller.
5. Which transactions don't balance in the ledger?
6. Why `NUMERIC` and not `FLOAT` for money?
7. Why index a foreign key in Postgres?
8. WHERE vs HAVING in one line.

Key

1. `SELECT * FROM transactions WHERE buyer_id=7 AND status='DELIVERED' ORDER BY created_at DESC;`
2. `SELECT status, COUNT(*) FROM transactions GROUP BY status HAVING COUNT(*)>5;`
3. `SELECT u.* FROM users u LEFT JOIN transactions t ON t.buyer_id=u.id WHERE t.id IS NULL;`
4. `SELECT seller_id, SUM(amount) FROM transactions WHERE status='RELEASED' GROUP BY seller_id;`
5. The `HAVING SUM(CASE ...) <> 0` query above.
6. Floating point is binary and can't represent decimals exactly — rounding errors accumulate in money. `NUMERIC` is exact decimal.
7. Postgres doesn't auto-index FKs, and joins/lookups filter on them — without it, joins become full scans.
8. WHERE filters rows before grouping; HAVING filters groups after aggregation.

Next: [06-security-jwt.md](#)

SECTION 06

06 — SECURITY: SPRING SECURITY + JWT + BCrypt + ROLES

Time: 25 min. This is your EXTREME DANGER ZONE. If you learn one file properly, learn this one.

STEP 0 — WHAT DOES "APPLICATION SECURITY" MEAN?

SAY: "Security in a web application means three practical things: only the right people can get in (authentication), they can only do what they're allowed to do (authorization), and their data and credentials are never exposed in transit, in the database, or in logs."

The attack surface you should be able to name (briefly):

- Anyone can call your API — it's a public URL. So the server must verify every request.
- Clients can send anything — so **never trust client input**. Validate on the server.
- Databases get leaked — so **never store plaintext passwords**.
- Tokens/URLs leak through logs, history, referrers — so never put secrets in a URL.
- Users shouldn't be able to read each other's data — that's **object-level authorization**, not just roles.

STEP 1 — AUTHENTICATION vs AUTHORIZATION (say it EXACTLY like this)

SAY: "**Authentication is *who you are*. Authorization is *what you're allowed to do*. Authentication happens first — you prove your identity, usually with a username and password — and then on every request authorization checks whether that identity has permission for this action.**"

Airport analogy: passport = authentication; boarding pass = authorization. You can be authenticated and still refused boarding.

Concrete example to give:

"In RevertPay, a seller can mark a transaction as shipped. Being logged in authenticates them, but if they try to ship *someone else's* transaction, that's an authorization failure — 403, not 401."

STEP 2 — SPRING SECURITY

What is it?

SAY: "Spring Security is the part of Spring that handles authentication and authorization. Its core is a **chain of servlet filters** that runs before my controller. Each request passes through the chain, which decides whether the user is authenticated, what authorities they have, and whether the request may reach the endpoint. I configure which paths are public and which need a role."

Why use it instead of writing checks by hand?

"Because security is cross-cutting: it has to apply to every endpoint, and one forgotten check is a hole. Spring Security centralises it. I declare the rules in one configuration class, and it also handles password hashing, session/token handling and CSRF protection."

Key concepts:

Concept	Meaning
<code>SecurityFilterChain</code>	the ordered list of filters each request passes through
<code>Authentication</code>	the object representing "the currently authenticated principal + authorities"
<code>SecurityContextHolder</code>	where the current authentication is stored for the duration of the request
<code>UserDetailsService</code>	loads a user's credentials and authorities by username from your DB
<code>UserDetails</code>	the interface representing "a user Spring Security understands"
<code>AuthenticationManager</code> / <code>AuthenticationProvider</code>	verifies credentials (delegates to <code>UserDetailsService</code> + <code>PasswordEncoder</code>)
<code>PasswordEncoder</code>	hashing/verifying passwords (we use BCrypt)
<code>GrantedAuthority</code>	a permission string like <code>ROLE_ADMIN</code>
<code>AuthenticationEntryPoint</code>	what to return when not authenticated (401)
<code>AccessDeniedHandler</code>	what to return when authenticated but not allowed (403)

The filter chain, at interview level

Request
→ CORS filter
→ CSRF filter (relevant for cookie/session auth; for stateless JWT APIs it's usually disabled)
→ Authentication filters ← MY JWT FILTER RUNS HERE
→ Authorization filters (match the request path against my rules)
→ <code>ExceptionHandler</code> (turns security errors into 401/403)
→ <code>FilterSecurityInterceptor</code> → finally the <code>DispatcherServlet</code> & my controller

SAY: "The JWT filter sits before the authorization filter: it reads the token, validates it, and if it's valid it puts the authenticated user into the `SecurityContext`. Then the authorization filter checks the path rules and the roles."

STEP 3 — PASSWORDS: WHY HASHING, AND WHY BCRIPT

The rule

Never store a password. Store a hash.

SAY: "Hashing is one-way: I can check whether a submitted password matches the stored hash, but I can't get the password back out of the hash. If the database leaks, the attacker doesn't get usable passwords."

One-way hash vs reversible encryption

	Encryption	Hashing
Direction	Two-way (encrypt ↔ decrypt)	One-way
Needs a key	Yes	No (only a salt)
Use for	Data you must read back (e.g. an API token you must replay, a card number vault)	Passwords — you only ever <i>verify</i> , never read back

TRAP: "Why not encrypt the password?" → "Because then I'd need the key on the server, and anyone who gets the key can decrypt every password. Passwords never need to be readable, so a one-way hash is both sufficient and safer."

Why plain hashing (like SHA-256) is not enough

- **Too fast** — a GPU can compute billions of SHA-256 hashes per second, so a leaked table of plain hashes is brute-forceable.
- **Rainbow tables** — precomputed lookup tables map hashes back to common passwords.
- **Identical passwords → identical hashes**, so cracking one reveals everyone with that password.

What BCrypt does about it

- **Salted automatically** — a random salt is generated per password and stored inside the hash string, so the same password produces different hashes and rainbow tables are useless.
- **Deliberately slow (adaptive)** — a configurable **work factor/cost** (e.g. 10 → 2¹⁰ rounds). It's a small enough delay for a login, but brutally slow for an attacker doing billions of guesses.
- The stored value looks like `$2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy` — algorithm, cost, salt and hash all in one string. `matches(raw, stored)` extracts the salt and cost from the stored value.

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(); // strength 10 by default
}

// registration
user.setPasswordHash(passwordEncoder.encode(rawPassword));

// login
if (!passwordEncoder.matches(rawPassword, user.getPasswordHash())) {
    throw new BadCredentialsException("Invalid email or password");
}
```

SAY: "At login I never decrypt anything. `matches()` re-hashes the submitted password with the same salt and cost and compares the results."

TRAP: "What does the salt do?" → "It makes each stored hash unique even for identical passwords, and it prevents precomputed rainbow-table attacks. The salt doesn't need to be secret — it just needs to be random and per-password." TRAP: "Should you hash before sending from the frontend?" → "No. The hash would then become the password — replayable. Passwords go over HTTPS, where TLS protects them in transit."

STEP 4 — SESSIONS vs JWT (understand the contrast before learning JWT)

Classic session-based auth

1. You log in; the server creates a session, stores it in memory (or Redis), and sends you a **session id** as a cookie.
2. The browser sends that cookie automatically on every request.
3. The server looks the id up in its session store to know who you are.

Pros: easy to revoke (delete the session), nothing sensitive in the client. **Cons:** the server holds state, so scaling to many servers needs shared storage (sticky sessions or Redis); cookies are vulnerable to CSRF.

Token-based (JWT) auth

1. You log in; the server **signs** a token containing your identity and returns it.
2. The client sends it in the `Authorization: Bearer ...` header.
3. The server **verifies the signature** — no lookup needed — and knows who you are.

Pros: stateless (any server can verify it), works well for APIs and mobile clients, no CSRF by default since the token isn't sent automatically by the browser. **Cons:** harder to revoke before expiry; if stored in `localStorage` it's exposed to XSS; tokens grow with claims.

SAY: "REST requires statelessness, so JWT fits naturally: the identity travels with the request and my server doesn't keep a session table."

STEP 5 — JWT ANATOMY (draw this on paper)

JWT = JSON Web Token. Three Base64URL-encoded parts separated by dots:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI0MiIsInJvbmGUiOiJCVVlFUlUiImV4cCI6MTc2MDAwMDAwMH0.dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

HEADER
PAYLOAD
SIGNATURE

1. Header

```
{ "alg": "HS256", "typ": "JWT" }
```

Which signing algorithm was used. **HS256** = HMAC-SHA256 with a **shared secret** (symmetric). **RS256** = RSA, signed with a private key and verified with a public key (better when multiple services verify tokens).

2. Payload (the "claims")

```
{
  "sub": "42",           // subject = user id (standard claim)
  "role": "BUYER",       // custom claim → becomes an authority for authorization
  "email": "siva@example.com",
  "iat": 1759800000,     // issued at
  "exp": 1760000000      // expiry ← the server checks this
}
```

Standard claims: `sub`, `iat`, `exp`, `iss` (issuer), `aud` (audience), `jti` (unique token id).

NOTE: THE MOST IMPORTANT SENTENCE ABOUT JWT: SAY: "The payload is only **Base64URL-encoded**, **not encrypted** — anyone can decode it and read it. So I never put secrets in a JWT. Its value is that it's **signed**: nobody can change the claims without invalidating the signature."

Anyone who says "JWT is encrypted" fails the question. Say "signed, not encrypted" and you pass.

3. Signature

```
signature = HMACSHA256(base64url(header) + "." + base64url(payload), SERVER_SECRET)
```

The server recomputes this on every request with its secret and compares. If someone edits the payload (e.g. changes `role` to `ADMIN`), the signature no longer matches → **rejected**.

TRAP: "What is the secret?" → "A server-side key, long and random, loaded from an environment variable — never committed to the repo, and rotated if leaked. If an attacker has the secret, they can mint valid tokens for any user."

STEP 6 — THE FULL FLOW (rehearse until you can draw it from memory)

Login (issue a token)

```
React: POST /api/auth/login { email, password }
↓
Spring Security filter chain – /api/auth/login is permitted without authentication
↓
AuthController.login(@Valid @RequestBody LoginRequest)
↓
AuthService.login()
1. userRepository.findByEmail(email) → 404/401 if absent
2. passwordEncoder.matches(raw, user.getPasswordHash()) → 401 if false
3. JwtService.generateToken(user) → sub, role, iat, exp → sign with HS256
↓
200 OK { "token": "eyJ...", "user": { id, name, role } }
```

Authenticated request (use the token)

```
React: POST /api/escrows/42/release
      Authorization: Bearer eyJ...
↓
JwtAuthenticationFilter (OncePerRequestFilter)
1. read the Authorization header, require the "Bearer " prefix
2. jwtService.extractUsername(token) / validate(token)
   - signature valid?           (else 401)
   - not expired?               (else 401)
   - user still exists/active?
3. build UsernamePasswordAuthenticationToken(user, null, authorities from the role claim)
4. SecurityContextHolder.getContext().setAuthentication(auth)
↓
Authorization filter: does this path + method require a role this user has?
   - no → 403 (authenticated but not permitted)
↓
Controller → Service → Repository → PostgreSQL
↓
200 OK { "status": "RELEASED" }
```

SAY: "The filter runs once per request, validates the token without touching the database for session state, and puts the authenticated user into the SecurityContext so `@PreAuthorize` and my ownership checks can use it."

STEP 7 — THE CONFIGURATION (be able to explain each line)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity // enables @PreAuthorize
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationFilter jwtFilter)
        throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless JWT API: no cookie → no CSRF vector
            .cors(Customizer.withDefaults()) // allow the React origin
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll() // register + login
                .requestMatchers(HttpMethod.GET, "/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated() // everything else needs a token
            )
            .exceptionHandling(e -> e
                .authenticationEntryPoint(restUnauthorizedEntryPoint()) // 401
                .accessDeniedHandler(restAccessDeniedHandler()) // 403
            )
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

Every line explained:

- `csrf.disable()` — CSRF attacks rely on the browser automatically attaching credentials (cookies). A bearer token is attached explicitly by my code, so there's no CSRF vector. **Mentioning *why* is what separates memorising from understanding.**
- `STATELESS` — never create or use an HTTP session; every request is authenticated from the token.
- `permitAll` — the auth endpoints must be reachable before you have a token, otherwise you could never log in.
- `hasRole("ADMIN")` — path-based authorization. Note Spring's `hasRole("ADMIN")` looks for the authority `ROLE_ADMIN`, so the prefix matters.
- `addFilterBefore(...)` — my JWT filter must run *before* the username/password filter so the authentication is available for the authorization decision.

TRAP: "You disabled CSRF — isn't that insecure?" → "For a stateful cookie-based app it would be. This API is stateless and expects an explicit bearer token, and browsers don't attach that automatically, so the CSRF attack model doesn't apply. If I switched to cookie-based tokens I'd re-enable CSRF protection with a token."

STEP 8 — THE JWT FILTER (the file they'd ask you to "show")

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                   FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader(HttpHeaders.AUTHORIZATION);
        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(req, res); // no token → continue; authorization decides later
            return;
        }

        String token = header.substring(7);
        try {
            String username = jwtService.extractUsername(token); // validates signature + exp
            if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails user = userDetailsService.loadUserByUsername(username);
                if (jwtService.isTokenValid(token, user)) {
                    var auth = new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());
                    auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(req));
                    SecurityContextHolder.getContext().setAuthentication(auth);
                }
            }
        } catch (JwtException | IllegalArgumentException ex) {
            SecurityContextHolder.clearContext(); // treat as anonymous → 401 via entry point
        }
        chain.doFilter(req, res);
    }
}
```

SAY: "Key points: it extends `OncePerRequestFilter`; it never throws to the client, it just leaves the context anonymous so Spring Security returns a clean 401 from the entry point instead of a stack trace; and it only sets the authentication if one isn't already set."

STEP 9 — ROLE-BASED AUTHORIZATION + OBJECT OWNERSHIP

Path-based (URL) rules

```
.requestMatchers("/api/admin/**").hasRole("ADMIN")
```

Method-based rules

```
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/api/escrows/{id}/resolve")
public ResponseEntity<?> resolve(@PathVariable Long id) { ... }

@PreAuthorize("hasAnyRole('BUYER', 'ADMIN')")
```

Ownership rules (roles are not enough!)

```
@Transactional
public void ship(Long txId, String email) {
    User actor = userRepository.findByEmail(email).orElseThrow();
    Transaction tx = transactionRepository.findById(txId).orElseThrow(NotFoundException::new);

    if (!tx.getSeller().getId().equals(actor.getId())) {
        throw new AccessDeniedException("Not your transaction"); // → 403
    }
    escrowService.transition(tx.getEscrow(), SHIPPED, actor, "seller marked shipped");
}
```

SAY: **This is the answer that wins the security question:**

"Roles alone aren't enough. A BUYER role doesn't mean 'any buyer can act on any transaction' — it means 'this buyer can act on *their own*'. So on top of role checks I do object-level ownership checks in the service layer: I compare the authenticated user's id with the buyer or seller on the transaction, and return 403 if it doesn't match. Security has to be enforced on the server; hiding a button in React is not authorization."

NOTE: If you say only that sentence about the React button, you sound like you understand the difference between UI and security. Very strong.

The three roles in RevertPay

Role	Can do
BUYER	create/fund an escrow, confirm delivery, open a dispute, view own transactions
SELLER	mark shipped, view own transactions, receive funds on release
ADMIN	view any transaction, resolve disputes (release or refund), investigate the ledger

SAY: "I model the role as an enum on the user and expose it as an authority. Both parties have an interest in the same transaction, so I keep `buyer_id` and `seller_id` as separate references and check which side the acting user is on."

STEP 10 — THE BAD-PATH ANSWERS (asked constantly)

"What happens if the JWT is invalid or expired?"

"The filter catches the signature/expiry exception, clears the security context, and continues. No authentication is set, so the authorization filter rejects the request and the `AuthenticationEntryPoint` returns **401 Unauthorized** with a JSON error body — not an HTML redirect, because it's an API. On the client, a 401 response means the token is stale, so React clears it and sends the user to the login screen."

"What if the user is authenticated but not authorized?"

"The authorization filter (or my ownership check in the service) throws an access-denied error, which the `AccessDeniedHandler` maps to **403 Forbidden**. 401 vs 403 matters to the client: 401 = log in again, 403 = you're logged in but this isn't yours."

"How do you handle expiry?"

"The token carries an `exp` claim and the server rejects anything past it. For a better UX I'd use a short-lived access token plus a refresh token, so a silent refresh happens without making the user log in again. In my project the access token simply expires and the user logs in again — a refresh flow is the next improvement."

"How do you log out?"

"JWT is stateless, so logging out is client-side: delete the token. That's the honest limitation — the token stays technically valid until it expires, which is why the expiry should be short. Real revocation needs server-side state: a blacklist or a token version on the user that invalidates older tokens."

"Where do you store the token on the client?"

"In my projects it's in browser storage and attached to the `Authorization` header by a small API helper. `localStorage` is simpler but vulnerable to XSS; a `HttpOnly` cookie is safer against XSS but reintroduces CSRF concerns, which is why it's stored there and not in a cookie. Being explicit about the trade-off is the point." NOTE: Honest, safe, and clearly shows you know the trade-off. **Do not claim you implemented `HttpOnly` cookies if you didn't.**

"What if someone steals the token?"

"They can act as that user until it expires. That's why TLS, short expiry, and not logging tokens matter. Mitigations are refresh-token rotation, binding to a device, or checking `jti` against a revocation list."

SELF-TEST (this one matters most — answer all 10 out loud)

1. Authentication vs authorization.
2. Why BCrypt and not SHA-256 for passwords?
3. Why is a hash better than encryption for passwords?
4. Name the three parts of a JWT and what each does.
5. **Is a JWT encrypted?**
6. What exactly does the server verify on each request?
7. Where does your JWT filter sit and what does it set?
8. 401 vs 403 — and which does your code return when a buyer touches someone else's transaction?
9. Why is CSRF disabled in your config, and why is that acceptable here?
10. "Show me role-based authorization." What do you open, and what do you say?

Key

1. Who you are / what you may do.
2. BCrypt is slow and salted by design → brute-force resistant; SHA-256 is fast and unsalted → GPU-crackable + rainbow tables.
3. Hashing is one-way so a leak doesn't reveal passwords; encryption requires a key on the server that would unlock everything.
4. Header = algorithm; payload = claims (sub, role, exp); signature = HMAC over the first two with the server secret.
5. **No — only Base64URL-encoded and signed; anyone can read it, nobody can change it undetected.**
6. Signature validity, expiry, and that the user still exists/is active — then it derives authorities.
7. Before `UsernamePasswordAuthenticationFilter`; it sets the `Authentication` in `SecurityContextHolder`.
8. 401 = not authenticated; 403 = authenticated but forbidden. Buyer on another's transaction → **403**.
9. CSRF attacks rely on browsers auto-attaching cookies; a bearer token is attached explicitly by the client, so the vector doesn't exist. Acceptable because the API is stateless and token-based.

10. `SecurityConfig` (path rules + `hasRole`) and a `@PreAuthorize`/service ownership check. Say: "Path rules say ADMIN-only for resolve; the service additionally verifies the actor is the buyer or seller on that transaction, because roles alone don't stop one buyer touching another buyer's escrow."

If you can answer all 10, you can defend JWT against almost any fresher-level interviewer. Next: [07-react-javascript.md](#)

SECTION 07

07 — JAVASCRIPT + REACT (including the AlgoViz engine)

Time: 20 min. P0.

PART A — JAVASCRIPT, FROM ZERO

What problem does JavaScript solve?

HTML gives structure, CSS gives style — but neither can *react to a user*. JavaScript is the programming language of the browser: it runs when you click, type, or scroll, and it can modify the page and talk to servers.

SAY: "JavaScript is the language that runs in the browser and makes a page interactive. The same language also runs on the server through Node.js, which is why I can use it on both ends."

Where does it run? (say this, it explains a lot)

The browser gives JS three things: the **DOM** (the page as objects you can change), the **event loop** (a queue for callbacks like clicks and network responses), and **APIs** like `fetch`, `localStorage` and `setTimeout`. JS itself is single-threaded — it can only run one thing at a time.

Variables: `var`, `let`, `const`

```
var x = 1; // OLD: function-scoped, hoisted, can be redeclared → bug-prone
let y = 2; // block-scoped, reassignable
const z = 3; // block-scoped, cannot be reassigned (the binding, not the contents)
```

SAY: "I use `const` by default and `let` when I need to reassign. I avoid `var` because it's function-scoped rather than block-scoped, so it leaks out of loops and `if` blocks." TRAP: "Is `const` object immutable?" → "No — the binding can't change, but I can still push into a `const` array or change a property. For real immutability I'd freeze it or just not mutate it."

JS types: `number` (no separate int/float — everything is a double), `string`, `boolean`, `null`, `undefined`, `object`, `symbol`, `bigint`. Arrays and functions are objects. JS is **dynamically typed** — no declared types.

NOTE: `==` vs `===`: `==` coerces types (`'1' == 1` is `true`), `===` compares without coercion. **Always use `===`**. (Nice parallel to Java: `==` vs `equals`, but in JS the fix is `===`.)

Functions & arrow functions

```
function add(a, b) { return a + b; } // declaration
const add2 = (a, b) => a + b; // arrow, concise, returns implicitly
const greet = (name) => `Hi ${name}`; // template literal
```

SAY: "The key difference beyond syntax is `this`: an arrow function doesn't have its own `this`, it inherits it from where it was defined. That's why arrow functions are convenient in callbacks and React event handlers — no `.bind(this)` needed."

Arrays + the four methods you must know cold

```
const nums = [1, 2, 3, 4];

nums.map(n => n * 2);           // [2,4,6,8]   → transform each, SAME length
nums.filter(n => n % 2 === 0);  // [2,4]       → keep some, SHORTER
nums.reduce((acc, n) => acc + n, 0); // 10      → boil down to ONE value
nums.forEach(n => console.log(n)); //           → side effects, returns undefined

nums.find(n => n > 2);           // 3          → first match (or undefined)
nums.some(n => n > 3);           // true        → does ANY match
nums.every(n => n > 0);          // true        → do ALL match
nums.includes(3);               // true
[...nums].sort((a,b) => a-b);    // sort – note: sort MUTATES the original!
```

SAY: "map, filter and reduce return new arrays and don't mutate the original — that matters a lot in React, because mutating state breaks re-rendering detection. In AlgoViz I copy array state before each step rather than mutating it in place."

TRAP: "map vs forEach?" → "map returns a new array and is used for transformation; forEach is for side effects and returns undefined."

Objects, destructuring, spread

```
const user = { id: 42, name: 'Siva', role: 'BUYER' };

const { name, role } = user;           // destructuring
const { id: userId } = user;           // rename
const updated = { ...user, role: 'ADMIN' }; // shallow copy + override (immutable update)

const arr = [1, 2, 3];
const copy = [...arr, 4];               // spread → new array
```

SAY: "Spread makes a shallow copy. That's how I update React state without mutating: `setItems([...items, newItem])` creates a new array so React sees a different reference and re-renders." TRAP: "Is spread a deep copy?" → "No, it's one level deep. Nested objects are still shared references."

Async: callbacks → promises → async/await

The problem: network calls take time. If JS blocked waiting, the page would freeze. So the browser handles the waiting and calls you back when the data arrives.

```
// Promise style
fetch('/api/transactions/42')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// async/await – same thing, reads like synchronous code
async function loadTransaction(id) {
  try {
    const res = await fetch(`/api/transactions/${id}`, { headers: authHeader() });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error('Failed to load', err);
  }
}
```

```
}
```

A Promise is an object representing a value that will exist later. States: **pending** → **fulfilled** / **rejected**. **async** makes a function return a Promise; **await** pauses *that function* (not the whole program) until the promise settles. **try/catch** handles a rejected promise when using **await** — that's why **async/await** is easier to reason about than nested **.catch()** chains.

TRAP: "Is JavaScript single-threaded? Then how is **await** not blocking?" → "JS runs on one thread with an event loop. **await** suspends the async function and returns control to the event loop; callbacks run when the async work finishes. So the thread is never idle-blocked." TRAP: "**fetch** and HTTP errors?" → "**fetch** only rejects on network failure. A 404 or 500 still resolves, so I must check **res.ok** or **res.status** myself." — Very common real bug; saying it shows experience.

fetch vs axios: **fetch** is built in; **axios** has interceptors and automatic JSON parsing. Either is fine.

JSON: **JSON.parse(text)** → object; **JSON.stringify(obj)** → text. That's the same JSON format your Spring API sends and receives.

Module syntax

```
export default function AlgoVizPage() { }           // default export (one per file)
export function bubbleSortFrames(input) { }         // named export (many per file)
import AlgoVizPage from './AlgoVizPage';           // importing the default
import { bubbleSortFrames } from './algorithms';    // importing a named export
```

SAY: "ES modules let me split code into files with explicit imports and exports; Vite understands them natively during development."

PART B — REACT

What problem does React solve?

SAY: "With plain JavaScript you update the page by finding DOM elements and changing them directly. As the app grows, you end up with lots of code that mutates shared state and forgets to update some part of the UI. React solves this: I describe what the UI should look like for a given state, and React figures out the minimal DOM updates when the state changes."

Analogy: a spreadsheet. You change a number in one cell and the dependent cells recalculate. You don't manually walk around the sheet updating cells. With plain JS you're the one doing all the updating by hand; React is the spreadsheet engine.

The three core ideas

1. **Component** — a reusable, self-contained UI unit: a function that returns markup.
2. **Declarative rendering** — you write what the UI should look like for the current state.
3. **State-driven updates** — when state changes, React re-renders the affected components.

JSX

```
function EscrowCard({ tx }) {
  return (
    <div className="card">
      <h3>{tx.id}</h3>
      <p>Status: {tx.status}</p>          {/* JS expression in braces */}
      {tx.status === 'DELIVERED' && <button>Release</button>}  {/* conditional */}
    </div>
  );
}
```

- Looks like HTML but it's JavaScript — it compiles to `React.createElement(...)` (or the modern JSX runtime).
- Use `className`, not `class`, because `class` is a reserved JS word.
- Expressions go in `{}`; you can't put statements (`if`, loops) directly — use a ternary, `&&`, or `map`.

Component = function returning JSX

```
const Greeting = () => <h1>Hello</h1>;          // simple
<Greeting />                                   // components are used like tags and must be capitalised
```

NOTE: Why capitalised? "JSX treats lowercase tags as HTML elements and capitalised ones as my components."

Props vs State NOTE: (guaranteed question)

	Props	State
Who owns it	the parent	the component itself
Mutable by the child?	No — read-only	Yes, via the setter
Change triggers re-render?	yes (parent re-renders)	yes
Analogy	arguments passed to a function	the component's own memory

SAY: "Props are inputs passed down from a parent and are read-only — a component must never modify its own props. State is data the component owns and can change, and changing it triggers a re-render. The rule of thumb: if the value changes over time and affects what's rendered, it's state; if it comes from outside, it's props." Analogy: props = a function's parameters; state = a variable inside the function that persists between calls.

TRAP: "Can you mutate state directly?" → "No. `items.push(x)` mutates the array in place, the reference doesn't change, so React may not re-render. I always replace the value: `setItems([...items, x])`."

useState

```
const [count, setCount] = useState(0);          // [value, setter]
const [txs, setTxs] = useState([]);

setCount(count + 1);
setCount(prev => prev + 1);                      // functional update – safer with batching
```

SAY: "useState returns the current value and a setter. Calling the setter with a new value schedules a re-render. When the next value depends on the previous one I use the updater form, because React batches state updates and `count + 1` twice in a row would only add one." TRAP: "Is `setCount` synchronous?" → "No. It schedules the update; the new value is visible on the next render. That's why `console.log(count)` right after setting shows the old value."

useEffect

```
useEffect(() => {
  fetchTransactions().then(setTxs);
}, []); // [] → run once after the first render

useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup on unmount / before the next run
}, []);

useEffect(() => {
  // runs when `txId` changes
}, [txId]);
```

SAY: "useEffect runs side effects after render — fetching data, timers, subscriptions, touching `localStorage`. The dependency array controls when it re-runs: empty means once on mount, `[x]` means whenever `x` changes. If I set up a timer or subscription, I return a cleanup function to tear it down — otherwise I leak and end up with duplicated timers or stale updates after the component unmounts." TRAP: "What happens with no dependency array?" → "It runs after **every** render — usually a bug, and if it sets state you get an infinite loop." TRAP: "Why does my effect fetch twice in development?" → "React 18 StrictMode deliberately mounts, unmounts and remounts components in development to reveal missing cleanup. In production it runs once."

Other hooks you can honestly claim

- **useRef** — a mutable box that persists across renders but doesn't trigger a re-render.

```
const timerRef = useRef(null);
timerRef.current = setInterval(tick, speed); // hold a timer id
// also used for DOM access: <input ref={inputRef} /> then inputRef.current.focus()
```

SAY: "useRef holds a value across renders without causing a re-render — I use it to keep timer ids so I can clear the right interval, and to reference DOM nodes directly."

- **useReducer** — like `useState` for state with several related transitions.

```
function reducer(state, action) {
  switch (action.type) {
    case 'PLAY': return { ...state, playing: true };
    case 'PAUSE': return { ...state, playing: false };
    case 'SEEK': return { ...state, cursor: action.index };
    case 'SPEED': return { ...state, speed: action.value };
    default: return state;
  }
}

const [state, dispatch] = useReducer(reducer, { cursor: 0, playing: false, speed: 1 });
dispatch({ type: 'PAUSE' });
```

SAY: "useReducer is better than several `useState` calls when the states are related and have named transitions — you get one state object and one place that defines every legal change. That's exactly what I used for AlgoViz's playback: the cursor, playing flag and speed live in one reducer, so pause and seek can't

contradict each other."

- `useMemo/useCallback` (mention as optimisation, be honest): "I use them only when a computation is expensive or when a reference identity matters for a dependency array. Premature memoisation adds noise."

Lists and key NOTE:

```
{values.map((v, i) => <Bar key={v.id ?? i} value={v} />)}
```

SAY: "React needs a `key` to identify which list item is which between renders. Stable, unique keys let it move and update the correct DOM nodes. Using the array index as a key is a problem when the list is reordered or filtered — React can reuse the wrong element and local state attaches to the wrong row. In AlgoViz I give each value a stable id so a bar keeps its identity even when duplicate values swap places."

That last sentence is straight out of your case study ("Stable IDs: values retain identity when duplicates move"). It's a genuinely strong answer — remember it.

Conditional rendering

```
{loading && <Spinner />}
{error ? <ErrorBox msg={error} /> : <List items={items} />}
{items.length === 0 && <EmptyState />}
```

Events and forms

```
<button onClick={() => setPlaying(p => !p)}>Pause/Play</button>
<input value={speed} onChange={e => setSpeed(Number(e.target.value))} /> // controlled input
<form onSubmit={handleSubmit}>...</form> // then e.preventDefault() inside
```

SAY: "Event handlers receive a synthetic event. For controlled inputs the value comes from state and every keystroke updates it, so the state is always the single source of truth for the form."

Lifting state up (a favourite question)

SAY: "When two sibling components need to share state, I move it to their closest common parent and pass the value down as props and the setter down as a callback. That keeps one source of truth. In AlgoViz the playback cursor lives in the lab component, the controls dispatch actions to change it, and the visualization and timeline both render from it."

Props drilling / context

"Passing props down several levels is props drilling. React Context solves it for genuinely global values like the current user or theme. For most things, keeping state local or lifting it one level is simpler."

Rendering, the virtual DOM, and reconciliation

SAY: "When state changes, React calls the component function again, producing a new tree of elements in memory — the virtual DOM. It diffs that against the previous tree (reconciliation) and applies only the necessary changes to the real DOM, because real DOM updates are the expensive part." TRAP: "Is the virtual DOM faster than the DOM?" → "Not inherently — it's a strategy to avoid unnecessary real DOM work by batching and diffing. The win is doing *fewer* DOM operations."

React Router (P1 — a resume claim)

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/algorithms/:id" element={<AlgorithmPage />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
</BrowserRouter>
```

```
const { id } = useParams();           // read /algorithms/:id
const navigate = useNavigate();       // navigate('/algorithms/bubble')
const [params, setParams] = useSearchParams(); // ?speed=2
```

SAY: "React Router maps URLs to components so the app has real pages with shareable links and working back/forward buttons, instead of everything living on one screen. I use `useParams` for the algorithm id, `useSearchParams` to keep settings like the selected algorithm in the URL, and a catch-all `*` route for not-found." TRAP: "Is React a single-page application framework?" → "Yes — one HTML document, and the router swaps components in the browser. The URL changes via the History API, so it feels like navigation without a full page reload."

API calls from the frontend (how React talks to Spring Boot)

```
const API = import.meta.env.VITE_API_BASE_URL ?? '';

async function api(path, options = {}) {
  const res = await fetch(`${API}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(localStorage.getItem('token')
        ? { Authorization: `Bearer ${localStorage.getItem('token')}` } : {}),
      ...(options.headers ?? {}),
    },
  });
};

if (res.status === 401) { localStorage.removeItem('token'); throw new Error('Session expired'); }
if (!res.ok) throw new Error((await res.json()).message ?? `HTTP ${res.status}`);
return res.status === 204 ? null : res.json();
}
```

SAY: "All API calls go through one small helper. It attaches the bearer token, parses JSON, and turns non-2xx responses into errors — including treating 401 as 'session expired, log in again'. Having it in one place means the auth header and error handling can't drift between screens." **This is a great thing to show if they say "show me how the API is called."**

PART C — ALGOVIZ: THE ENGINE (your best technical story)

"What problem does AlgoViz solve?"

SAY: "Animation can show motion without showing reasoning. If you animate an algorithm *while* it runs, it's hard to pause at a specific operation, go back to an earlier state, or compare two algorithms. So I separated execution from presentation: the algorithm runs first and **records** every step as a frame, and playback just renders one already-recorded frame at a time."

The mental model (memorize)

```

INPUT (2–24 integers, validated)
  ↓
ALGORITHM RUNS ONCE      bubble/insertion/selection/merge/quick sort ...
  ↓
FRAMES ARRAY              each frame = a complete snapshot + a message,
                           e.g. { values: [7,3,6,2,5,1], comparing: [0,1],
                           swapped: false, sortedUpTo: -1,
                           message: "compare 7 and 3" }
  ↓
PLAYBACK REDUCER          { cursor, playing, speed } ← paused is just playing:false
  ↓
RENDERER                  renders frames[cursor] ← it never runs the algorithm

```

The engineering point to hammer

SAY: "Playback is not time travel. The renderer doesn't reverse anything — it indexes into a precomputed array. So pausing, seeking backwards and step-by-step are all the same operation: move the cursor and re-render."

Pause / resume / speed / step — exactly what happens

```

function reducer(state, action) {
  switch (action.type) {
    case 'PLAY': return { ...state, playing: true };
    case 'PAUSE': return { ...state, playing: false };
    case 'STEP': return { ...state, cursor: Math.min(state.cursor + 1, action.lastFrame) };
    case 'SEEK': return { ...state, cursor: action.index };
    case 'SPEED': return { ...state, speed: action.value };
    case 'RESET': return { ...state, cursor: 0, playing: false };
    default: return state;
  }
}

```

```

useEffect(() => {
  if (!state.playing) return; // paused → no timer exists at all
  if (state.cursor >= frames.length - 1) {
    dispatch({ type: 'PAUSE' }); return; // stop at the end
  }
  const delay = 1000 / state.speed; // speed 2x → 500ms per frame
  const id = setTimeout(() => dispatch({ type: 'STEP', lastFrame: frames.length - 1 }), delay);
  return () => clearTimeout(id); // cleanup on every dependency change
}, [state.playing, state.cursor, state.speed, frames.length]);

```

SAY: "When the user clicks PAUSE, `playing` becomes false, the effect cleans up and no timer is scheduled — so time stops. Resume sets `playing` true and the timer starts again from the same cursor. Speed is just the delay between frames: 1x is 1000ms, 2x is 500ms. Step-by-step is the same STEP action without the timer. All of them are the same state machine, which is why pause/resume at any point is consistent."

Follow-ups you must survive

"What if the user clicks pause twice?" → "It's idempotent — the reducer just sets `playing` to `false` again and no timeouts exist to double-cancel. There's no counter to corrupt, which is exactly why the state is a reducer rather than ad-hoc booleans." "What if the user drags the timeline while playing?" → "`SEEK` updates the cursor; the effect's cleanup cancels the pending timeout and a new one is scheduled for the new cursor, so there's never a stale frame applied." "Why not animate the algorithm as it runs?" → "Because then the visualization and the computation are entangled: pausing means pausing the algorithm, and going backwards is impossible. Recording first makes every playback control trivial and makes the trace inspectable — you can compare recorded operation counts between algorithms." "Why cap the input at 24 numbers?" → "Full snapshots cost memory. A quadratic algorithm copying a full array per frame grows fast — roughly cubic trace storage in the worst case. I bounded the input so the cost stays manageable and honest rather than pretending it scales; compact events or checkpoints are the future fix." "What's the trade-off you accepted?" → "Memory for simplicity and inspectability. I documented the boundary instead of hiding it. That's a deliberate engineering decision, not an oversight."

Reusable components (the resume claim)

SAY: "The sorting lab is shared by every sorting algorithm because they all produce the same frame shape. `SortCanvas` renders arrays as bars, `PlaybackControls` owns play/pause/step/speed, `Timeline` seeks, and `AlgorithmPicker` selects the module. Adding an algorithm means adding a frame generator, not a new UI. The non-sorting modules — grid search, tree traversal, two pointers — have different visual state, so they keep their own renderers; I'd rather have that than force everything through one generic renderer and make it worse at both."

This is a genuinely mature answer — it shows you know where not to abstract. NOTE: Honest limit from your own case study: **don't claim Dijkstra or Floyd's cycle detection are modules** — your case study says they aren't. And don't claim it's a general-purpose code debugger or a benchmark.

Why React Router, Tailwind, Vite here

- **Router:** "Each algorithm gets its own URL so a student can share a link to bubble sort, and lazy loading keeps the initial bundle small."
- **Tailwind:** "Utility classes meant I could build a responsive layout without inventing class names or shipping a large custom CSS file; consistency came from reusing the same spacing and colour scale."
- **Vite:** "Instant dev server startup and hot module replacement, so iterating on a visualization is fast, plus a simple production build."

SAY: "Why is AlgoViz not just a static animation?"

"Because the frames are generated from the actual algorithm implementation, with the real array state at every comparison and swap — a static animation just replays a hand-authored clip. And because playback is data-driven, I can seek to any point, compare operation counts between algorithms, and run the same frame engine for any input within the cap. The algorithm isn't drawn; it's recorded."

SELF-TEST

1. Props vs state — who owns what, and which is mutable?
2. Why not mutate state directly? What do you do instead?
3. `useEffect` dependency array — the three cases.
4. Why do keys matter, and what's wrong with the array index as a key?
5. What does `useReducer` give you over several `useState` calls?
6. How does pause work in AlgoViz, mechanically?
7. How does speed adjustment work?
8. `map` vs `filter` vs `reduce` vs `forEach`.
9. What is `await` actually doing, and does it block the browser?

10. In AlgoViz, why record frames instead of animating execution?

Key

1. Props come from the parent (read-only); state is owned by the component and is mutable via its setter.
2. The reference doesn't change so React may not re-render; replace with a new object/array.
3. Empty → once on mount; `[x]` → when `x` changes; absent → every render.
4. Keys identify list items across renders; index keys mis-associate DOM nodes and state when the list reorders.
5. One state object with named transitions in one place — impossible states are avoided.
6. `playing=false` → the scheduling effect runs, returns early, and its cleanup has cleared the timer. No timer, no advance.
7. The delay between frames is `1000 / speed`.
8. Transform (same length) / keep some (shorter) / one value / side effects only.
9. It suspends that async function and returns control to the event loop; no, the UI stays responsive.
10. Recording decouples compute from presentation, makes seeking/pausing/stepping trivial and traces comparable; the cost is memory, so the input is bounded.

Next: `08-html-css-tailwind-vite.md`

SECTION 08

08 — HTML, CSS, TAILWIND, VITE

Time: 10 min. P1/P2 — low effort, easy marks. If time is short: read the Tailwind + Vite sections and the "how I used it" lines.

PART A — HTML

SAY: "HTML describes the structure and meaning of a page using elements. It isn't a programming language — it's a markup language. CSS styles it, and JavaScript makes it interactive."

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>AlgoViz</title>
  </head>
  <body>
    <header>
      <nav><a href="/algorithms">Algorithms</a></nav>
    </header>
    <main>
      <section aria-labelledby="lab-title">
        <h1 id="lab-title">Sorting Lab</h1>
        <button type="button" aria-label="Pause playback">Pause</button>
      </section>
    </main>
    <footer>© 2026</footer>
  </body>
</html>
```

Key ideas:

- **Elements** = `<tag>content</tag>`; some are self-closing (`<img />`, `<input />`).
- **Attributes** = extra info on the tag: `href`, `src`, `id`, `class`, `type`, `aria-*`.
- **Semantic HTML** = using `<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`, `<button>` rather than `<div>` everywhere. SAY: "Semantic elements tell browsers, search engines and screen readers what the content *is*, not just how it looks. It gives accessibility for free, and it's the reason I use `<button>` for actions and `<a>` only for navigation."

TRAP: "`<div>` vs `<button>` for a clickable control?" → "A `button` is focusable, keyboard-activatable and announced correctly by screen readers. A `div` needs `role`, `tabindex` and key handling re-implemented. I use the native element."

PART B — CSS

SAY: "CSS describes how the page looks. You select elements and set properties on them; the browser computes the final layout from all the rules."

Selectors

```
h1           { } /* element */
.card        { } /* class – reusable, my default */
#submit-btn  { } /* id – unique, avoid for styling (too specific) */
nav a        { } /* descendant */
.card > .bar { } /* direct child */
button:hover { } /* pseudo-class */
input:focus  { }
li:nth-child(2) { }
```

The box model (a classic question)

Every element is a box: **content** → **padding** → **border** → **margin**, outside in.

```
.box {
  box-sizing: border-box; /* width INCLUDES padding + border – set globally */
  width: 320px;
  padding: 16px;
  border: 1px solid #ddd;
  margin: 24px;
}
```

SAY: "Content is the actual thing; padding is space inside the border; the border surrounds it; margin is space outside. With `box-sizing: border-box` the declared width includes padding and border, which makes layouts predictable — that's why it's usually set globally." TRAP: "Margin vs padding?" → "Padding is inside the border and takes the element's background; margin is outside and is invisible spacing."

Display & positioning

```
display: block; /* full width, stacks (div, p, h1) */
display: inline; /* flows with text, no width/height (span, a) */
display: inline-block; /* flows but accepts sizing */
display: flex; /* 1-D layout ← what I use most */
display: grid; /* 2-D layout ← good for card grids */
display: none; /* removed from layout */

position: static | relative | absolute | fixed | sticky;
```

SAY: "`relative` keeps the element in flow and gives me a positioning context; `absolute` takes it out of flow and positions it relative to the nearest positioned ancestor; `fixed` is relative to the viewport; `sticky` toggles between relative and fixed on scroll."

Flexbox (know these four properties)

```
.row {
  display: flex;
  justify-content: space-between; /* along the MAIN axis */
  align-items: center; /* along the CROSS axis */
  gap: 12px;
  flex-wrap: wrap;
}
.row .grow { flex: 1; } /* grow / shrink / basis */
```

SAY: "Flexbox is one-dimensional — I use it for rows and toolbars. `justify-content` works on the main axis and `align-items` on the cross axis; with the default `flex-direction: row`, that means horizontal and vertical respectively." TRAP: "Why does `justify-content: center` not vertically centre?" → "Because it acts on the main axis. To centre vertically in a row I need `align-items: center`, or I change the direction and swap them."

Grid

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}
```

SAY: "Grid is two-dimensional, so it's what I use for a card layout that has to reflow to different column counts. `auto-fit` with `minmax` gives me a responsive card grid without a single media query."

Responsive design & media queries

```
@media (max-width: 640px) {
  .sidebar { display: none; }
}
```

SAY: "Responsive means the layout adapts to the screen instead of assuming a desktop width. I use a fluid layout first — percentages, flex, grid, relative units — and then `@media (max-width: ...)` for layout changes that can't be fluid. The `viewport` meta tag is required for this on mobile, and I size from mobile up."

Specifics from your projects: AlgoViz and the portfolio both use a bounded visualization that stays readable on a phone, plus reduced-motion support — `@media (prefers-reduced-motion: reduce)` to stop animations for users who've asked the OS for that. Mentioning `prefers-reduced-motion` is a genuine accessibility win.

Specificity (one line)

SAY: "When rules conflict, inline styles beat ids, ids beat classes, classes beat element selectors — and later rules win if specificity ties. I keep specificity low by styling with single classes, which is also what makes Tailwind painless."

PART C — TAILWIND CSS (P1 — resume claim)

SAY: "**Utility-first CSS means instead of writing a class name like `.card` and defining its styles in a stylesheet, I compose small single-purpose classes directly in the markup.**"

```
<div class="flex items-center justify-between gap-4 rounded-lg border border-slate-200 p-4 shadow-sm">
  <h3 class="text-lg font-semibold text-slate-900">Bubble Sort</h3>
  <button class="rounded bg-indigo-600 px-3 py-1.5 text-sm font-medium text-white
    hover:bg-indigo-500 focus-visible:outline focus-visible:outline-2">
    Run
  </button>
</div>
```

Each class does exactly one thing: `flex`, `items-center`, `gap-4`, `rounded-lg`, `p-4`, `text-lg`.

Why use it?

SAY: "Three reasons. First, I never have to invent class names or hunt for where a style is defined — the styling is visible where it's used. Second, a design scale is enforced: spacing and colour come from the framework's scale, so the UI stays consistent instead of accumulating arbitrary pixel values. Third, unused CSS is removed at build time, so the production stylesheet only contains what I actually used." **Bonus you should say:** "Any repeated pattern I extract into a reusable React component, so the duplication is solved at the component level instead of with a CSS class."

Disadvantages (saying these makes you credible)

SAY: "The markup gets verbose, and complex components end up with long class strings. There's a learning curve for the class names, and if the same long string is repeated across files without being componentised, it's harder to change globally. For heavy design-heavy work, a mix of utility classes plus a component library or a few custom classes is reasonable."

How you used it

"In AlgoViz, Tailwind handled the layout and responsive behaviour of the lab — the bar canvas, the playback controls and the timeline — so I could share spacing and colour consistently across modules without writing a large custom stylesheet. Repeated combinations became components rather than duplicated class strings."

TRAP: "Is Tailwind just inline styles?" → "No. Inline styles can't do media queries, hover/focus states or pseudo-elements, and they have maximal specificity. Tailwind classes are real CSS rules with variants — `md:`, `hover:`, `focus-visible:`, `dark:` — and they follow a shared scale."

PART D — VITE (P1 — resume claim)

SAY: "**Vite is the build tool and dev server for my React projects. It gives me an instant dev server with hot module replacement, and an optimised production build.**"

What problem does it solve?

Before Vite, tools like Webpack bundled your *entire* application before the dev server could serve anything — so startup got slower as the project grew. Vite takes a different approach in development.

How it works

- **Dev:** Vite serves your source files as **native ES modules** — the browser requests modules itself, and Vite transforms only the file that's requested, on demand. So there's no upfront bundling, and startup is nearly instant regardless of project size.
- **HMR (Hot Module Replacement):** when you save a file, Vite invalidates only that module and pushes the update over a WebSocket. The browser swaps in the new module **without reloading the page**, so component state can survive an edit.
- **Dependencies:** the few big libraries (React, etc.) are pre-bundled once with esbuild and cached, because many small requests to `node_modules` would be slow.
- **Build:** for production, Vite switches to **Rollup** and produces an optimised bundle with tree-shaking, minification and hashed filenames for caching.
- **Env vars:** `import.meta.env.VITE_API_URL` — only `VITE_`-prefixed variables are exposed to the client, which prevents accidentally shipping server secrets.

SAY: "**Why Vite over Create React App?**"

"CRA bundled everything with Webpack up front, so startup and rebuilds got slow as the app grew, and it wasn't being maintained. Vite serves ES modules on demand in dev, so it starts in milliseconds and hot-reloads just the changed module, and it uses Rollup for a fast optimised build. It's also the default for the current React tooling."

How you used it

"Both RevertPay's frontend and AlgoViz are React apps on Vite. I use it for the dev server with HMR while iterating on visualizations, the production build, and `import.meta.env` to configure the API base URL so the frontend isn't hard-coded to localhost."

TRAP: "Is Vite a framework?" → "No. It's a build tool and dev server. React is the framework; Vite serves and bundles the code."

Sequence to remember (React + Vite):

```
index.html → <div id="root"> → main.jsx → ReactDOM.createRoot(document.getElementById('root'))  
                                     .render(<App />)  
→ App.jsx renders components; the router swaps pages;  
Vite's dev server handles module requests + HMR
```

SELF-TEST

1. What is semantic HTML and why care?
2. Box model in four words.
3. Flexbox main axis vs cross axis — which property for each?
4. Grid vs flexbox — when do you choose which?
5. What is utility-first CSS?
6. Two advantages and one disadvantage of Tailwind.
7. What does Vite do in dev that Webpack didn't?
8. What is HMR and why does it matter for AlgoViz?

Key

1. Elements that describe meaning (`nav`, `main`, `button`) → accessibility, SEO, keyboard behaviour for free.
2. Content, padding, border, margin.
3. Main = `justify-content`, cross = `align-items`.
4. Flex = one-dimensional rows/columns; Grid = two-dimensional layouts like card grids.
5. Small single-purpose classes composed in the markup instead of named component classes.
6. No naming/hunting, enforced design scale, tree-shaken output / verbosity in markup.
7. Serves native ES modules on demand (no up-front bundling) with esbuild pre-bundling of deps.
8. It swaps only the changed module over a WebSocket without a page reload — so I keep the current visualization state while editing a component.

Next: `09-node-express-flask.md`

SECTION 09

09 — NODE.JS, EXPRESS, FLASK, PYTHON

Time: 12 min. P1.

PART A — NODE.JS

What problem does it solve?

JavaScript was originally trapped in the browser — it couldn't touch the file system, open a network port, or run as a program. **Node.js took the V8 JavaScript engine out of the browser and put it in a runtime where JS can run on a server.**

SAY: "Node.js is a runtime that lets me run JavaScript on the server, outside the browser. It gives JS access to the file system and the network, which the browser sandbox doesn't."

Key properties:

- **Single-threaded + non-blocking I/O.** Node runs your JS on one thread, but I/O (network, files, database) is handed to the OS and completed asynchronously. While one request waits on the database, Node serves the next. That's the "event loop".
- **Great for I/O-bound work** — APIs, real-time apps, websockets. **Not ideal for CPU-bound work** — one heavy computation blocks everything (that's what worker threads or a separate service are for).
- **npm** — the package manager and the world's largest package registry.
- **package.json** — manifest: dependencies, scripts, version. `node_modules/` holds installed packages; `package-lock.json` pins exact versions for reproducible installs.

TRAP: "Is Node a language/framework?" → "It's a **runtime**. The language is JavaScript; Express is the framework."

PART B — EXPRESS (resume claim: Node.js, Express.js)

SAY: "Express is a minimal web framework for Node. It gives me routing — mapping a method and path to a handler — and middleware, a chain of functions that process a request before it reaches the handler."

Routes

```
const express = require('express');
const app = express();

app.use(express.json()); // middleware: parse JSON bodies into req.body

app.get('/api/transactions', (req, res) => {
  const { status, page = 0 } = req.query; // query params
  res.json({ items: [], page: Number(page) });
});

app.get('/api/transactions/:id', (req, res) => {
  const tx = db.find(req.params.id); // path param
  if (!tx) return res.status(404).json({ message: 'Not found' });
  res.json(tx);
});

app.post('/api/transactions', (req, res) => {
  const created = db.create(req.body); // request body
  res.status(201).json(created);
});

app.listen(3000, () => console.log('listening'));
```

Notice the shape is **identical to your Spring controllers** — method + path → handler, path params, query params, JSON body, status codes. **Say that out loud in the interview**; it shows you understand concepts rather than syntax:

"The routing model is the same as Spring's `@GetMapping/@PostMapping`, just with a function instead of an annotated method — so the REST design carries over."

Middleware (the concept to know)

SAY: "Middleware is a function that runs in order for each request: (`req`, `res`, `next`). It can read or modify the request, end the response, or call `next()` to pass control to the next function. That's how I'd add authentication, logging, validation or error handling."

```
function requireAuth(req, res, next) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Authentication required' }); // stops the chain
  }
  try {
    req.user = verifyToken(header.slice(7));
    next(); // continue to the route handler
  } catch {
    res.status(401).json({ message: 'Invalid token' });
  }
}

app.use('/api/escrows', requireAuth);
```

SAY: "This is conceptually the same idea as Spring's filter chain — Spring Security's filters are middleware in Java."

Error handling

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status ?? 500).json({ message: err.message });
});
```

SAY: "Express recognises an error handler by its four parameters, and it runs when a handler calls `next(err)` or throws. It's the equivalent of my Spring `@RestControllerAdvice`."

SAY: "HOW IS EXPRESS DIFFERENT FROM SPRING BOOT?" (they asked for this — learn it)

"They solve the same problem — building a REST API — with different philosophies."

Spring Boot is a full, opinionated framework for the JVM. It has dependency injection and an IoC container, a layered structure with controllers/services/repositories, Spring Data JPA for persistence, Spring Security for auth, and an embedded Tomcat. It's compiled, statically typed Java, so many errors are caught at compile time. It scales well for large enterprise codebases, but there's more structure and more to learn.

Express is a minimal, unopinionated runtime for Node. There's no built-in DI container, no ORM, no security layer — I add middleware and libraries as needed, and the whole thing is a few hundred lines. It's fast to start and great for small services and APIs, but the architecture is on me, and JavaScript's dynamic typing means mistakes show up at runtime.

My preference for a big business domain like escrow and money is Spring Boot: strong typing, transactions and JPA give me safety. Express is what I'd reach for in a small service or when the team is already JavaScript."

TRAP: "Which is faster?" → "Node/Express benchmarks higher on raw throughput for simple I/O-bound endpoints, but for real applications the database and business logic dominate, and correctness tools like transactions matter more than raw speed."

NOTE: **Be honest about depth.** Your resume lists Node and Express, and your portfolio is literally a Node project with Vercel serverless functions, validation, rate limiting and tests. So you can legitimately say:

"I've used Node with plain `node --test` and Vercel serverless functions, plus Express-style routing, for smaller services. Spring Boot is where I've built the more complex business logic." **Do not claim** you built a large Express backend if you didn't — the follow-up questions will be Spring-shaped and you'd be out of your depth.

PART C — PYTHON + FLASK (EDU2JOB's model service)

Why Python?

SAY: "Python is a readable, dynamically typed language with an enormous data-science ecosystem. For anything ML-related, the libraries I need — scikit-learn, Pandas, NumPy — are Python-first, so the model lives in Python even though the application logic is Java."

Python basics you should be able to speak to (they won't go deep):

```
x = 10                                # no type declaration, dynamically typed
name: str = "Siva"                    # type hints are optional and not enforced at runtime
def greet(name): return f"Hi {name}"  # significant indentation, no braces
nums = [1, 2, 3]                      # list → ordered, mutable (like Java ArrayList)
tup = (1, 2)                          # tuple → immutable
s = {1, 2}                            # set → unique
d = {"a": 1}                          # dict → key/value (like Java HashMap)
squares = [n*n for n in nums]         # list comprehension
try: ... except Exception as e: ...   # error handling
```

TRAP: "Python list vs tuple?" → "A list is mutable and used for collections you change; a tuple is immutable, so it's hashable and safe to use as a dict key or a fixed record."

What is Flask?

SAY: "Flask is a small Python web framework. Like Express, it's minimal: I define routes and return JSON. It's a natural fit for wrapping a trained model in an HTTP endpoint, because that's all the service needs to do."

```
from flask import Flask, request, jsonify
import joblib, numpy as np

app = Flask(__name__)
model = joblib.load("model/random_forest.joblib")  # loaded ONCE at startup

@app.get("/health")
def health():
    return jsonify(status="ok")

@app.post("/predict")
def predict():
    payload = request.get_json(silent=True) or {}
    try:
        features = build_features(payload)          # encode + scale, same as training
    except SchemaError as e:
        return jsonify(error=str(e)), 400           # invalid input
    proba = model.predict_proba([features])[0]      # class probabilities
    ranked = sorted(
        ({ "role": label, "confidence": float(p)} for label, p in zip(model.classes_, proba)),
        key=lambda r: r["confidence"], reverse=True
    )[:5]
    return jsonify(predictions=ranked), 200
```

SAY: **Key points about this endpoint:**

"The model is loaded once at startup, not per request — loading it inside the handler would make every prediction slow. `predict_proba` gives a probability per class, which I sort to return the top five ranked roles. Preprocessing is applied inside the service using the *same* encoders that were fitted during training, because if the encoding differs, the numbers mean something different to the model. Invalid input returns 400 instead of crashing."

NOTE: **This is the number-one real bug in ML services.** The fit-time and serve-time transformations must match. Say it — it's exactly the issue your case study describes.

SAY: "How would Spring Boot communicate with Flask?" (they asked for this)

```
@Service
public class PredictionClient {

    private final RestClient restClient; // Spring 6+ RestClient (RestTemplate/WebClient are alternatives)

    public PredictionClient(RestClient.Builder builder,
                           @Value("${ml.service.url}") String baseUrl) {
        this.restClient = builder.baseUrl(baseUrl).build();
    }

    public PredictionResponse predict(StudentProfile p) {
        try {
            return restClient.post()
                .uri("/predict")
                .contentType(MediaType.APPLICATION_JSON)
                .body(toFeaturePayload(p))
                .retrieve()
                .body(PredictionResponse.class);
        } catch (RestClientException ex) {
            throw new PredictionServiceUnavailableException(ex); // → 503
        }
    }
}
```

SAY: "Spring calls the Flask service like any other HTTP API — a POST with the student's features as JSON, and the ranked roles come back as JSON. The Flask URL is a config property, not hard-coded, and if the call fails I raise a service-unavailable error so the client gets a clean 503 instead of a 500 stack trace. The browser never talks to Flask; only Spring does."

NOTE: **Honest boundary from your own case study:** the README lists **timeouts and an externalised ML-service URL as follow-up work**. So the safe phrasing is:

"The call is a plain REST call from the Spring service; configuring an explicit timeout and moving the URL fully into configuration are the next hardening steps — right now a slow model response is a known risk."

Never claim an async queue, a circuit breaker, or a gRPC boundary. Not on your resume, and not implemented.

SAY: "Why two runtimes instead of doing everything in Java?" (very likely question)

"Because the model ecosystem is Python — scikit-learn, Pandas and NumPy are where this work happens, and rewriting a trained model in Java would be throwaway effort. Java is better for the web application: strong typing, transactions, JPA and Spring Security. Splitting them means each side uses the right tool. The cost is a network hop, a second service to run, and a contract that has to stay aligned — so I keep the boundary explicit: the browser only calls Spring, and Python only does feature transformation and inference."

Follow-up traps:

- "What if Flask is down?" → "Spring raises an exception, I map it to 503 and log it; the UI shows that predictions are temporarily unavailable. The rest of the app keeps working because auth, profiles and history don't depend on the model."
- "What if the model is slow?" → "Set an explicit client timeout so a slow model doesn't tie up request threads, and surface a 503. Adding a cache for repeated identical inputs is a reasonable next step."
- "Why not call Flask directly from React?" → "Then the model service would need its own auth and input validation and would be exposed publicly, and the prediction history and user context wouldn't be saved. Keeping one entry point preserves a single security and business boundary."

SELF-TEST

1. What is Node.js, in one sentence, and what is it *not*?
2. What is middleware? Give one that stops a request.

3. Spring Boot vs Express — three real differences.
4. Why does EDU2JOB have both Java and Python?
5. Where is the model loaded in the Flask service, and why does that matter?
6. What breaks if preprocessing at prediction time differs from training time?
7. What happens if Flask is unreachable?

Key

1. A runtime that runs JavaScript outside the browser with filesystem/network access; not a language or framework.
2. A `(req, res, next)` function in a chain; the auth middleware returns 401 instead of calling `next()`.
3. DI container + opinionated structure + compile-time typing & transactions vs minimal/runtime-typed/flexible.
4. Python owns the ML ecosystem; Java owns typing, transactions, JPA, security.
5. Once at startup; otherwise every request pays the load cost.
6. The model receives numerically different values and produces wrong predictions — silently, with no exception.
7. Spring catches the failure, returns 503, logs it; the rest of the app still works.

Next: [10-ml-pandas-numpy.md](#)

SECTION 10

10 — MACHINE LEARNING: SCIKIT-LEARN, RANDOM FOREST, PANDAS, NUMPY, PREPROCESSING

Time: 18 min. P1 — this is EDU2JOB's brain and a HIGH danger claim.

STEP 0 — WHAT IS MACHINE LEARNING?

Traditional programming vs ML

SAY: "In normal programming I write the rules: input plus my logic produces the output. In machine learning I give the computer many examples of input and output, and it *learns* the rule — the model — that maps one to the other. Then I apply that rule to new inputs it has never seen."

Analogy: a rulebook vs experience. A rulebook says "if CGPA > 8 and the person knows Java, suggest backend roles". A person who has seen thousands of placements learns that pattern implicitly and can apply it to a new student.

Traditional: data + RULES → answers

Machine learning: data + ANSWERS (labels) → RULES (a model) → answers on new data

Vocabulary (memorize — every follow-up uses these words)

Term	Meaning	EDU2JOB example
Dataset	the collection of examples	historical student records with their job roles
Feature	an input variable	degree, major, CGPA, skills
Label / target	what we predict	the job role
Sample / row	one example	one student
Training	the model learns from data	fitting the forest
Testing / validation	evaluate on unseen data	held-out split
Model	the learned function	the Random Forest
Prediction / inference	applying the model to new input	the ranked roles for a new student
Overfitting	memorising training data, failing on new data	a perfect training score but bad real predictions
Underfitting	too simple to capture the pattern	always predicting the most common role

Classification vs regression

- **Classification** = predict a **category** → EDU2JOB: which job role. (Even with probabilities, the task is classification.)
- **Regression** = predict a **number** → e.g. predicting salary. SAY: "EDU2JOB is multi-class classification, because the output is one of several discrete roles."

Why accuracy alone is a bad metric

SAY: "If 70% of the training data is one job role, a model that always predicts that role gets 70% accuracy and is useless. That's why I'd look at per-class precision and recall, a confusion matrix, and for ranked output, **top-k accuracy** — whether the correct role appears in the top five. For a recommendation like this, top-k is the honest metric." (Your case study lists top-k accuracy and calibration metrics as planned evaluation work — so you can say you know *what* to measure without claiming you measured it.)

PART A — NUMPY

What problem does it solve?

SAY: "Python lists are flexible but slow for numeric work, because each element is a full Python object and operations loop in Python. NumPy gives me a fixed-type, contiguous array where the operations run in compiled C, plus broadcasting so I can apply an operation to an entire array without writing a loop."

```
import numpy as np

a = np.array([1, 2, 3, 4])          # dtype=int, contiguous memory
a * 2                             # array([2, 4, 6, 8]) – vectorised, no Python loop
a[a > 2]                           # array([3, 4])          – boolean masking
a.mean(), a.std()                  # statistics without loops
m = np.array([[1,2],[3,4]])        # 2-D array (matrix)
m.T                                # transpose
np.dot(m, m)                       # matrix multiplication
```

In ML: "scikit-learn consumes NumPy arrays internally. `model.predict(X)` expects a 2-D array of shape `(n_samples, n_features)`, so my preprocessing output ends up as a NumPy array. Pandas DataFrames convert to that with `.to_numpy()` or `.values`."

TRAP: "Why is NumPy fast?" → "Fixed dtypes in contiguous memory, vectorised operations implemented in C instead of Python loops, and no per-element object overhead." TRAP: "What's broadcasting?" → "Rules that let arrays of compatible shapes combine without me manually copying data — e.g. subtracting a per-feature mean vector from a whole matrix."

Shape bug to avoid (very common): `model.predict([[1,2,3]])` vs `model.predict([1,2,3])` — sklearn wants a 2-D array, one row per sample.

PART B — PANDAS

SAY: "Pandas is the library for tabular data in Python. Its main object is the **DataFrame** — a table with named columns and a row index — and a **Series** is a single column. It's what I use to load, inspect, clean and transform data before it becomes a NumPy array for the model."

Analogy: an in-memory spreadsheet with a programming interface, plus SQL-like operations.

```
import pandas as pd

df = pd.read_csv("students.csv")

df.head()                          # first 5 rows
df.shape                           # (rows, columns)
df.info()                          # dtypes + non-null counts
df.describe()                      # numeric summary: mean, std, min, max

df["cgpa"]                         # one column (a Series)
df[["degree", "major", "cgpa"]]    # several columns
df[df["cgpa"] > 8.0]               # filter rows (boolean mask)
```

```
df[(df["degree"] == "BTech") & (df["cgpa"] > 8)] # combine conditions
df.isna().sum() # count missing per column
df["cgpa"] = df["cgpa"].fillna(df["cgpa"].median()) # fill missing
df = df.dropna(subset=["degree"]) # drop rows missing a key field

df["cgpa_scaled"] = (df["cgpa"] - df["cgpa"].mean()) / df["cgpa"].std() # normalise
df["skills"] = df["skills"].fillna("").str.split("|") # parse multi-valued

df.groupby("role")["cgpa"].mean() # SQL-like aggregation
df.sort_values("cgpa", ascending=False)
merged = pd.merge(students, roles, on="role_id", how="left") # like a JOIN
df.to_csv("clean.csv", index=False)
```

In ML, Pandas does: loading, inspection, cleaning, type conversion, encoding, and train/test splitting of the columns. Then `.to_numpy()` → NumPy → sklearn. TRAP: "Why not just use lists?" → "Named columns, mixed dtypes, missing-value handling, group-by and joins — rewriting that by hand is error-prone." TRAP: "SettingWithCopyWarning?" → "I was modifying a slice rather than a copy. The fix is `.copy()` or using the proper assignment path — it's the classic Pandas gotcha."

PART C — DATA PREPROCESSING (from your resume: "Data Preprocessing")

Raw data is messy

SAY: "Real data has missing values, inconsistent types, free-text categories, different scales and duplicated rows. Also, a model can only compute with numbers — it cannot multiply 'BTech' by a weight. Preprocessing turns messy raw data into a clean, numeric, consistently-scaled feature matrix."

Analogy: cooking. The ingredients arrive unwashed and unchopped. The stove (model) can only cook prepared ingredients, and all pieces need to be cut to comparable sizes.

The steps (in order)

Step	What	Why
1. Inspect	<code>head()</code> , <code>info()</code> , <code>describe()</code> , missing counts	understand dtypes, ranges, gaps before touching anything
2. Clean	handle missing values, drop duplicates, fix types	models fail or learn garbage from NaNs
3. Encode categoricals	text → numbers	the model needs numbers
4. Scale numerics	CGPA into a comparable range	distance/impurity-based splits shouldn't be dominated by a large-scale feature
5. Expand multi-valued fields	"Java,SQL,React" → binary columns	each skill becomes its own boolean feature
6. Split	train / test (fit transforms on train only)	honest evaluation
7. Fit the model	on the transformed training matrix	

Encoding, three ways

```
# One-hot encoding – a column per category (no false ordering)
# degree: BTech → [1, 0, 0] ; MCA → [0, 1, 0] ; BSc → [0, 0, 1]
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(handle_unknown="ignore")
# handle_unknown="ignore" → an unseen category at prediction time becomes all-zeros
# instead of crashing. Critical for a live service.

# Ordinal encoding – only where order genuinely exists
# "beginner/intermediate/advanced" → 0/1/2

# Multi-label skills → MultiLabelBinarizer
from sklearn.preprocessing import MultiLabelBinarizer
mlb = MultiLabelBinarizer()
X_skills = mlb.fit_transform([["Java","SQL"], ["React","JS"], ["Java"]])
# → [[1,1,0,0], [0,0,1,1], [1,0,0,0]] with mlb.classes_ = ['Java','JS','React','SQL']
```

SAY: **"Why not just map categories to 1, 2, 3?"**

"Because that invents an order and a distance the data doesn't have — the model would think 'BSc is between BTech and MCA' and that the gap between categories has meaning. That's wrong for a nominal category. I use one-hot so each category gets its own independent flag. Ordinal encoding is only correct where the order is real — like beginner → advanced."

Scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler() # (x - mean) / std → mean 0, std 1
X_train_scaled = scaler.fit_transform(X_train) # FIT on train only
X_test_scaled = scaler.transform(X_test) # TRANSFORM with the train statistics
```

SAY: **"Why fit on the training set only?"** → "Because `fit_transform` on the whole dataset computes the mean and standard deviation using the test data too — that's **data leakage**. The model would indirectly see information from the test set and my evaluation would be optimistic. The rule is: fit the transformers on training data, then transform everything else with the fitted objects — including in production." **Being able to say "data leakage" and explain it is a strong signal.** Memorize this answer.

IMPORTANT THE MOST IMPORTANT PREPROCESSING POINT FOR YOUR PROJECT

SAY: "The preprocessing must be identical at training time and at prediction time. So I persist the fitted encoders and scaler alongside the model and apply exactly the same transformations in the Flask service. If the Java boundary sends a category the encoder never saw, `handle_unknown='ignore'` produces a safe all-zeros vector instead of an exception. If the transformations drifted apart, the model would still return a confident prediction — just a meaningless one, and nothing would raise an error. That's why I keep preprocessing *next to* the model artifact rather than reimplementing it in Java." (This is the core insight of your own case study's "Challenges" section. **Use it.**)

Class imbalance (bonus, brief)

SAY: "If some job roles appear far more often in the data, the model can favour them. Options are resampling, class weights, or simply reporting per-class metrics so the imbalance is visible instead of hidden behind accuracy."

PART D — SCIKIT-LEARN

SAY: "scikit-learn is the standard Python library for classical machine learning. It provides preprocessing, model implementations, and evaluation tools behind a **consistent API**: every model has `fit`, `predict` and often `predict_proba`, and every transformer has `fit/transform`. That consistency is the point — I can swap a Random Forest for logistic regression by changing one line."

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, top_k_accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)
model = RandomForestClassifier(n_estimators=300, max_depth=None,
                              random_state=42, class_weight="balanced")
model.fit(X_train, y_train) # TRAIN

preds = model.predict(X_test) # class labels
probs = model.predict_proba(X_test) # probability per class
report = classification_report(y_test, preds) # precision/recall/f1 per class
k_acc = top_k_accuracy_score(y_test, probs, k=5) # did the right role appear in top 5?

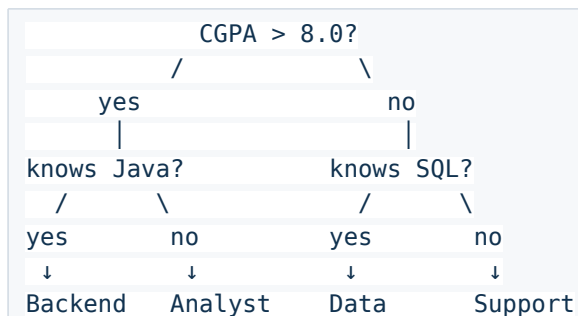
joblib.dump(model, "model/random_forest.joblib") # persistence for the Flask service
```

Why `random_state=42`? → "Reproducibility — the same data and seed give the same model, so a result can be re-checked." **Why `stratify=y`?** → "It keeps the class proportions in both splits, so every role is represented in the test set." **Why `predict_proba`?** → "I need a ranked shortlist with confidence, not just one label."

PART E — RANDOM FOREST (the HIGH claim — learn this properly)

Step 1: what is a decision tree?

SAY: "A decision tree asks a series of yes/no questions about the features. At each node it picks the question that best separates the classes, and each leaf holds a prediction."



Analogy: the game "20 questions" — each question narrows down the possibilities.

How it chooses a split: it measures impurity — **Gini impurity** or **entropy** — and picks the feature/threshold that reduces impurity the most. A node where all samples share one label has impurity 0.

NOTE: The problem with one tree: a deep tree memorises the training data — including its noise. It has **high variance**: change a few rows in the training set and the tree changes drastically. It overfits.

Step 2: the ensemble idea

SAY: "Instead of trusting one tree, I grow many and let them vote. Individual trees make different mistakes, so their errors cancel out — that's the wisdom-of-crowds idea. Averaging many noisy, roughly-unbiased predictors reduces the variance without adding much bias."

Analogy: ask one person to guess the number of sweets in a jar — unreliable. Ask 300 people and average — startlingly accurate.

Step 3: what makes the forest *random* (this is the answer to "how is it different from just many trees?")

SAY: "Two sources of randomness:

1. **Bagging (bootstrap aggregating)** — each tree is trained on a random sample drawn *with replacement* from the training data, so each tree sees a slightly different dataset.
2. **Feature randomness** — at each split, each tree may only consider a random **subset of features** (by default `sqrt(n_features)` for classification). This prevents one dominant feature from being chosen by every tree and making them all identical.

Because the trees are decorrelated, their average has much lower variance than any single tree."

Step 4: how a prediction is made

SAY: "For classification, every tree produces a class probability — the fraction of its training samples of each class in the leaf the input reaches. The forest **averages those probabilities across all trees** (soft voting) and takes the highest. That's what gives me `predict_proba`, which I sort to return the top five roles." TRAP: "Hard voting vs soft voting?" → "Hard voting takes the majority class label; soft voting averages the predicted probabilities. Soft voting is generally better because it uses the confidence information, and it's the default in `RandomForestClassifier.predict_proba`, which is what `predict()` uses."

Step 5: advantages and limitations (be balanced — it earns credibility)

Advantages

- Handles **non-linear** relationships and **feature interactions** without me specifying them.
- Works on **mixed feature types** after encoding; needs little scaling.
- Robust to outliers and to irrelevant features.
- Gives **feature importance**, so I can see which inputs drive the prediction.
- Needs less hyperparameter tuning than boosting methods to get a decent baseline.

Limitations

- **Less interpretable** than a single tree — you can't read 300 trees.
- Can still **overfit** on noisy data if trees are very deep / `min_samples_leaf` is 1.
- Weaker than **gradient boosting** (XGBoost/LightGBM) on many tabular benchmarks.
- **Probabilities are not perfectly calibrated** — averaging votes pushes confidence towards the middle, so a predicted 0.8 isn't literally an 80% chance. (This is exactly why EDU2JOB has an **isotonic calibration** layer — see file 12 for how to talk about it honestly.)
- Doesn't extrapolate: it can't predict outside the range it has seen.
- Memory/time grow with the number of trees.

NOTE: **Trap question: "Why Random Forest and not a neural network?"**

"The data is small, tabular, and made of mixed categorical and numeric features. On that kind of data, tree ensembles like Random Forest typically match or beat neural networks, need far less data and tuning, train in seconds on a CPU, and give feature importances I can inspect. A neural network would add complexity without a clear accuracy gain on this dataset size."

NOTE: **Trap: "Why not logistic regression?"**

"Logistic regression is a strong, interpretable baseline and I would compare against it. But my features are categorical and the patterns are likely non-linear with interactions — for example CGPA mattering differently depending on the degree or the skill set — and a forest captures that automatically rather than me hand-engineering interaction terms."

NOTE:NOTE: **TRAP: "What accuracy did you get?" Do not invent a number.** Your own case study says no accuracy figure is claimed. Say:

"I don't want to quote a number I can't verify. The model was fitted with a train/test split, and the evaluation harness — stratified validation, top-k accuracy and calibration metrics — is the next step in the repo README. What I can tell you precisely is the pipeline: the features, the encoding, the calibration and how the prediction is served. If you want a number, I'd rather run the harness and give you a real one."

That answer *raises* your score. An invented "94% accuracy" destroys your credibility the moment they ask how it was measured.

PART F — EDU2JOB END TO END (say this whole flow out loud)

```

Student fills: degree, major, CGPA, skills
      ↓
React → POST /api/predictions (JWT in the Authorization header)
      ↓
Spring Boot
  1. authenticate the JWT, identify the user
  2. validate the input (@Valid: CGPA 0–10, known degree values)
  3. build the feature payload
  4. call the Flask service: POST {ml-url}/predict
      ↓
Flask
  1. one-hot encode degree and major with the SAVED encoders
  2. scale CGPA with the SAVED scaler
  3. expand the skill list with the SAVED MultiLabelBinarizer
  4. model.predict_proba(...) → probability per role
  5. (isotonic calibration layer) sort desc, take the top 5
      ↓
Spring Boot
  5. persist the submitted inputs + raw model response as prediction history
  6. return the ranked roles to React
      ↓
React renders the top-5 ranked roles with confidence
  
```

SAY: "The reason this design works: the browser has one entry point, so auth and validation live in one place; the model artifacts and the encoding that produced them stay together in Python; and every prediction is stored, so the history endpoint can show what was predicted and with what inputs."

Predictable follow-ups:

- **"What is a feature here?"** → "One input column the model uses: the encoded degree, major, scaled CGPA, or one of the binary skill indicators."
- **"What is a prediction?"** → "The model's output for a new input: a probability per job role, ranked to give a top-5 shortlist."
- **"What is prediction history?"** → "A table row per prediction storing the user, the submitted inputs and the raw model output, so a student can revisit previous suggestions and I have an audit trail."
- **"What does authentication mean here?"** → "Only the logged-in student can request a prediction, and only their own prediction history is visible — the user id comes from the token, not from a request parameter, so nobody can request another user's history by changing an id."
- **"What if the model gives a bad prediction?"** → "A recommendation isn't a guarantee. The ranked list with confidence is more honest than one asserted label, the UI shouldn't present confidence as certainty, and the fix path is measuring top-k accuracy and calibration and retraining on better data — not hiding the uncertainty."
- **"What if the user enters invalid input?"** → "Bean validation rejects it at the Spring boundary with 400, and the Flask service validates its own payload too — because it's a separate service that could be called by anything."

SELF-TEST

1. Traditional programming vs ML, in one sentence.
2. Feature, label, overfitting.
3. Classification vs regression — which is EDU2JOB?
4. Why one-hot instead of 1/2/3 for categories?
5. Why fit the scaler on the training set only? What's the term?
6. Why is one decision tree not enough?
7. What are the two sources of randomness in a Random Forest?
8. How does a forest produce a prediction and a probability?
9. Two advantages and two limitations of Random Forest.
10. What's the most important preprocessing rule for a live model service?
11. What's a DataFrame in one line?

Key

1. Normal programming: write rules. ML: learn the rules from labelled examples.
2. Input variable / target to predict / memorising training data so performance on new data drops.
3. Classification (multi-class — discrete job roles).
4. Integer mapping invents a false order and distance between categories.
5. Using test statistics leaks information from the test set into training — **data leakage**.
6. High variance: it memorises noise and changes drastically with small data changes.
7. Bootstrap sampling of rows, and a random subset of features considered at each split.
8. Each tree outputs class probabilities; the forest averages them (soft voting) and takes the max.
9. Non-linear + interactions, little tuning, feature importance, robust to outliers / uncalibrated probabilities, less interpretable, weaker than boosting, can overfit.
10. Preprocessing must be identical at fit time and serve time — persist the encoders with the model.
11. Pandas' tabular data structure: named columns, typed, with a row index.

Next: [11-git-dsa.md](#)

SECTION 11

11 — GIT / GITHUB + DSA

Time: 10 min. Git = P1 (free marks). DSA = P1 (you claimed 500+ LeetCode — you invited this).

PART A — GIT

What problem does version control solve?

SAY: "Without version control, you keep `final_v2_really_final.zip` copies, you can't tell what changed or why, and two people editing the same file overwrite each other. A version control system records every change with a message, an author and a timestamp, lets me go back to any point, and lets several people work on the same project without overwriting each other."

Analogy: save points in a game. You can load an earlier state instead of re-playing from the beginning — and each save point has a note explaining what you did.

Git vs GitHub (asked almost every time)

SAY: "**Git is the version control system that runs on my machine — it tracks changes locally, and it works completely offline. GitHub is a hosting service for Git repositories on the internet: it stores a copy, adds collaboration features like pull requests, issues, code review and CI, and gives me a backup and a portfolio.**"

TRAP: "Can you use Git without GitHub?" → "Yes — Git is fully functional locally. GitHub is one possible remote."

The vocabulary

Term	Meaning
Repository (repo)	the project plus its full history (the <code>.git</code> folder)
Working directory	your current files on disk
Staging area (index)	what you've marked to include in the next commit
Commit	a snapshot of the project with a message, author and hash — the local save point
Branch	an independent line of development — a movable pointer to a commit
HEAD	where you currently are
Remote	another copy of the repo, usually <code>origin</code> on GitHub
Clone	copy a remote repo (with its history) to your machine
Push	send your commits to the remote
Pull / fetch	bring remote commits down (<code>pull</code> = <code>fetch</code> + <code>merge</code>)
Merge	combine two branches into one
Rebase	replay your commits on top of another branch for a linear history
Conflict	Git can't decide between two changes to the same lines — you resolve by hand
Pull request (PR)	a request to merge your branch into another, with review and discussion
<code>origin/main</code>	the remote's <code>main</code> branch

The commands you should be able to say from memory

<code>git init</code>	# start a repo
<code>git clone <url></code>	# copy a remote repo
<code>git status</code>	# what's changed / staged
<code>git add .</code>	# stage everything
<code>git add src/EscrowService.java</code>	# stage one file
<code>git commit -m "Add escrow release transition validation"</code>	
<code>git log --oneline --graph</code>	# history
<code>git diff</code>	# unstaged changes
<code>git diff --staged</code>	# what's about to be committed
<code>git branch feature/escrow-expiry</code>	# create a branch
<code>git switch -c feature/escrow-expiry</code>	# create and switch (modern)
<code>git switch main</code>	# go back to main
<code>git pull origin main</code>	# update from the remote
<code>git push origin feature/escrow-expiry</code>	# publish your branch
<code>git merge feature/escrow-expiry</code>	# merge into the current branch

```
# the branches diverge and the merge conflicts
# Git marks the file:
<<<<<<< HEAD
status = RELEASED;
=====
status = REFUNDED;
>>>>>>> feature/refund
# I edit the file to the correct final state, remove the markers, then:
git add <file>
git commit
```

Fixing mistakes (mention one or two — it shows real use):

```
git restore <file>           # discard unstaged changes
git restore --staged <file>  # unstage, keep the edits
git commit --amend          # fix the last commit message/content (not if pushed)
git revert <hash>           # new commit that undoes an old one – safe on shared branches
git reset --soft HEAD~1     # undo the last commit locally, keep changes (rewrite history)
```

SAY: "**reset rewrites history, revert adds a new commit that undoes one. On a shared branch I use revert — rewriting pushed history forces everyone else to fix their clone.**" *That sentence proves you understand it, not just memorized it.*

Common questions

- **"What does .gitignore do?"** → "Lists paths Git should never track — `node_modules/`, `target/`, `.env`, build output, IDE files. Secrets stay out of the repository."
- **"What does a commit hash identify?"** → "The exact snapshot and its ancestry — it's the content-addressed id of that commit."
- **"What's a pull request for?"** → "Proposing a branch for merge with review. It gives a place for discussion, a diff for review, automated checks, and an audit trail of why the change happened."
- **"What's the difference between merge and rebase?"** → "Merge preserves the branch history with a merge commit; rebase replays my commits on top of the target branch for a linear history, at the cost of rewriting commit hashes. I rebase my own local branch before a PR and merge for the actual integration."
- **"How would you undo a bad commit already pushed to main?"** → "`git revert <hash>` and push the revert commit — history stays intact and everyone gets the fix. Only rewrite with `reset/force-push` on a branch nobody else has."

SAY: "**How did you use Git in your projects?**" (natural answer)

"Every project is a Git repository on GitHub. I commit in small logical steps with messages that say what changed — for example adding the escrow state machine, then the JWT filter, then the frontend wiring. I work on feature branches and merge into main, I use `.gitignore` to keep build output and environment files out, and I keep secrets in environment variables rather than in the repo. For my portfolio I also opened a pull request so the change went through review before merging." **Honest and specific — no need to claim a team workflow you didn't have.**

PART B — DSA AND THE "500+ LEETCODE" CLAIM

First: the framing

Your resume claims 500+ problems. That's a **reasonable** claim to make and it's your strongest DSA signal — but it invites one live coding question. You do not need to be able to solve hard problems tomorrow. You need to be **fast and confident** on easy/medium array, string, hash-map and two-pointer problems, and you must be able to talk about complexity.

SAY: If asked to solve something:

1. **Repeat the problem back** and ask clarifying questions (input size, duplicates, sorted?, return value).
2. **State a brute-force approach and its complexity** out loud before optimising. (*This alone earns points even if you don't finish coding.*)
3. **Name the pattern** you'll use ("this is a hash-map frequency count").
4. **Code it, talking as you go.**
5. **Walk through a small example** and check edge cases: empty input, one element, all duplicates, negative numbers.

What each structure is for (this is what they test)

Structure	Use it when	Complexity (avg)
Array	indexed access, contiguous data	access $O(1)$, search $O(n)$
String	text; treat as a char array	—
HashMap	counting, lookups, "have I seen this?", pairing	get/put $O(1)$
HashSet	dedup, membership, "seen before"	add/contains $O(1)$
Stack	LIFO: brackets, undo, DFS iteratively, monotonic stack	push/pop $O(1)$
Queue / Deque	FIFO: BFS, sliding window	$O(1)$
Linked List	insert/remove at a known node in $O(1)$; no random access	access $O(n)$
Binary Tree	hierarchical data, traversal	most ops $O(n)$
BST	sorted search/insert	$O(\log n)$ if balanced, $O(n)$ if degenerate
Heap / PriorityQueue	repeatedly get the min/max, top-k	insert/pop $O(\log n)$, peek $O(1)$
Graph	relationships, networks, dependencies	BFS/DFS $O(V+E)$

Big-O — the answer you must have ready

SAY: "Big-O describes how the running time or memory grows as the input size grows — it's the growth rate, not a stopwatch. It ignores constants and lower-order terms, because at scale the fastest-growing term dominates. For example $O(2n)$ and $O(n)$ are both linear."

Complexity	Name	Example
$O(1)$	constant	array index, hash-map lookup
$O(\log n)$	logarithmic	binary search, balanced BST
$O(n)$	linear	one pass through an array
$O(n \log n)$	linearithmic	merge sort, heap sort, good comparison sorts

Complexity	Name	Example
$O(n^2)$	quadratic	nested loops, bubble/insertion/selection sort
$O(2^n)$	exponential	naive recursive subsets/Fibonacci
$O(n!)$	factorial	permutations

Time vs space: "Space complexity is the same idea for extra memory. Sometimes I trade one for the other — a hash map uses $O(n)$ extra space to turn an $O(n^2)$ scan into $O(n)$."

TRAP: "What's the complexity of your solution?" → Always answer with both: " $O(n)$ time, and $O(n)$ extra space for the map. If I sorted first it'd be $O(n \log n)$ time but $O(1)$ extra space."

Binary search

SAY: "Binary search finds a value in a **sorted** array by halving the range each step: check the middle, then discard the half that can't contain the target. That's $O(\log n)$, because $\log_2 n$ halvings reduce n to 1."

```
int search(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // avoids int overflow vs (lo+hi)/2
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

TRAP: "Why `lo + (hi - lo) / 2`?" → "`(lo + hi)` can overflow a 32-bit int for very large arrays; this equivalent form can't." TRAP: "What if the array isn't sorted?" → "Binary search is invalid — I'd sort first ($O(n \log n)$) or use a hash map instead."

HashMap

SAY: "A HashMap stores key-value pairs, hashing the key to find a bucket, so lookups are $O(1)$ on average. It's the structure I reach for when the question is 'have I seen this before?' or 'how many times does this appear?'."

```
// count character frequencies — the most common interview pattern
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray())
    freq.put(c, freq.getOrDefault(c, 0) + 1);
```

NOTE: Worst case $O(n)$ when everything collides into one bucket (which Java 8+ mitigates by converting to a tree). NOTE: Requires `equals/hashCode` to be consistent for custom keys.

BFS vs DFS (near-certain question if graphs come up)

	BFS	DFS
Data structure	Queue	Stack (or recursion)
Explores	level by level	as deep as possible first
Finds	shortest path in an unweighted graph	any path; good for exhaustive search
Space	$O(\text{width})$	$O(\text{depth})$
Use for	shortest route, nearest match, grid steps	cycle detection, topological sort, connected components, backtracking

SAY: "BFS explores in layers using a queue, so the first time it reaches a node it's via the shortest path in an unweighted graph. DFS goes deep first using a stack or recursion — it's simpler and cheaper on memory for deep, narrow graphs, and it's the natural fit for backtracking and cycle detection."

```
// BFS skeleton
Queue<Integer> q = new ArrayDeque<>();
boolean[] seen = new boolean[n];
q.add(start); seen[start] = true;
while (!q.isEmpty()) {
    int node = q.poll();
    for (int nxt : adj.get(node))
        if (!seen[nxt]) { seen[nxt] = true; q.add(nxt); } // mark on ENQUEUE, not on dequeue
}
```

NOTE: **Mark visited when enqueueing, not when dequeueing** — otherwise a node can be enqueued twice. That's a real bug interviewers watch for, and it's exactly the pattern in your own `assets/algorithms.js` portfolio demo.

Sorting & searching summary

Algorithm	Best	Average	Worst	Space	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

SAY: "Merge sort is $O(n \log n)$ in every case and stable, but needs $O(n)$ extra space. Quicksort is usually faster in practice and sorts in place, but its worst case is $O(n^2)$ on already-sorted input with a bad pivot choice — randomised or median-of-three pivots fix that in practice." **Stable** = equal elements keep their relative order. NOTE: Quick check: "Is `Arrays.sort(int[])` stable?" → "No, the primitive version is a dual-pivot quicksort and isn't stable; `Arrays.sort(Object[])` uses Timsort and is stable." (*Saying that is a strong detail.*)

Recursion — how to explain it

SAY: "Recursion is a function that calls itself on a smaller version of the problem, with a **base case** that stops it. Each call sits on the stack until the base case returns. Every recursive solution can be written iteratively, and recursion costs stack space — deep recursion can overflow." NOTE: "What's the complexity of naive recursive Fibonacci?" → $O(2^n)$, because the same subproblems are recomputed. With memoisation it's $O(n)$ time and space."

The questions you will actually be asked about your claim

"What's your strongest DSA topic?"

"Arrays and hash maps with two pointers and sliding window. That combination covers a large share of medium problems — counting, frequency, subarray and substring work — and I'm comfortable reasoning about the time and space complexity of each approach. I'm also solid on BFS/DFS on grids, which I used in AlgoViz for the pathfinding visualization."

"How did you practise 500+ problems?"

"Mostly targeted practice: I pick one pattern at a time — two pointers, sliding window, prefix sums, BFS — solve several problems in that pattern until the recognition is automatic, then revisit the ones I got wrong. I also keep notes on the *pattern* rather than the solution, and I re-solve problems I struggled with after a gap."

"Give me an example of a problem that took you a long time." → Have one real story ready. Describe the problem, the wrong approach you started with, what you realised, and the insight. **Do not invent a problem you can't discuss.** "What is a prefix sum and why is it useful?" → "A cumulative-sum array where `prefix[i]` is the sum of the first `i` elements, so the sum of any range is `prefix[r+1] - prefix[l]` in $O(1)$ instead of $O(n)$. It turns repeated range-sum queries from $O(n)$ each into $O(1)$." (*Your AlgoViz demo has a prefix-sum module — this is consistent.*) "Sliding window?" → "For contiguous subarray/substring problems, I keep two pointers and expand the right edge, shrinking from the left when a constraint is violated. It turns an $O(n^2)$ check of all windows into $O(n)$, because each element enters and leaves the window once."

One realistic live question (you should be able to do this in 5 minutes):

"Given an array of integers, return indices of the two numbers that add up to a target."

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>(); // value -> index
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) return new int[]{seen.get(need), i};
        seen.put(nums[i], i);
    }
    throw new IllegalArgumentException("No two sum solution");
}
```

SAY: "Brute force is $O(n^2)$ — check every pair. Using a hash map that stores each value's index, I check whether the complement has already been seen: $O(n)$ time, $O(n)$ space. The insight is that I'm looking for `target - current`, so I can trade memory for speed." **Practise saying that out loud. It's the single most likely live question, and this version covers brute force, complexity and the insight.**

SELF-TEST

1. Git vs GitHub.
2. Commit vs push.
3. What do you do in a merge conflict?

4. `reset` vs `revert` — which on a shared branch?
5. What's $O(n \log n)$ and name two algorithms with it.
6. Binary search: what's required, and what's the complexity?
7. HashMap lookup complexity, and the worst case.
8. BFS vs DFS — data structures and when to use which.
9. Why is the visited flag set on enqueue in BFS?
10. Two-pointer target-sum: give the $O(n)$ approach.

Key

1. Git = local VCS; GitHub = remote hosting + collaboration features.
2. Commit saves locally; push sends commits to the remote.
3. Edit to the correct final state, remove the markers, stage, commit.
4. `reset` rewrites history (local only); `revert` adds an undo commit — use `revert` on shared branches.
5. Growth rate for comparison sorting / merge sort and heap sort (also Timsort).
6. A sorted collection; $O(\log n)$.
7. $O(1)$ average, $O(n)$ worst case with pathological collisions.
8. BFS = queue, shortest path in unweighted graphs; DFS = stack/recursion, cycle detection, backtracking, components.
9. Otherwise the same node can be discovered by two neighbours and enqueued twice.
10. Hash map of value $\rightarrow$ index; for each element check whether `target - current` is already present.

Next: `12-project-defense.md` — **the highest-value file in this pack.**

SECTION 12

12 — PROJECT DEFENSE (REVERTPAY · EDU2JOB · ALGOVIZ)

Time: 30 min to read, but this is the HIGHEST-VALUE file in the pack. the target company will spend most of the interview here.

Read this first: these scripts are aligned with what your public case studies actually claim, so nothing here contradicts your own repo — and they flag exactly where to stay honest.

★ PROJECT 1 — REVERTPAY V1 (your flagship)

1. The 30-second version (memorize verbatim)

"RevertPay is an escrow and payment-reversal platform. The idea is that a buyer's money is held safely until the transaction completes: the buyer funds the escrow, the seller ships, the buyer confirms delivery, and only then are the funds released to the seller. I built it as a Spring Boot backend with PostgreSQL, secured with JWT and role-based authorization, plus a React frontend. The core design decision was to model the escrow as a validated state machine and to record money movements as ledger entries, so every state change is auditable."

2. The 1-minute version

"The problem RevertPay solves is trust between two people who don't know each other. If the buyer pays first, the seller might not ship; if the seller ships first, the buyer might not pay. Escrow solves that by having a neutral party hold the money until the conditions are met."

I modelled the escrow as a state machine — created, funded, shipped, delivered, released, with dispute and review and expiry paths. Transitions are validated centrally, so an escrow can't jump from funded straight to released; an illegal transition returns a conflict rather than corrupting the record.

The money side is a ledger. Each movement writes a paired debit and credit entry, and an account balance is derived by summing those entries rather than storing a balance column — so the balance can never disagree with the history, and an audit is just re-running the same arithmetic.

Technically: Java and Spring Boot for the API, Spring Security with JWT for authentication and role-based plus ownership checks for authorization, Spring Data JPA with Hibernate over PostgreSQL for persistence, and React with Vite on the frontend. Everything that changes money or state happens inside a single transactional service method, so it either all commits or all rolls back."

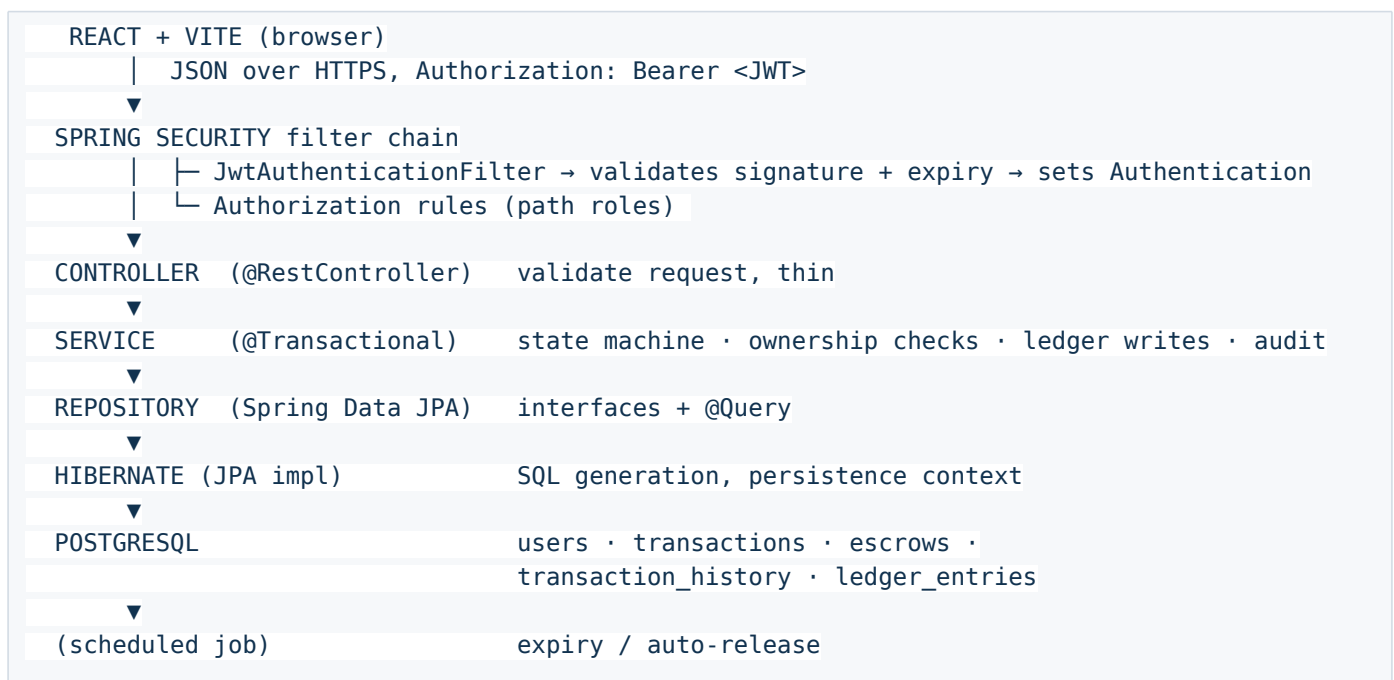
3. The 3-minute deep version — add these layers

- **Why a state machine:** "The whole risk in escrow is being in a state that shouldn't exist — funds released twice, or released while the item was never delivered. Encoding the legal edges in one place means no controller can set a status directly, and every transition is logged with the actor, the reason and the timestamp."
- **Transaction boundaries:** "Both legs of the money movement, the escrow status update and the audit record are in the same transactional service operation. If the ledger write fails, the status change rolls back too. There's no intermediate state where money moved but the state didn't."
- **Concurrency:** "Two requests could try to release the same escrow at once. Two protections — the transaction re-reads the state and rejects an invalid one, and an `@Version` optimistic-lock column means a

conflicting write fails loudly instead of silently overwriting. Escrow creation also takes an idempotency key with a unique constraint, so a retried POST can't create two escrows."

- **Expiry:** "A scheduled job handles escrow expiry and auto-release rules, so an escrow that's never confirmed doesn't sit forever."
- **Frontend:** "React with Vite: the buyer and seller see different actions for the same transaction based on their role and the current state, and all requests go through one API helper that attaches the bearer token and handles 401 by clearing the session."

4. Architecture (draw this — practice it twice)



5. Technologies and why each

Tech	Why
Java	statically typed, mature, the language my backend skills and course work are in; Spring's ecosystem
Spring Boot	removes boilerplate, embedded server, auto-config, first-class transactions and security
Spring Data JPA / Hibernate	entity mapping + repository abstraction, so persistence code is small and the domain model is clear
PostgreSQL	ACID transactions, real constraints (unique/FK/check), exact <code>NUMERIC</code> for money
Spring Security + JWT	centralised auth on every endpoint; stateless fits a REST API
BCrypt	passwords must never be stored reversibly
React + Vite	component model for role/state-dependent UI; fast dev loop
@Transactional	money and state changes must be atomic

6. Main features

Escrow create & fund · seller ship · buyer confirm delivery · release to seller · dispute → review → resolve/refund · escrow expiry via scheduled job · transaction history/audit view · derived account balance · JWT login and registration · buyer/seller/admin roles with ownership checks.

7. SAY: The hardest part (have this ready — "what was hard?" is always asked)

"Keeping the state change and the money movement consistent. It's not enough for the escrow to say RELEASED — the ledger entries have to exist, the audit row has to exist, and none of it can happen twice. So I put all of it inside one transactional service method, added a version column so a concurrent update fails instead of double-releasing, and centralised the legal state transitions so the rule can't be bypassed from a controller."

8-10. Biggest bug / problem and how you solved it

Pick one and tell it as a story (STAR shape). Honest, plausible options:

Option A (concurrency): "During testing I fired two release requests back to back and both got past the status check before either committed — the second one would have moved money twice. I fixed it by adding an `@Version` column for optimistic locking, so the second transaction fails with a locking exception and I map that to a 409. A stronger option I'd consider is a database-level constraint on the ledger so a duplicate release is physically impossible."

Option B (invalid transitions): "Initially the status was just a field that the controller could set from the request body, which meant the API accepted nonsense like FUNDED → RELEASED. I refactored it so the status is only ever changed inside the service through validated transitions, and the request DTO no longer contains a status field at all. That's the moment I understood why DTOs should be separate from entities."

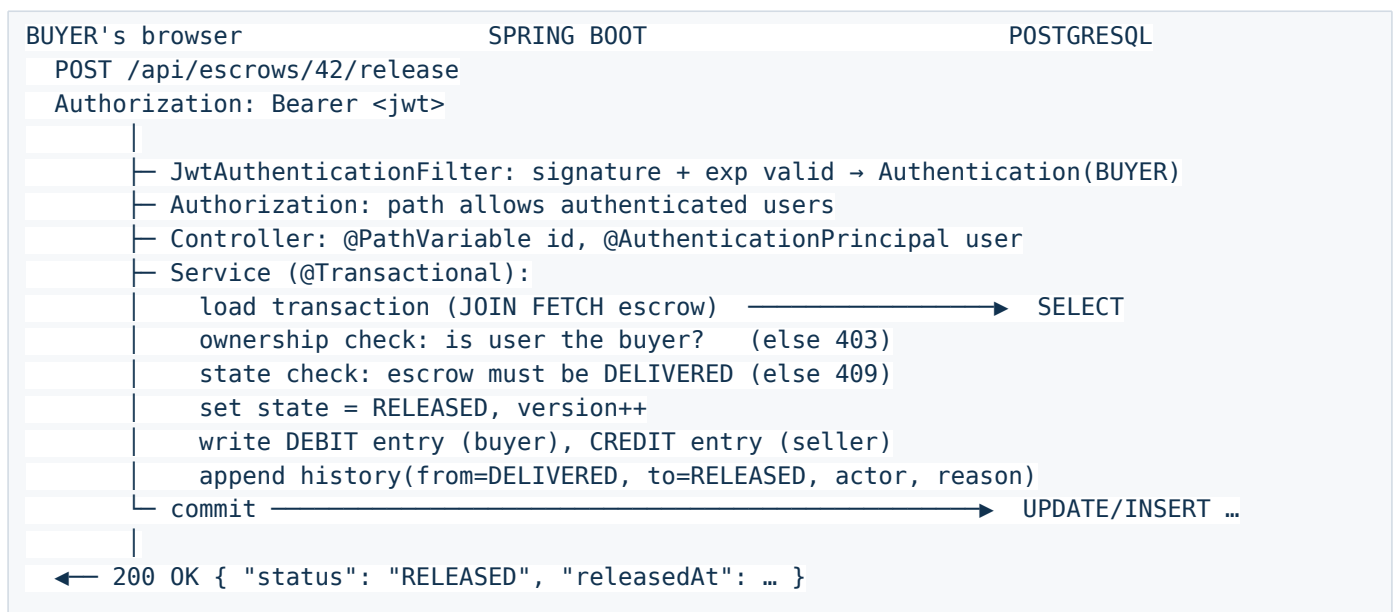
NOTE: Both are honest, technical, and end with a lesson. **Do not claim you found a production incident** — you didn't have production.

11. Database design

```
users(id, name, email UNIQUE, password_hash, role, version, created_at)
transactions(id, buyer_id→users, seller_id→users, amount NUMERIC(12,2),
             status, idempotency_key UNIQUE, version, created_at)
escrows(id, transaction_id→transactions UNIQUE, state, expires_at)
ledger_entries(id, transaction_id→transactions, account_id→users,
               direction, amount, created_at)
transaction_history(id, transaction_id→transactions, from_state, to_state,
                  actor_id→users, reason, created_at)
```

SAY: "Users relate to transactions twice — as buyer and as seller. Each transaction has exactly one escrow holding its state and expiry, and many history rows forming the audit trail. Ledger entries reference the transaction and an account, and balances are derived, never stored." **Design points to volunteer:** `NUMERIC(12,2)` for money (never float); `CHECK` constraints on valid states and positive amounts; unique constraint on `idempotency_key`; indexes on the foreign keys I filter by; `version` for optimistic locking.

12. API flow — end-to-end release



13. Security (keep it tight)

- JWT in the `Authorization: Bearer` header; the filter validates signature + expiry and populates the `SecurityContext`.
- BCrypt for password storage; `matches()` at login, never decryption.
- Path-based role rules (`/api/admin/**` → ADMIN) **plus** service-layer ownership checks, because roles alone don't stop one buyer touching another buyer's escrow.
- DTOs on the boundary so a client can't post `status` or `role` directly (mass-assignment).
- Validation with `@Valid`; consistent JSON errors; no stack traces or internals leaked; secrets in environment variables.

14. Testing — NOTE: BE CAREFUL HERE (this is a bluff trap)

Your case study states: state-machine test matrix and a concurrent release-race test with Testcontainers are **planned**, and **no comprehensive financial correctness test suite is claimed**. SAY: **Safe answer:**

"Honestly, that's the weakest part of the project right now. I verified behaviour manually through the API and I have the state-machine matrix and a concurrency race test using Testcontainers written up as the next step. Unit-testing the transition service and the ledger arithmetic, plus an integration test that hammers the release endpoint concurrently, is exactly what I would add first." NOTE: **Never say** "yes, fully tested." If pushed: "I can describe what I tested manually and what the test plan is."

15. Honest limitations (say these before they find them)

- No live deployment; verification is the public repo.
- Stored-response replay for idempotent retries is a roadmap item — the current unique constraint rejects duplicates rather than returning the original response.
- No real refund/partial-settlement flow beyond the review outcome.
- No refresh-token flow; the access token expires and the user logs in again.
- Tokens are stored in browser storage, so XSS is a consideration; `HttpOnly` cookies plus CSRF protection would be the alternative.
- No comprehensive automated test suite yet.

16. What I'd improve next

"Concurrency-focused integration tests with Testcontainers PostgreSQL, a fully retry-safe idempotency implementation with stored responses, refresh tokens, and moving beyond a single-instance deployment with proper observability."

17. Likely interviewer questions (rehearse all of these)

1. What is escrow, and why does it exist?
2. Walk me through the escrow lifecycle.
3. What stops a buyer from releasing funds before delivery? (*state machine*)
4. Two release requests at the same time — what happens?
5. Why a ledger instead of a balance column?
6. What is a transaction in the database sense, and what's `@Transactional` doing?
7. Why `BigDecimal/NUMERIC` for money?
8. How did you decide which endpoints need which role?
9. What happens if the JWT is expired mid-flow?
10. Where would a bug hurt most in this system? (*money movement / state transitions*)
11. How would you find money that doesn't balance?
12. What if the seller never ships?
13. What if the escrow expires?
14. How would you add a refund?
15. What's the difference between your `escrows.state` and `transactions.status`? (*be careful — explain that the escrow holds the lifecycle state and the transaction is the money record; if your code differs, say "in my implementation the escrow row holds the lifecycle state and the transaction row holds the payment record and status"*)

18. Follow-up attacks

- **"You said role-based authorization. What if the ADMIN user is also a buyer?"** → "Roles and ownership are separate checks: ADMIN passes the path rule, and for party actions the ownership check requires the user to be the buyer or seller on that specific transaction. A user can hold one role in my model; in a richer model I'd make roles a set and check both the required capability and the relationship to the resource."
- **"What if someone replays the exact same release request twice?"** → "The second one finds the state already RELEASED and gets a 409 — the transition is rejected because RELEASED isn't a legal edge from RELEASED."
- **"What if the database commits but the response never reaches the client?"** → "The client retries, and the retry is rejected as an invalid transition. That's why an idempotency key with a stored response is the correct end state — the retry should return the original success, not an error. It's on my roadmap list."
- **"Where is your balance computed — in the DB or in Java?"** → "It's derived from the ledger entries; the sum query is a database aggregate, and I expose it as a read-only value with no setter. There's no balance column to drift."
- **"Why not just use a status enum and a queue?"** → "For this scope, a synchronous transactional service is simpler and easier to reason about. A queue would help if the downstream effects were slow or unreliable, and it introduces its own exactly-once problem."

19. "Show me the code" — what to open and say

They say	You open	You say
"Show me your controller"	<code>EscrowController</code>	"Thin by design — path variable, authenticated principal, delegate to the service. No business logic."
"Show me your service"	<code>EscrowService</code>	"The <code>@Transactional</code> method. State validation, ownership check, ledger writes and the history append all inside one transaction."
"Show me your state machine"	transition service / enum	"The legal edges. Every transition goes through here, so a controller can't set a status directly."
"Show me your repository"	<code>TransactionRepository</code>	"An interface extending <code>JpaRepository</code> ; derived query names plus a <code>@Query</code> with a join fetch."
"Show me an entity"	<code>Transaction</code> / <code>Escrow</code>	The annotations: <code>@Entity</code> , <code>@Id</code> , <code>@Version</code> , <code>@ManyToOne</code> with <code>@JoinColumn</code> , <code>@OneToOne(mappedBy=...)</code> .
"Show me the relationship"	entity fields or the ER diagram	The diagram from section 11.
"Show me your JWT implementation"	<code>JwtService</code> , <code>JwtAuthenticationFilter</code> , <code>SecurityConfig</code>	"Generation and validation in the service; the filter reads the bearer header, validates, and sets the context; the config declares the path rules."
"Show me the ledger"	ledger service/entity	"Paired debit and credit entries; the balance is a derived sum."

NOTE: **Never open a file you can't read.** If you don't recognise the code, say: "Let me find the exact file — the logic is in the escrow service, I'll show you the method that enforces the transition."

★ PROJECT 2 — EDU2JOB (Infosys Springboard internship)

1. The 30-second version

"During my Infosys Springboard internship I worked on EDU2JOB, a job-role recommendation system. A student enters their education, CGPA and skills, and a Random Forest model returns a ranked shortlist of suitable job roles. I built the Spring Boot backend with REST APIs for authentication, profiles, predictions and prediction history, integrated a React frontend, persisted everything in PostgreSQL, and worked with the Python scikit-learn model service that does the prediction."

2. The 1-minute version (add the architecture)

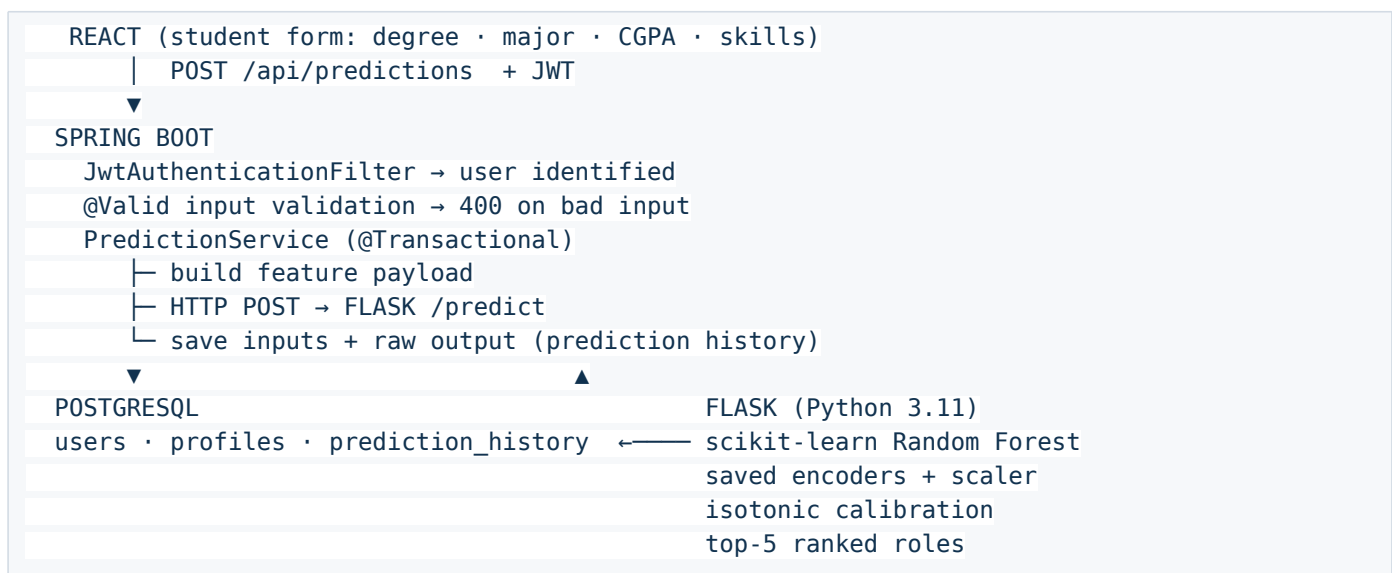
"The interesting part is the architecture, because it's deliberately split. React calls Spring Boot, and only Spring Boot. Spring handles authentication with JWT, validates the input, then forwards the feature payload to a Python Flask service that wraps the trained Random Forest. That service applies the same preprocessing that was used at training time — one-hot encoding of degree and major, scaling the CGPA, and expanding the skills list into binary indicators — and returns a probability per role. Spring sorts and stores the raw inputs and output as prediction history in PostgreSQL, and returns the top five roles to React.

I kept the browser away from Flask on purpose: one entry point means one place for authentication and validation, and the model service doesn't need to be publicly exposed or implement its own security. The trade-off is a network hop and a contract that has to stay aligned between Java and Python."

3. The 3-minute deep version — add these

- **Why Random Forest:** non-linear tabular data, mixed categorical and numeric features, small dataset, fast to train, gives feature importance. Compare against logistic regression (interpretable baseline, but assumes a simpler boundary) and neural networks (needs much more data and tuning for no clear gain here).
- **Preprocessing as a contract:** "The encoders are fitted at training time and saved with the model, then reused at serve time. If they drifted, the model would still return a confident prediction that's simply wrong, with no error anywhere — that's the failure mode I was most careful about."
- **Calibration:** "Averaged tree votes tend to be pushed towards the middle, so the raw probabilities aren't perfectly calibrated. The pipeline has an isotonic calibration step before ranking, so a 0.8 confidence means closer to an 80% empirical rate."
- **Prediction history:** "Every prediction stores the inputs and the raw model response, so a student can revisit past suggestions and I have an audit trail for debugging what the model was actually given."

4. Architecture



SAY: "**The browser calls Spring. Spring calls Flask. Flask calls the model. PostgreSQL sits behind Spring.**" — Say exactly this sentence; it's the answer to half the questions.

5-6. Technology choices

Tech	Why
React	form-driven UI, component reuse, clear state for the prediction result
Spring Boot	auth, validation, transactions, REST, JPA — the application boundary
PostgreSQL	users, profiles and prediction history; relational integrity
Python + scikit-learn	the ML ecosystem lives in Python; sklearn's consistent <code>fit/predict</code> API
Random Forest	strong on small tabular data with mixed feature types
Flask	minimal way to expose the model as an HTTP endpoint
JWT	stateless auth so the browser has one secure entry point

7. Main features

Registration/login with JWT · student profile (degree, major, CGPA, skills) · prediction returning ranked roles with confidence · prediction history · validation on both boundaries · Java↔Python service integration.

8. SAY: Hardest part

"Keeping the Java and Python sides in agreement about the features. The model expects a specific vocabulary of degrees, majors and skill tokens. If the Spring payload and the Python encoder disagree about a field name or a category, the prediction either fails or, worse, silently means something different. So I made the feature contract explicit, kept preprocessing next to the model artifact, and made the encoders handle unseen categories safely instead of crashing — with the awareness that an unseen category becomes an all-zeros vector, which is safe but low-information."

9-10. Bug / problem story

"The clearest problem I hit was the boundary between the two services: a field name mismatch between the Spring DTO and the Flask payload meant the model received a default value instead of the real one, and it returned a confident but wrong prediction with no exception anywhere. Nothing failed loudly — that was the lesson. I made the payload explicit and validated it on both sides, so a missing or unexpected field is a 400 rather than a silent default. That changed how I think about service boundaries: an integration is a contract, not just an HTTP call." [YES] This is honest, specific, and memorable. **Use it.**

11. Database design

```
users(id, name, email UNIQUE, password_hash, role, created_at)
student_profiles(id, user_id→users UNIQUE, degree, major, cgpa, updated_at)
skills(id, name UNIQUE)
profile_skills(profile_id→student_profiles, skill_id→skills)      -- many-to-many
predictions(id, user_id→users, degree, major, cgpa, skills_snapshot,
             model_response_json, created_at)                    -- prediction history
```

SAY: "`student_profiles` is one-to-one with users. Skills are many-to-many through a join table rather than a comma-separated column, so they're queryable and normalised. The predictions table is effectively an

immutable audit log of what was asked and what came back."

12. API flow

```
POST /api/auth/login      → 200 { token }
GET  /api/profile/me      → 200 { degree, major, cgpa, skills }
PUT  /api/profile/me      → 200 (validated)
POST /api/predictions     → 201 { predictions: [{role, confidence}, ...5] }
GET  /api/predictions     → 200 (my history, paged)
```

SAY: "The prediction endpoint is the only one that crosses into Python. The user id always comes from the JWT, never from a request parameter, so a student can't ask for another student's history."

13. Security

JWT authentication; a user can only read and write their own profile and history; passwords hashed with BCrypt; validation at the Spring boundary *and* at the Flask boundary; the model service isn't publicly exposed. NOTE: **Honesty note from your case study:** "JWT handling is present in the project; role-based access control is roadmap work rather than a shipped feature." So say: "*Authentication is implemented with JWT; role-based access control isn't really needed here because every user is a student acting on their own data — ownership checks cover it. RBAC is listed as future work if we add mentor or admin views.*" **Never claim an admin role you didn't build.**

14. Testing

Same rule as RevertPay. Your case study says the evaluation harness (stratified validation, top-k accuracy, calibration metrics) is **planned**. SAY: "The honest status is that the model evaluation harness is the next step — stratified validation, top-k accuracy and calibration metrics. What I verified was the pipeline end to end: that the preprocessing produces the expected feature vector and that a request round-trips correctly through Java and Python."

15. Limitations

- No public deployment link.
- Model evaluation is not yet measured with a proper harness — no accuracy figure is claimed.
- No explicit timeout configured on the Spring→Flask call; a slow model response is a known risk.
- The ML service URL needs to be fully externalised rather than configured ad hoc.
- No versioned database migration path (Flyway/Liquibase) — a follow-up.
- An unseen category encodes to all-zeros: safe, but low information.

16. Improvements

"A measured evaluation harness first, then a client timeout and retry policy on the Java→Python call, then schema migrations and containerised deployment. After that, a feedback loop — recording which suggestions students actually acted on — because that's the real signal for improving the model."

17. Likely questions

1. Walk me through a prediction request end to end.
2. Why Python for the model and Java for the app?
3. Why Random Forest? Why not logistic regression or a neural network?
4. What is a feature? What is a prediction?
5. How does the model handle a skill it has never seen?
6. What is preprocessing, and why does it have to match at training and serving time?

7. What happens if the Flask service is down?
8. What's stored in prediction history, and why?
9. What does authentication mean in this project?
10. How would you know if the model is good? (*top-k accuracy, per-class metrics, calibration*)
11. What if two students submit at the same time? (*stateless service; no shared state; each request is independent*)
12. Why not call the model directly from React?

18-19. Attacks and "show me"

- **"How do you know the model isn't just predicting randomly?"** → "I don't claim an accuracy number yet — that's the honest gap. Verifying it means stratified validation with top-k accuracy and calibration metrics, plus a baseline comparison against always predicting the most common role. That's the next step in the repo README."
- **"Isn't a recommendation system dangerous if it's wrong?"** → "It's a recommendation, not a guarantee. That's why it returns a ranked list with confidence rather than one asserted answer, and why the UI shouldn't present confidence as certainty."
- **"Show me the Flask endpoint"** → `app.py` / prediction route: model loaded once at startup, `request.get_json`, `predict_proba`, sort, top-5, 400 on invalid input.
- **"Show me the model code"** → training script: `train_test_split(stratify=y)`, preprocessing pipeline, `RandomForestClassifier`, `predict_proba`, `joblib.dump`.
- **"Show me the Java→Python call"** → the client class: `RestClient` POST to `/predict`, DTO mapping, exception → 503.

★ PROJECT 3 — ALGOVIZ 2.0

1. The 30-second version

"AlgoViz is an interactive algorithm visualization platform built with React, Vite, Tailwind and React Router. The key design decision is that algorithms **record** their execution as frames before playback, and the UI renders one recorded frame at a time. That makes pause, resume, speed control and step-by-step inspection trivial — they're all just moving a cursor — and it means you can seek backwards through the trace, which is impossible if you animate while the algorithm runs."

2. The 1-minute version

"The problem is that most algorithm animations show motion but not reasoning. When you animate execution live, you can't pause at a specific comparison, you can't step back, and you can't compare two algorithms fairly.

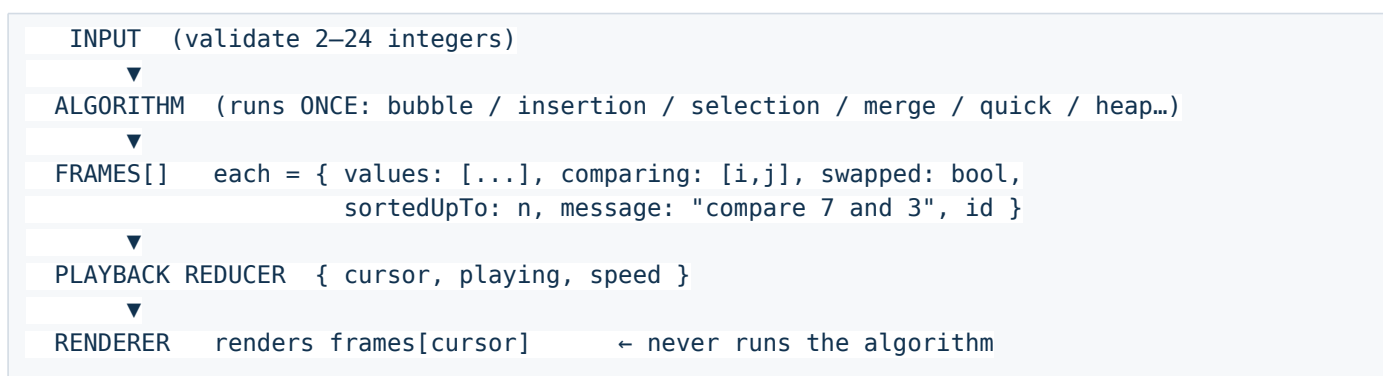
So I separated execution from presentation. The algorithm runs once and produces an array of frames — each frame is a complete snapshot of the array plus what's happening: which indices are being compared, whether a swap occurred, and a message explaining the step. The UI then renders `frames[cursor]`.

Playback state lives in a `useReducer` reducer — cursor, playing, and speed — so pause, step, seek and speed are all named state transitions instead of scattered booleans. Speed is just the delay between frames. And because the frame shape is shared, every sorting algorithm reuses the same visualization and controls; adding an algorithm means adding a frame generator, not a new UI."

3. The 3-minute deep version — add these

- **Trade-off you chose deliberately:** "Full snapshots make seeking and inspection easy, at a memory cost. For a quadratic algorithm, copying the whole array per frame grows roughly cubically in trace size, so I capped the sorting input at 2-24 integers instead of pretending it scales. Compact events or checkpoints are the future fix — and saying that cost out loud is better than hiding it."
- **Stable IDs:** "Each value keeps a stable identity, so when duplicate values swap places the trace stays reproducible — otherwise React's list reconciliation could reuse the wrong bar and the animation would show something that didn't happen."
- **Comparison:** "Because traces are recorded data, I can compare algorithms by **recorded operation counts**, which is honest — I explicitly don't claim wall-clock timing or memory benchmarks."
- **Reusable components:** "SortCanvas, PlaybackControls, Timeline and AlgorithmPicker are shared across sorting modules. The non-sorting modules — grid search, tree traversal, two-pointer — have genuinely different visual state, so they keep their own renderers rather than being forced through one generic abstraction that would be worse at all of them."

4. Architecture



5. Main features

Multiple algorithm modules · play/pause · step forward · speed control · timeline seeking · comparison by operation counts · responsive layout · saved experiments in `localStorage` with JSON import/export · lazy-loaded module routes.

6. Why each technology

Tech	Why
React	component model; state drives what's rendered, which is exactly what a frame renderer needs
<code>useReducer</code>	playback is a set of related transitions; one place defines every legal change
<code>useState/useEffect</code>	frame generation and the timer that advances the cursor
React Router	each algorithm has its own URL, so it's shareable and bookmarkable; lazy loading keeps the initial bundle small
Tailwind	responsive layout with a consistent spacing/colour scale, no custom CSS naming
Vite	fast dev server and HMR so iterating on visualizations is quick; optimised production build
localStorage + JSON	let students keep experiments without needing a backend

7. SAY: Hardest part

"Making the playback state impossible to get wrong. My first version used separate booleans and a couple of `setInterval` calls, and it was easy to end up playing while paused, or with two timers running after a fast double-click. Moving it to a reducer with one `playing` flag, one `cursor` and one `speed`, and letting a single effect own the timer with proper cleanup, removed the entire class of bug — the effect only schedules a timeout when `playing` is true, and cleanup always cancels the previous one."

8-10. Bug story

"The first version animated the algorithm as it ran. It looked fine until I tried to add a back button — you can't un-run a comparison. That's when I restructured it to record frames first. The rewrite made pause, seek and step all trivially correct, and it also made the traces comparable. That's the design decision I'd defend hardest in this project."

11-12. Data / state flow for a PAUSE click (they asked specifically)

```

1. user clicks Pause
2. onClick={() => dispatch({ type: 'PAUSE' })}
3. reducer returns { ...state, playing: false }           ← new state object
4. React re-renders with the new state
5. the scheduling useEffect re-runs because `playing` changed:
   if (!state.playing) return;                           ← returns early, no timer created
   and its cleanup already ran: clearTimeout(id)          ← the pending timer is cancelled
6. the renderer draws frames[cursor] — the same frame; nothing advances
7. clicking Play sets playing true → the effect schedules the next STEP after
   1000/speed ms → the cursor advances again
  
```

SAY: "So pause isn't pausing a computation — the computation is already finished. Pause just stops scheduling the next cursor increment. That's why resuming is instant and always lands on the exact right

frame."

13. Security / 14. Testing (be honest)

- Security: "No backend and no user accounts — it's a client-side tool. Data stays in the browser's `localStorage`, with JSON import/export for sharing. Nothing is uploaded, so there's no server-side attack surface."
- Testing: **your case study says 14 documented test groups plus a sliding-window reference check against a brute-force oracle across 768 cases** — those are input/comparison cases, not performance claims. Say exactly that, and say you used the Node test runner. Do not claim exhaustive correctness proof.

15. Limitations

- Not a general-purpose debugger, doesn't execute arbitrary user code.
- Doesn't claim to be a runtime benchmark — no wall-clock or heap measurement.
- Sorting input is capped at 24 values because full snapshots don't scale.
- Non-sorting modules use their own visual execution patterns, so the frame/reducer path isn't a universal framework.
- Dijkstra and Floyd's cycle detection are not modules in the repository (*your case study says this explicitly — don't claim them*).
- No offline/PWA support or server-side persistence.

16. Improvements

"Compact event-based traces or periodic checkpoints instead of full snapshots, a Web Worker so large traces don't block the UI, more algorithm modules, and moving saved experiments from `localStorage` to an optional backend so students don't lose work."

17. Likely questions

1. Why record frames instead of animating live?
2. What's in a frame?
3. How does pause work?
4. How does speed work?
5. What if the user clicks pause twice?
6. What if the user seeks while playing?
7. Why `useReducer` and not `useState`?
8. Why do keys matter here?
9. What is a reusable component in this project, and where did you stop reusing?
10. Why React Router? Why Tailwind? Why Vite?
11. What's the memory cost of your design?
12. How do you know the visualization matches what the algorithm did?

18. Attacks

- **"Your visualization could be wrong even if the algorithm is right."** → "That's why the frames are generated inside the algorithm implementation — the frame records the actual array state at that moment, it isn't hand-drawn — and why each value has a stable id, so a frame can't be misattributed during reconciliation. The tests compare the algorithm output against simple oracles."
- **"Why not just use CSS animations?"** → "Because a CSS animation is a fixed clip. Mine is data — I can seek to any frame, step through, and compare recorded operation counts between algorithms."

- **"Doesn't copying the array every step kill performance?"** → "It's a real cost, and I bounded it instead of hiding it: inputs are capped at 24 values, and I document that trace storage grows roughly cubically in the worst case. Compact events or checkpoints are the fix if I lift the cap."
- **"Is this really engineering, or a UI toy?"** → "The engineering is the separation of execution from presentation and the playback state machine. The interesting question isn't how to draw bars — it's how to make an execution trace inspectable, seekable and comparable without the UI and the algorithm interfering with each other."

19. "Show me" table

They say	You open	You say
"Show me your React component"	<code>SortCanvas/lab</code> component	"It's a pure function of the current frame — no algorithm logic lives in the render."
"Show me your router"	<code>App.jsx/routes</code>	Route per algorithm, <code>:id</code> param, lazy-loaded modules, catch-all 404.
"Show me how pause works"	reducer + the scheduling effect	The 7-step flow above.
"Show me how speed works"	the same effect	<code>delay = 1000 / speed</code> .
"Show me a reusable component"	<code>PlaybackControls</code>	"Shared by every sorting module; props in, dispatch out — it has no idea which algorithm is running."
"Show me your algorithm code"	a frame generator	"It returns frames, it doesn't render anything."

[TARGET] CROSS-PROJECT COMPARISON QUESTIONS (prepare these — they're common)

"Which project are you most proud of and why?"

"RevertPay, because it's the one where correctness actually mattered. In AlgoViz a wrong frame is a poor demo; in RevertPay a wrong transition moves money. Building the state machine and putting the ledger writes inside the transaction is where I learned the most about why the boundaries are where they are."

"Which was hardest?"

"RevertPay, for the concurrency and consistency reasons. EDU2JOB was harder to *integrate* — the Java/Python contract — and AlgoViz was hardest to *design* cleanly, because the first approach was subtly wrong."

"What would you do differently?"

"On RevertPay I'd write the concurrency and state-transition tests *before* the implementation — if the tests had existed first, I'd have found the double-release race immediately instead of reasoning about it afterwards. On EDU2JOB I'd define the Java/Python payload contract up front and validate it on both sides from day one. In general: tests and contracts first on things where a silent wrong answer is possible."

"You used React in two projects and Spring Boot in two — which side do you prefer?"

"Backend. I like the fact that the rules are explicit and testable there, and that a design decision like where a transaction boundary sits has a visible consequence. Frontend is where I verify the whole thing actually works for a person, so I don't want to lose it — but backend is where I want to go deeper."

"What did you NOT have time to do?" → Answer honestly with the limitations list above. Interviewers trust candidates who name limits.

[IMPORTANT] FIVE THINGS THAT WILL GET YOU CAUGHT (fix these tonight)

1. **Quoting an accuracy number for the Random Forest.** No number is claimed in your repo. Say the pipeline and that the harness is next.
2. **Claiming a comprehensive test suite.** Your own case study says the state-machine matrix, race test and model evaluation harness are *planned*. Say "manual verification plus a written plan" and use your time well instead of bluffing.
3. **Claiming a production deployment or real money movement.** Neither exists. Say it's a project verified through the repository.
4. **Explaining "persistence context" or "dirty checking" as if you used them deliberately** when you're not sure. Learn the one-line versions in file 04 — they're enough — and stop there.
5. **Saying "JWT is encrypted"** or "I store the token in an HttpOnly cookie" when you don't. Say "signed, not encrypted" and "browser storage, with the XSS trade-off acknowledged".

Deliver these five correctly and you'll sound like a careful engineer — which is exactly what they're hiring for.

Next: [13-code-show-and-tell.md](#)

SECTION 13

13 — "SHOW ME" · "WHY?" · "WHAT IF?"

Time: 12 min. These three question types are where freshers lose interviews. Read all three sections — they're short and immediately usable.

PART A — "SHOW ME."

The rule for this entire section

SAY: "I open the file, name the class and its one responsibility, then point at the specific lines — validation, the transaction boundary, the query — and stop. I never scroll around hoping. And if I don't recognise what I see, I say so and offer the part I do have."

NOTE: Three failure modes to avoid:

1. Opening a file and going silent while you read.
2. Claiming you wrote code that's clearly library code.
3. Guessing at a class name — say "the escrow service" rather than inventing `EscrowTransitionManagerImpl`.

The table (rehearsed answers)

They say	File / concept to look for	What to explain while showing
"Show me your controller."	<code>EscrowController</code> / <code>AuthController</code>	"Thin on purpose. <code>@RestController</code> + <code>@RequestMapping</code> for the base path; <code>@PathVariable</code> for the id; the user comes from <code>@AuthenticationPrincipal</code> , never from the request body; then it delegates to the service. No business rules here."
"Show me your service."	<code>EscrowService</code>	"This is the transaction boundary — <code>@Transactional</code> . Inside: load the transaction, check ownership, validate the state transition, update the state, write both ledger legs, append the history row. All of it commits or rolls back together."
"Show me a repository."	<code>TransactionRepository</code>	"An interface extending <code>JpaRepository<Transaction, Long></code> — I get <code>save/findById/paging</code> for free. Simple lookups come from the method name (<code>findByBuyerIdOrderByCreatedAtDesc</code>); anything complex is a <code>@Query</code> , and I use a <code>JOIN FETCH</code> to avoid an N+1."
"Show me your entity."	<code>Transaction</code> / <code>Escrow</code>	Walk the annotations: <code>@Entity</code> , <code>@Table</code> , <code>@Id</code> + <code>@GeneratedValue(IDENTITY)</code> , <code>@Version</code> for optimistic locking, <code>@Column(precision/scale)</code> for the amount, <code>@ManyToOne</code> + <code>@JoinColumn</code> for the buyer, <code>@OneToOne(mappedBy=...)</code> for the escrow. Then say: "the other side owns the foreign key, <code>mappedBy</code> names the field, not the column."
"Show me the relationship."	entity fields, or draw the ER diagram	"Users relate to transactions twice — buyer and seller. One transaction has one escrow and many history rows. Ledger entries reference the transaction and an account." Then draw it (file 12, section 11).
"Show me your JWT implementation."	<code>JwtService</code> → <code>JwtAuthenticationFilter</code> → <code>SecurityConfig</code>	"Generation and validation live in the service (sign with the secret, include <code>sub, role, exp</code>). The filter reads the <code>Authorization: Bearer</code> header, validates signature and expiry, and sets the authentication in the <code>SecurityContext</code> . The config declares which paths are public and which need which role, disables CSRF because it's a stateless token API, and registers the filter before the username/password filter."

They say	File / concept to look for	What to explain while showing
"Show me how a password is stored."	<code>PasswordEncoder</code> bean + registration/service code	" <code>BCryptPasswordEncoder</code> — <code>encode()</code> at registration stores a salted hash, <code>matches()</code> at login. Never decryption, because it's one-way."
"Show me your React component."	<code>SortCanvas</code> , <code>PlaybackControls</code> , or an escrow card	"It's a pure function of its props/state — it takes the current frame (or transaction) and renders it. No algorithm or API logic inside; the parent owns the data."
"Show me your router."	<code>App.jsx</code> / route definitions	" <code><Routes></code> with a route per page, <code>:id</code> params for the algorithm, lazy loading so modules load on demand, and a catch-all <code>*</code> for not-found. <code>useParams</code> reads the id; <code>useSearchParams</code> keeps selected settings in the URL so they're shareable."
"Show me how the API is called."	the shared <code>api()</code> helper	"One function for every call: sets <code>Content-Type</code> , attaches <code>Authorization: Bearer <token></code> when a token exists, treats 401 as session-expired, and throws on non-2xx so the component can show an error. Putting it in one place stops the auth header drifting between screens."
"Show me how pause works."	the reducer + the scheduling <code>useEffect</code>	The 7-step flow from file 12 (AlgoViz, section 11): dispatch PAUSE → <code>playing: false</code> → the effect returns early and its cleanup has already cleared the timeout → the same frame stays on screen. "Nothing is paused; the algorithm already finished. I stopped scheduling the next cursor increment."
"Show me how speed adjustment works."	same effect	" <code>const delay = 1000 / state.speed</code> — 1x is a 1000 ms step, 2x is 500 ms. The timer is rescheduled whenever the cursor, playing flag or speed changes, and cleanup cancels the previous one, so there's never a stale frame."
"Show me your Flask endpoint."	<code>app.py</code> prediction route	"Model loaded once at startup with <code>joblib.load</code> — not per request. <code>request.get_json</code> , build the feature vector with the saved encoders, <code>predict_proba</code> , sort descending, return the top five. Invalid payload returns 400."

They say	File / concept to look for	What to explain while showing
"Show me your Random Forest model."	training script	" <code>train_test_split(stratify=y)</code> so every role appears in both splits, the preprocessing pipeline (one-hot for degree/major, scaling for CGPA, multi-label binarizer for skills), <code>RandomForestClassifier</code> with a fixed <code>random_state</code> , then <code>joblib.dump</code> the model and the fitted encoders together so serving can't drift from training."
"Show me your database."	<code>sql/schema.sql</code> or the entity classes	Start at <code>users</code> , then <code>transactions</code> , then <code>escrows/ledger_entries/transaction_history</code> . Call out <code>NUMERIC(12,2)</code> , the <code>CHECK</code> on state, the unique constraint on <code>idempotency_key</code> , the FKs, and the indexes on the FK columns.
"Show me where you did authorization (not just authentication)."	<code>SecurityConfig</code> + the ownership check in the service	"Two layers. Path rules in the config — <code>/api/admin/**</code> needs <code>ADMIN</code> . And in the service, an ownership check: the acting user must be the buyer or seller on <i>that</i> transaction, otherwise 403. Roles alone don't stop one buyer touching another buyer's escrow."

If you get stuck:

"I know the logic and where it lives, but I'd rather find the exact file than guess the class name — let me show you the method in the escrow service that enforces the transition."

PART B — "WHY?" (the follow-up machine)

The pattern: whatever you say, they ask "why?" — often twice. Pre-load these.

Question	Your answer (short, natural)
Why Spring Boot?	"Auto-configuration, starters and an embedded server — so I write business logic instead of configuring a servlet container and a data source, and I still get transactions and security as first-class features."
Why not plain Java servlets / Node?	"Servlets mean writing all that plumbing myself. Node is fine, but for a money-flow domain I want static typing, compile-time checking and Spring's transaction management."
Why PostgreSQL?	"ACID transactions across multiple rows, real constraints at the database level, exact <code>NUMERIC</code> for money, and it's the standard choice in Java shops."

Question	Your answer (short, natural)
Why not MySQL / MongoDB?	"MySQL would also work; Postgres has stronger standards compliance and richer types. MongoDB is out for this because I need multi-document transactions and relational integrity between escrow, ledger and history."
Why JWT?	"The API is stateless and consumed by a SPA, so the identity has to travel with each request. The server verifies a signature instead of looking up a session, which scales without shared session storage."
Why not sessions?	"Sessions need server-side storage and shared state across instances, and they fit browser cookie flows better. JWT fits a REST API. The trade-off is revocation — sessions can be deleted, tokens can't without extra machinery."
Why JPA/Hibernate?	"I declare the mapping once and work with objects instead of JDBC and manual row mapping; relationships and change tracking come for free. I still need to understand the SQL it generates — that's the cost, and it's why I check queries."
Why not raw JDBC / jOOQ?	"For this domain the entity graph is simple and the repository abstraction saves a lot of code. If I needed very tight control over generated SQL for a hot path, I'd drop to a <code>@Query</code> or native SQL."
Why React?	"The UI is state-driven — role- and status-dependent actions in RevertPay, and frame-by-frame rendering in AlgoViz. A component model with declarative rendering fits both far better than manual DOM manipulation."
Why not Angular/Vue?	"React's model and ecosystem are what I know best, and for AlgoViz the ecosystem around it — Vite, React Router, Tailwind — covers everything the project needs."
Why separate frontend and backend?	"Two deployables, one contract. The frontend can change without redeploying the API, other clients can reuse the API later, and each side can be tested independently. The cost is CORS, an API contract to maintain, and needing both running."
Why Vite?	"Instant dev startup and hot module replacement — it serves ES modules on demand instead of bundling everything up front — and a fast Rollup production build."
Why Tailwind?	"No inventing class names, a consistent spacing/colour scale by default, and only the used classes end up in the production CSS. Repeated patterns become components."
Why Random Forest?	"Small, tabular, mixed categorical/numeric data — tree ensembles handle non-linearity and feature interactions without me engineering them, train in seconds, and give feature importances."

Question	Your answer (short, natural)
Why not a neural network?	"It would need much more data and tuning for no clear gain on a dataset this size, and I'd lose interpretability."
Why two services (Java + Python)?	"The model ecosystem is Python; the web application needs Java's typing, transactions and security. Rather than forcing one language to do both badly, I split them — and I pay for it with a network hop and a contract to keep aligned."
Why escrow at all?	"Because neither party wants to move first. A neutral hold of the funds until both conditions are met removes the trust problem."
Why a ledger instead of a balance field?	"A stored balance can drift out of sync with the history it's supposed to represent. Deriving it from append-only entries makes the ledger the single source of truth and makes auditing just re-running the arithmetic."
Why a state machine and not just an enum field?	"Because legality has to be enforced in one place. Without it, any code path can set a status, and you get states that should be impossible."
Why record frames in AlgoViz?	"So playback is data, not computation. Pause, seek and step all become cursor moves, and going backwards becomes possible."
Why Git/GitHub?	"History, reversible changes, branch isolation, and a reviewable record of why each change happened — plus the repo is the evidence for everything on my resume."

The meta-answer when you're stuck on a "why":

"The honest reason is that it was the right level of tooling for a project at that scale — I didn't need more, and I wanted to understand what I was using rather than hide behind abstraction. If the requirements changed in [specific way], I'd reconsider [specific alternative]."

PART C — "WHAT IF?" (failure-mode thinking)

This is where you show judgement. Rule: **name what happens** → **name the guard** → **name what you'd improve**.

Attack	Your answer
"What if the database is down?"	"Spring throws a data-access exception on the connection attempt. My global handler maps unexpected failures to a 500 with a generic message and logs the detail server-side — no internals leak to the client. The user sees a retryable error, and the transaction never starts, so nothing is half-written."
"What if the API call fails in React?"	"The shared <code>api()</code> helper throws on non-2xx, the component catches it and renders an error state instead of a blank screen. 401 specifically clears the token and sends the user back to login, because retrying with a dead token is pointless."

Attack	Your answer
"What if the JWT expires mid-session?"	"The server returns 401 on the next request; the client treats 401 as 'session expired', clears the token and redirects to login. For a smoother experience I'd add a short-lived access token plus a refresh token."
"What if the user is unauthorized?"	"403, not 401 — the difference matters. 401 means log in; 403 means you're recognised but this isn't yours. In RevertPay the service checks that the acting user is the buyer or seller on that specific transaction."
"What if two requests arrive at the same time?"	"The database transaction is the guard. Both read the escrow; the state check allows only one to see a legal transition. The @Version optimistic-lock column makes the loser fail with a locking exception, which I map to 409 — so a double release is rejected, not silently applied twice. SELECT ... FOR UPDATE or a unique constraint on the ledger entries are the stronger alternatives."
"What if the entity doesn't exist?"	"findById returns an Optional; I call orElseThrow with a not-found exception, and the advice handler maps it to 404 with a clean error body. Never a 500, and never a NullPointerException leaking out."
"What if the model gives a bad prediction?"	"It's a recommendation, not a guarantee — that's why it returns ranked roles with confidence rather than one asserted label, and why the confidence shouldn't be presented as certainty. The real fix is measurement: top-k accuracy and calibration metrics, then retraining on better data."
"What if the user enters invalid input?"	"@Valid on the request DTO with constraints like @Email, @NotBlank, a CGPA range and an amount check. A failure produces a 400 with field-level messages before any business logic runs. The Flask service validates its payload too, because it's a separate service that could be called by anything — and I also validate at the React form level for a better experience, while treating that as convenience, not security."
"What if React receives an error?"	"The component renders an error state with a retry action; the error is logged in development. The important thing is that a failed request never leaves the UI in a false success state — a 'saved' toast must only appear after a 2xx."
"What if the algorithm is wrong?"	"The frames are generated from the actual algorithm implementation, and the tests compare outputs against simple oracles — for example the sliding-window result against a brute-force check across many inputs. That doesn't prove every input, which is why I say verified with oracles rather than proven correct."
"What if the user clicks pause twice?"	"It's idempotent — the reducer sets playing to false again and there are no timers to double-cancel, because the effect only creates a timer when playing is true. That's the benefit of the reducer: there's no counter to corrupt."

Attack	Your answer
"What if the server crashes mid-request?"	"If the transaction hadn't committed, the database rolls it back on recovery — atomicity is exactly for this. If it had committed and the response was lost, the client's retry must be safe, which is what the idempotency key is for; stored-response replay is the remaining gap I'd close."
"What if someone sends a huge payload?"	"The request DTO's constraints and the container's body-size limit reject it before it reaches business logic. Rate limiting at the edge is the next layer if abuse were a concern."
"What if the Flask service is slow or down?"	"Spring's outbound call fails or times out and I map it to 503 with a clear message, so the user knows predictions are temporarily unavailable rather than seeing a generic server error. Setting an explicit timeout is the next hardening step — it's listed in the project README."
"What if the same student submits the same profile twice?"	"Two predictions, two history rows. Predictions are additive, so duplication is harmless here — unlike a money operation, which is exactly why idempotency matters in RevertPay and not here. Recognising where idempotence is actually required is part of the design."
"What if two users have the same email?"	"The unique constraint on <code>users.email</code> rejects it at the database level, and I check before insert to return a friendly 409 instead of a raw constraint violation. The database constraint is the guarantee; the check is for a good error message."
"What if a malicious user edits the JWT payload to become ADMIN?"	"The signature check fails — the payload is signed with the server's secret, so any modification invalidates it. The service rejects the token and returns 401. And since the payload is only base64-encoded and readable, I keep nothing sensitive in it."
"What if the frontend sends a wrong field name?"	"It's a 400 — unknown or missing required fields fail validation. And in <code>EDU2JOB</code> specifically, a mismatch across the Java/Python boundary is the failure mode I'm most careful about, because there the model can return a confident wrong answer with no error at all."
"What if the money doesn't balance?"	"The ledger is double-entry, so per transaction the debits must equal the credits. A <code>GROUP BY transaction_id HAVING SUM(direction-signed amount) <> 0</code> query surfaces any imbalance, and that belongs in an automated check — not just a manual query."

The fallback for any "what if" you don't know:

"I haven't hit that case. My assumption would be [X], which I'd verify with a test before shipping — and if I'm wrong, the fix is probably at [layer], because that's where that class of problem belongs." That answer is honest, structured, and better than a guess.

Next: [14-bullet-attack-nobluff.md](#)

SECTION 14

14 — RESUME BULLET ATTACK + THE NO-BLUFF SYSTEM

Time: 12 min. Every bullet on your resume, decoded.

Format for each bullet: **What it means** → **technical truth** → **how you implemented it** → **how you prove it** → **the trap** → **the ideal answer**. [FULL] = you can claim this fully · [PARTIAL] = claim it with a stated boundary · [AVOID] = do not overclaim

PART A — THE DANGER RANKING (your Part 33)

Rank	Claim	Why it's dangerous	What you MUST know	Say this and no more
EXTREME	JWT authentication + role-based authorization	Fakeable in one sentence; three follow-ups expose it	header/payload/signature; signed-not-encrypted; filter validates signature + exp; roles → authorities; 401 vs 403; ownership checks	The full flow in file 06, §6 and §9. Never say "encrypted".
EXTREME	Escrow workflow with 6 states	If "escrow" is fuzzy, the whole flagship project collapses	escrow = neutral hold until conditions met; state machine = legal edges only; each state has a guard	The lifecycle in file 12, plus "the server decides the next state, the client can't post a status."
EXTREME	"Designed entity relationships"	They will ask you to draw it live	5 tables: users(×2 roles) → transactions → escrow + history; ledger entries; FK ownership	Draw file 12 §11 from memory. Twice.
EXTREME	Spring Data JPA + JPA/Hibernate	ORM is where beginners get destroyed	JPA = spec, Hibernate = impl, Spring Data = repository layer; @Entity/@Id/@ManyToOne(mappedBy)	File 04 §3 and §6.
HIGH	"Users / transactions / escrow / history" (schema claim)	Any column question must have an answer	why NUMERIC, why @Version, why unique idempotency_key, why no balance column	File 05 create-table block.
HIGH	Random Forest	"Why not one tree?" kills the unprepared	bagging + feature randomness; averaging probabilities; advantages/limits	File 10 §E, especially "two sources of randomness."
HIGH	"Integrated a React frontend with the backend"	Reveals whether you understand the API contract	fetch, JSON, Authorization header, 401 handling, props/state	File 07 §"API calls" + the api() helper.
HIGH	500+ LeetCode	They can hand you a problem now	two-sum with complexity; Big-O table; BFS vs DFS; HashMap	File 11 §B. Practise two-sum out loud.

Rank	Claim	Why it's dangerous	What you MUST know	Say this and no more
HIGH	Spring Security	Barely mentionable without the filter chain	filter chain order; permitAll for auth paths; STATELESS; why CSRF is off	File 06 §7 — be able to explain each line.
IMPORTANT	Node.js / Express	You may be much stronger in Spring, and the follow-up expects parity	Node runtime; middleware; <code>(req, res, next)</code> ; Spring-vs-Express comparison	File 09 — and be honest that Spring is where your depth is. [PARTIAL]
IMPORTANT	scikit-learn / Pandas / NumPy / preprocessing	Standard ML follow-up chain	DataFrame; fit-on-train-only + data leakage; one-hot vs ordinal	File 10 §B-§C.
IMPORTANT	Flask / Python service	"Why two runtimes?"	Java owns web/auth/DB; Python owns the model; browser never calls Flask	File 09 §C.
IMPORTANT	Hooks / React Router / reusable components	Binary questions	useState/useEffect/useReducer; <code>:id</code> params; props-in/dispatch-out	File 09 §B; Ref;
IMPORTANT	Vite / Tailwind	Easy, but don't oversell	on-demand ES modules + HMR; utility-first + trade-offs	File 08 §C-§D.
IMPORTANT	Data Structures and Algorithms / OOP	Foundational, guaranteed	four pillars with project examples; Big-O	Files 01 §H-§I, 11 §B.
Medium	HTML / CSS / responsive	Rarely deep	box model, flex vs grid, media queries, <code>prefers-reduced-motion</code>	File 08 §A-§B.
Medium	Git / GitHub	Free marks if prepared	commit vs push, branch, merge conflict, <code>reset</code> vs <code>revert</code>	File 11 §A.
Medium	CGPA 8.51 / certifications	Low risk	one sentence each, no defensiveness	"CodeTantra for OOP in Java, NPTEL for DSA in Python — both consolidated fundamentals I use daily."

PART B — BULLET-BY-BULLET

REVERTPAY

[FULL] B1. "Implemented an escrow workflow covering funding, shipping, delivery, release, expiry, transaction review"

Means: you designed and coded the lifecycle of a payment held by a neutral party. **How you implemented it:** a status field on the escrow with a validating transition service; each action endpoint maps to one legal edge; expiry is a scheduled job; disputes move to review where an admin resolves. **Prove it:** `EscrowService` + the transition rules + the history table showing `from_state` → `to_state` with an actor and timestamp. **Trap:** "Why not let the client send the new status?" → "Because then the client controls money. The API exposes actions, and the server decides the resulting state after validating the current one — the request DTO doesn't even contain a status field." **NOTE: Honest boundary:** your case study says the simulator is a portfolio-only visualization and the live demo isn't deployed. If asked how it was verified, use the testing answer from file 12 §14.

[FULL] B2. "Developed REST APIs with Spring Data JPA and secured application access using JWT authentication and role-based authorization"

Means: two separate things — the endpoints and the security layer. **Prove it:** `SecurityConfig` (path rules), the JWT filter, and one controller method showing the delegation to the service. **Trap:** "Role-based — so what stops a BUYER calling the seller's endpoint?" → "Roles and ownership are separate. A BUYER passing the role check still has to be the buyer *on that transaction*; the service verifies it and returns 403 otherwise." **NOTE: If pushed on depth:** "The token carries the role as a claim; the filter turns it into an authority, and the config maps paths to roles."

[FULL] B3. "Designed entity relationships using JPA/Hibernate and PostgreSQL to persist users, transactions, escrow details, transaction history"

Prove it: the entity classes with their annotations, and the ER diagram. **Trap:** "What's the difference between the escrow and the transaction?" → "The transaction is the payment record — parties, amount, status. The escrow row holds the lifecycle state and expiry. Keeping them separate means the money record doesn't get repurposed for workflow state." **NOTE: Be ready:** if your actual code merges these, say "in my implementation the escrow row holds the lifecycle state and the transaction holds the payment record" — and don't invent a mapping your code doesn't have.

[PARTIAL] B4 (only if you raise it). "Ledger with derived balances / idempotency key / optimistic locking / scheduled expiry"

These are in your public case study, not your resume bullets. You may raise them — they're genuinely impressive — but the moment you do, you own them:

- *Ledger:* paired debit/credit entries; $\text{balance} = \Sigma \text{credits} - \Sigma \text{debits}$; no mutable balance column.
- *Idempotency key:* a unique constraint so a retried create can't duplicate an escrow. NOTE: Honest limit: stored-response replay is still roadmap.
- *Optimistic locking:* `@Version` column; a conflicting concurrent update fails instead of losing data. NOTE: Don't say "pessimistic locking" — you didn't claim it.
- *Scheduled expiry:* a job that expires stale escrows. NOTE: Fully safe to mention (it's in your case study); if asked how it's implemented, describe it as a scheduled job that queries expired escrows and transitions them. **If you're unsure about any of these, don't volunteer it.** Answering well about the state machine and transactions is worth far more than an uncertain ledger tangent.

EDU2JOB (Infosys Springboard)

[FULL] B1. "Developed a job-role recommendation system using Python and Random Forest based on user education, CGPA and skills"

Prove it: the training script — split, preprocessing pipeline, classifier, `predict_proba`, `joblib.dump`. **Trap:** "Why Random Forest?" → file 10 §E. "What accuracy?" → **Do not invent.** Use the honest answer from file 12 §17. **Trap:** "Is CGPA really predictive of a job role?" → "Probably weakly on its own — that's exactly why it's one feature among several and why the output is a ranked shortlist with confidence rather than a verdict. And a model is only as good as its labels: if the training labels came from subjective mapping, the model inherits that. Measuring that is the evaluation work still to do."

[FULL] B2. "Built a Spring Boot backend with REST APIs for user authentication, profiles, job predictions, prediction history"

Prove it: four controller groups; the prediction endpoint is the one that crosses to Flask. **Trap:** "How is prediction history different from a log?" → "It stores the user, the exact inputs and the raw model response, so I can show the student what they asked and what came back — and debug what the model was actually given." **NOTE: Honest boundary:** your case study says RBAC is roadmap, not shipped. Every user acts on their own data, so say ownership checks, not roles.

[FULL] B3. "Integrated a React frontend with the backend and PostgreSQL database to provide a complete user workflow"

Prove it: the shared `api()` helper and the profile → predict → history screens. **Trap:** "Did React talk directly to Flask?" → **No. React only calls Spring Boot; Spring calls Flask. One entry point means auth and validation live in one place, and the model service isn't publicly exposed.** This is the sentence that proves architectural understanding.

ALGOVIZ

[FULL] B1. "Interactive algorithm visualization platform using React.js to demonstrate sorting and data-structure algorithms"

Prove it: the frame generators + the module list. **Trap:** "How many algorithms?" → Answer honestly with the modules you actually have (sorting lab plus the other modules in the app). **Don't list Dijkstra or Floyd's — your case study explicitly says they're not modules.**

[FULL] B2. "Controls for algorithm execution, speed adjustment, pause/resume, step-by-step visualization"

Prove it: the reducer + the scheduling effect. **Trap:** "What happens when you click pause?" → the 7-step flow. **Highest-probability question in this project.**

[FULL] B3. "Responsive interface using React, Tailwind CSS, React Router with reusable components"

Prove it: the routes file + `PlaybackControls` + the layout classes. **Trap:** "Why Tailwind?" → file 08 §C. "Why the router?" → shareable per-algorithm URLs + lazy loading.

PART C — THE NO-BLUFF SYSTEM (your Part 34)

The three-sentence protocol

SAY: "I know [the level above]. I haven't worked with [the specific thing]. I'd find out by [concrete action]."

Then **stop talking**. Silence is fine. Guessing is fatal.

Worked examples

"Explain Hibernate's persistence context."

"What I know is that within a transaction Hibernate keeps the loaded entities in a first-level cache — the persistence context — and tracks changes on them, so it can flush the right `UPDATE` automatically. I've relied on that behaviour, but I haven't tuned eviction or second-level caching." NOTE: Don't claim: sessions, `clear()/flush()` strategies, second-level cache providers.

"How does Hibernate decide when to flush?"

"At commit, and before a query that needs the pending changes visible. I'd have to check the exact flush-mode ordering rather than guess."

"What isolation level does your code use?"

"Whatever the database default is — Postgres defaults to `READ COMMITTED`. I didn't set it explicitly, so I'd verify that rather than claim it."

"Did you write any tests?"

"Yes on the portfolio side — the algorithm frame generators are covered by tests, including a sliding-window check against a brute-force oracle across many inputs. On the RevertPay backend, the state-transition matrix and the concurrency race test are written up as the next step, not done. I'd rather tell you that than pretend." [YES] **This is the single most important honest answer you'll give.** It converts your weakest point into evidence of judgement.

"Do you know Docker/Kubernetes/Kafka/Redis/microservices?"

"Not hands-on. I know the ideas — containers for consistent environments, a service registry and orchestration for many services, queues for decoupling, caching for read load — but I haven't built with them, so I wouldn't claim experience. My projects are single-service deployments, and that's the honest boundary." NOTE: **They are not on your resume — a "no" costs you nothing here.** Bluffing would cost you the interview.

"What's the most complex thing you've built?"

"Probably the escrow service in RevertPay — not because the code is large, but because the correctness constraints overlap: state validation, two ledger legs, an audit record, ownership checks and concurrency protection all have to hold at the same time, inside one transaction."

"What's your biggest weakness?"

"I'm still building depth in production concerns — testing strategy, deployment, observability. My projects taught me to design systems, but they didn't have monitoring or a real incident. What I've started doing about it is writing tests first where a silent wrong answer is possible, because that's the failure mode I'm most worried about — a wrong result that doesn't throw."

Rules of honesty

1. **Never invent a class name, a metric or a library version.**
2. **Never claim production experience** — you have three projects and an internship. That's plenty.
3. **Never claim a technology not on your resume** unless asked directly, and then answer "not hands-on."
4. **"I don't know, but here's how I'd find out" scores 3-4/5. Bluffing scores 0 and taints everything after it.**
5. **If you catch yourself mid-bluff, stop and say:** "Actually, let me be precise — what I know is X, and I'd want to verify the rest."

The five things you must never say

1. "The accuracy was 94%" (no measurement exists).
2. "It's fully unit tested" (the suites are planned).
3. "JWT encrypts the payload."
4. "I deployed it and handled thousands of users."
5. "Yes" to a technology question you can't then explain.

Next: [15-cheatsheet-rapidfire.md](#)

SECTION 15

15 — THE FINAL INTERVIEW CHEAT SHEET + RAPID FIRE

This is the file to re-read in the last 15 minutes and in the car. Dense on purpose. Every line is a complete answer.

★ TOP 30 QUESTIONS I MUST KNOW

1. **Tell me about yourself.** → The 60-second intro in [00-START-HERE](#)§3. Three projects, the stack, no filler.
2. **Explain your strongest project.** → RevertPay 1-minute version ([12](#)).
3. **What is OOP?** → Data + behaviour bundled in objects, with encapsulation, abstraction, inheritance, polymorphism to keep change contained.
4. **Four pillars?** → Encapsulation = hide internals, control legal changes. Abstraction = simple contract over complexity. Inheritance = reuse via is-a. Polymorphism = one call, correct behaviour per type.
5. **What is Java?** → A language + a runtime: compile to bytecode once, the JVM runs it anywhere.
6. **JDK/JRE/JVM?** → Dev tools / runtime environment / the engine that runs bytecode.
7. **== vs equals?** → `==` identity (reference), `equals` content. Use `equals` for strings.
8. **Why is String immutable?** → Security, string pooling, safe map keys, thread safety.
9. **Collections?** → List (ordered, dups), Set (unique), Map (key→value). ArrayList O(1) get; HashMap O(1) avg lookup via hashing.
10. **Interface vs abstract class?** → Capability contract + multiple inheritance vs shared state/code + single parent.
11. **What is Spring?** → A Java framework; the container creates and injects your objects (IoC/DI) instead of you wiring them.
12. **What is Spring Boot?** → Spring without the manual configuration: auto-configuration, starters, embedded Tomcat.
13. **Dependency injection?** → Dependencies are provided from outside, usually via constructor — explicit, final, and testable without Spring.
14. **What is a bean?** → An object the Spring container creates and manages. Default scope is singleton.
15. **What is a REST API?** → Resources with URLs, standard HTTP methods, stateless requests, JSON representations.
16. **GET vs POST?** → GET reads and must be safe/idempotent; POST creates or triggers actions and isn't idempotent.
17. **Status codes?** → 200/201/204 ok-created-no content · 400 bad input · 401 not authenticated · 403 not permitted · 404 missing · 409 state/duplicate conflict · 500 server bug.
18. **What is JPA?** → A specification for object-relational mapping. **Hibernate** is the implementation. **Spring Data JPA** generates repositories.
19. **What is ORM?** → Maps classes→tables and objects→rows, generating SQL so you work in objects.
20. **Entity relationships?** → `@ManyToOne` holds the FK; `@OneToMany` uses `mappedBy`; `@OneToOne` + unique; many-to-many needs a join table.
21. **What is @Transactional?** → Wraps the method in one DB transaction: all changes commit or all roll back. Unchecked exceptions roll back by default.
22. **What is authentication?** → Proving who you are (login, JWT).
23. **What is authorization?** → What you may do (roles + ownership checks).
24. **How does JWT work?** → header.payload.signature; the server signs with a secret and verifies signature + expiry on each request; payload is readable, not encrypted.
25. **Why BCrypt?** → One-way, salted, deliberately slow — a leaked table isn't crackable and identical passwords produce different hashes.

26. **What is React?** → A library for state-driven UI with components; you describe the UI for the current state and React updates the DOM.
27. **Props vs state?** → Props come from the parent and are read-only; state is owned by the component and changing it re-renders.
28. **What is Random Forest?** → Many decision trees, each trained on a bootstrap sample with a random feature subset at each split; probabilities are averaged, reducing variance.
29. **What is Big-O?** → How time/memory grows with input size, ignoring constants. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$.
30. **Why should we hire you?** → "I have solid Java and Spring fundamentals, three projects where I had to make real design decisions — a state machine, a two-service contract, and a playback engine — and I prioritise correctness where a wrong answer is silent. I'm early in my career, which means I'll absorb your codebase and standards quickly, and my GitHub shows I finish things."

★ TOP 20 PROJECT QUESTIONS

1. **Walk me through RevertPay in a minute.** → 12, RevertPay \$2.
2. **What is escrow and why does it exist?** → A neutral party holds funds until the conditions are met, so neither side has to trust the other.
3. **Walk me through the escrow lifecycle.** → created → funded → shipped → delivered → released, with dispute → review → resolve/refund, plus scheduled expiry.
4. **What's the state machine and why?** → Legal transitions in one place; no code path can set a status directly, so impossible states don't exist.
5. **What happens on two simultaneous releases?** → The transaction's state check rejects one; @Version optimistic locking makes the loser fail → 409. Never a silent double release.
6. **Why a ledger instead of a balance column?** → A stored balance can drift from history; deriving it makes the ledger the single source of truth and auditing just re-running arithmetic.
7. **Draw your database.** → 12, §11 diagram (5 tables).
8. **Which endpoints need which role?** → Parties act on their own transactions (BUYER funds/delivers/disputes, SELLER ships); ADMIN resolves reviews; ownership is checked separately from role.
9. **How did you end up with this architecture?** → Layered for separation of concerns: HTTP in the controller, rules + transaction boundary in the service, persistence in repositories.
10. **What was hardest?** → Keeping state, ledger and audit consistent inside one transaction under concurrency.
11. **What's your biggest bug story?** → The double-release race, or the client-controlled status field (12, §8-10).
12. **How did you handle money types?** → BigDecimal in Java and NUMERIC(12,2) in Postgres; never floats.
13. **What's the difference between your transaction and escrow tables?** → The transaction is the payment record (parties, amount, status); the escrow holds lifecycle state and expiry.
14. **What happens if a token is invalid or expired?** → 401 from the entry point; the client clears the token and redirects to login.
15. **What about EDU2JOB — walk me through a prediction.** → 12, EDU2JOB \$2 and §4: React → Spring (auth, validate) → Flask (encode, predict_proba, calibrate, top-5) → Spring stores history → React renders.
16. **Why Python for the model?** → The ML ecosystem is Python-first; the model artifacts and the fitted encoders stay together with the code that runs them.
17. **Why not call the model from React?** → One entry point means auth and validation in one place, and the model service stays unexposed.
18. **How is AlgoViz not just an animation?** → The frames are generated by the actual algorithm implementations with stable value ids, and playback is data — seekable, steppable, and comparable by

recorded operation counts.

19. **How does pause work in AlgoViz?** → The reducer sets `playing: false`; the effect returns early and its cleanup clears the timer, so the cursor stops. The algorithm already finished — nothing is being paused.

20. **What's the first thing you'd improve in each project?** → RevertPay: concurrency integration tests + retry-safe idempotency. EDU2JOB: the evaluation harness (top-k, calibration) + a client timeout. AlgoViz: compact event traces instead of full snapshots.

★ TOP 20 JAVA / SPRING QUESTIONS

1. What is the JVM and why does bytecode exist?
2. Difference between `==` and `equals`, and the `hashCode/equals` contract.
3. Why is String immutable?
4. ArrayList vs LinkedList, and when you'd use a Set instead of a List.
5. How does HashMap work and what's the worst-case lookup?
6. Overloading vs overriding.
7. `final` on a variable, method, class — and why a final list is still mutable.
8. Access modifiers — why fields are private with public accessors.
9. Interface vs abstract class; why Java has single inheritance but multiple interfaces.
10. Checked vs unchecked exceptions; what `finally` guarantees.
11. What is `static` and why can't a static method use an instance field?
12. What are generics for? (Compile-time type safety, no casting.)
13. What is IoC and what is a bean?
14. Why constructor injection over field injection?
15. Why must a `@Service` be stateless? (Singleton scope.)
16. `@Component` vs `@Service` vs `@Repository` vs `@RestController`.
17. What does `@SpringBootApplication` combine, and is auto-configuration conditional?
18. Why can a `@Transactional` method silently lose its transaction? (Self-invocation bypasses the proxy.)
19. Which exceptions roll back by default, and how do you change that?
20. What happens if two beans share a type? (`@Primary` / `@Qualifier`.)

★ TOP 15 REACT QUESTIONS

1. What is React and what problem does it solve? (Declarative, state-driven UI.)
2. What is a component?
3. Props vs state.
4. What is JSX and what does it compile to?
5. Why can't you mutate state directly?
6. `useState` — and why the functional updater form?
7. `useEffect` — the three dependency-array cases.
8. Why does an effect need a cleanup function?
9. Why do keys matter, and what breaks with index keys?
10. When would you use `useReducer` instead of `useState`?
11. What is `useRef` for?
12. What is lifting state up?
13. Event handling and controlled inputs.
14. What does React Router give you, and what do `useParams/useSearchParams` do?

15. What is the virtual DOM / reconciliation?

★ TOP 10 SQL QUESTIONS

1. WHERE vs HAVING.
2. INNER vs LEFT JOIN — and how a LEFT JOIN keeps users with zero rows.
3. Find duplicate emails.
4. Users with no transactions (anti-join).
5. Primary key vs foreign key vs unique.
6. What an index is and why FKs need one in Postgres.
7. What is a transaction, and what does ACID stand for?
8. Why `NUMERIC` and not `FLOAT` for money?
9. Write the ledger-balance check (`GROUP BY transaction_id HAVING SUM(...) <> 0`).
10. Normalisation — what 1NF/2NF/3NF prevent (duplication and update anomalies).

★ TOP 10 SECURITY QUESTIONS

1. Authentication vs authorization.
2. JWT structure and what each part does.
3. **Is a JWT encrypted?** (No — signed, base64-encoded, readable.)
4. What exactly does the server verify per request?
5. Why BCrypt, and what does the salt do?
6. Why hashing instead of encryption for passwords?
7. 401 vs 403. Which do you return for a buyer touching someone else's transaction?
8. Why is CSRF disabled, and why is that acceptable for a stateless bearer-token API?
9. How do you log out with JWTs, and what's the honest limitation?
10. Roles vs ownership — why a role check alone isn't authorization.

★ TOP 10 ML QUESTIONS

1. Traditional programming vs machine learning.
 2. Feature, label, sample, model, prediction.
 3. Classification vs regression — which is EDU2JOB?
 4. What is preprocessing and why is it needed?
 5. One-hot vs ordinal encoding — why not 1/2/3?
 6. Why fit transformers on the training set only? (**Data leakage.**)
 7. What is a decision tree and how does it choose a split?
 8. What are the two sources of randomness in a Random Forest?
 9. How does a forest produce a probability?
 10. Two advantages and two limitations of Random Forest, and why not a neural network here.
-

★ TOP 10 QUESTIONS THAT CAN EXPOSE BLUFFING

(Each with the correct, safe answer.)

1. **"What accuracy did your model get?"** → No number is claimed; describe the pipeline and say the evaluation harness is the next step.
2. **"Show me your test suite."** → Portfolio algorithm tests exist; the RevertPay state-matrix and race tests are planned. Say that.
3. **"Is your JWT encrypted?"** → **No**. Signed, readable payload, secret never in the client.
4. **"Where do you store the token, and is that safe?"** → Browser storage; the honest trade-off is XSS exposure vs the CSRF concerns of cookies.
5. **"Explain the persistence context and dirty checking."** → First-level cache/unit of work per transaction that auto-flushes changed managed entities. Stop there — don't invent eviction tuning.
6. **"How does your state machine handle concurrency?"** → `@Transactional` + `@Version` optimistic locking → the loser gets 409. Don't claim pessimistic locking.
7. **"Is this deployed and used?"** → No. Repository-verified. RevertPay and EDU2JOB have no live deployment.
8. **"Which ORM methods did you write? Show me."** → Open the repository interface; derived queries plus a `@Query` with `JOIN FETCH`.
9. **"Have you used Docker / Kubernetes / Kafka / Redis?"** → Not hands-on. Don't claim it; none of it is on the resume.
10. **"You claim 500+ problems — solve this."** → Two-sum with the hash-map approach, said out loud with complexity. Practise it until it's automatic.

★ TOP 10 "WHY?" QUESTIONS

1. Why Spring Boot? 2. Why PostgreSQL? 3. Why JWT and not sessions? 4. Why JPA/Hibernate and not JDBC? 5. Why React? 6. Why Random Forest? 7. Why two services (Java + Python)? 8. Why a ledger instead of a balance column? 9. Why a state machine? 10. Why record frames in AlgoViz? → All ten answers are in `13-...md`, Part B. **That's the list to memorize cold.**

★ TOP 10 "WHAT IF?" QUESTIONS

1. What if the database is down? 2. What if the API call fails in React? 3. What if the JWT expires? 4. What if the user is unauthorized? 5. What if two release requests arrive together? 6. What if the entity doesn't exist? 7. What if the model predicts badly? 8. What if the user enters invalid input? 9. What if Flask is down? 10. What if the frontend and model disagree about field names? → All ten in `13-...md`, Part C. **This is your judgement showcase.**

⚡ RAPID FIRE — SAY THESE OUT LOUD, 1-2 SECONDS EACH

Say the answer immediately. If you hesitate more than 2 seconds, mark it and go back.

Q	A
What is Java?	Bytecode language + JVM; write once, run anywhere
What is JVM?	The engine that executes bytecode
JDK vs JRE?	Dev tools vs runtime
What is OOP?	Organising code as objects with data + behaviour
Encapsulation?	Hide internals, control legal changes
Abstraction?	Simple contract over hidden complexity
Inheritance?	is-a reuse from a parent class
Polymorphism?	One call, behaviour chosen by the actual type
<code>==</code> vs <code>equals</code> ?	Reference vs content
hashCode contract?	Equal objects must have equal hash codes
Why is String immutable?	Security, pooling, safe map keys
ArrayList get?	$O(1)$
HashMap lookup?	$O(1)$ average, $O(n)$ worst case
List vs Set?	Ordered with duplicates vs unique
Interface vs abstract class?	Contract + multi vs shared state + single
Big-O?	Growth rate of time/memory with input size
$O(n \log n)$ algorithms?	Merge sort, heap sort, Timsort
Binary search?	Sorted input, halve the range, $O(\log n)$
BFS vs DFS?	Queue/level-order/shortest path vs stack/recursive/deep
Mark visited when?	On enqueue in BFS
What is Spring?	Java framework; the container wires your objects
What is Spring Boot?	Spring without manual config; auto-config + embedded server
What is IoC?	The container, not you, controls object creation
What is DI?	Dependencies injected from outside, usually via constructor
What is a bean?	A Spring-managed object (singleton by default)
<code>@Component</code> vs <code>@Service</code> ?	Same mechanism, different semantic layer
<code>@RestController</code> ?	Controller + automatic JSON response body
What is REST?	Resources + HTTP verbs + stateless + JSON

Q	A
GET vs POST?	Read/safe/idempotent vs create/non-idempotent
PUT vs PATCH?	Full replace vs partial update
Path vs query param?	Identifies the resource vs filters/shapes it
401 vs 403?	Not authenticated vs not permitted
404 vs 409?	Missing vs conflicts with current state
What is an API?	A contract for how programs talk
Statelessness?	The server stores nothing between requests
What is JPA?	The ORM specification
What is Hibernate?	An implementation of JPA
What is ORM?	Objects ↔ tables mapping
@Entity?	Marks a class as a mapped table
Which side owns the FK?	The @ManyToOne side
mappedBy?	Names the field on the owning side, not a column
Lazy vs eager?	Load on access vs load immediately
What is the persistence context?	Hibernate's per-transaction first-level cache
Dirty checking?	Auto-UPDATE for changed managed entities at flush
@Transactional?	All-or-nothing boundary around the method
Which exceptions roll back?	Unchecked (RuntimeException) by default
Optimistic locking?	@Version — a conflicting write fails instead of overwriting
What is a primary key?	Unique non-null row identifier; auto-indexed
What is a foreign key?	Column referencing another table's PK; enforces integrity
What is a JOIN?	Combining rows from tables on a matching key
INNER vs LEFT JOIN?	Only matches vs all left rows + matches
WHERE vs HAVING?	Row filter before grouping vs group filter after
What is an index?	Extra structure for fast lookups; slower writes
ACID?	Atomicity, Consistency, Isolation, Durability
Why NUMERIC for money?	Exact decimals; floats lose precision
Why PostgreSQL?	ACID, real constraints, rich types, open source

Q	A
What is authentication?	Proving who you are
What is authorization?	What you're allowed to do
JWT parts?	Header, payload, signature
Is JWT encrypted?	No — signed, base64-encoded
What does the server verify?	Signature, expiry, user still valid
Bearer token?	The <code>Authorization: Bearer <jwt></code> header
What is BCrypt?	Slow, salted, one-way password hashing
What is a salt?	Random per-password value stored in the hash
Spring Security core?	A servlet filter chain before the controller
Why disable CSRF here?	Token is sent explicitly, not auto-attached by the browser
What is React?	A library for state-driven component UIs
What is a component?	A function returning JSX
Props vs state?	Read-only from parent vs owned and mutable
<code>useState</code> ?	Value + setter; setting schedules a re-render
<code>useEffect</code> ?	Side effects after render; deps control re-runs
<code>useReducer</code> ?	Named state transitions for related state
<code>useRef</code> ?	Mutable value across renders without re-rendering
Why keys?	Stable identity for list items across renders
What is JSX?	JavaScript syntax that produces element objects
Virtual DOM?	An in-memory tree diffed to minimise real DOM updates
<code>map/filter/reduce</code> ?	Transform / keep / fold
<code>let</code> vs <code>const</code> vs <code>var</code> ?	Block reassignable / block fixed / function-scoped legacy
<code>===</code> vs <code>==</code> ?	No coercion vs coercion — always use <code>===</code>
Promise?	Placeholder for a future value: pending→fulfilled/rejected
<code>async/await</code> ?	Sync-looking promise handling; suspends the function, not the thread
Event loop?	Single-threaded loop processing callbacks; I/O is async
<code>fetch</code> and errors?	Only rejects on network failure — check <code>res.ok</code>
React Router?	URL → component mapping; real links, back button, params

Q	A
What is Node.js?	A runtime for JavaScript outside the browser
What is Express?	Minimal Node web framework; routes + middleware
Middleware?	(req, res, next) chain before the handler
Express vs Spring?	Minimal/dynamic/no structure vs opinionated/typed/DI+transactions
What is Python?	Readable dynamically typed language with the ML ecosystem
What is Flask?	Minimal Python web framework for endpoints
Why Flask here?	To expose the trained model as an HTTP service
What is Pandas?	Library for tabular data; the DataFrame
What is NumPy?	Fast typed arrays with vectorised operations
What is scikit-learn?	ML library with a consistent fit/predict API
What is a feature?	An input variable the model uses
What is preprocessing?	Cleaning + encoding + scaling raw data into a numeric matrix
Why one-hot encode?	Integer codes invent false order and distance
Data leakage?	Using test-set statistics during training — fit on train only
What is a decision tree?	Split on the most impurity-reducing question, repeatedly
What is Random Forest?	Many trees on bootstrap samples + random feature subsets, averaged
Why many trees?	Each tree's errors differ, so averaging lowers variance
What is overfitting?	Memorising training data, failing on new data
What is Top-k accuracy?	Whether the correct answer is in the top k predictions
Git vs GitHub?	Local version control vs remote hosting + collaboration
Commit vs push?	Local snapshot vs upload to the remote
Merge conflict?	Git can't pick between changes — you edit and re-commit
reset vs revert?	Rewrite history (local) vs undo with a new commit (shared)
What is a PR?	A request to merge a branch, with review and checks
What is Vite?	Dev server + bundler; on-demand ES modules and HMR
What is HMR?	Swapping only the changed module without a full reload

Q	A
What is Tailwind?	Utility-first CSS: small single-purpose classes in the markup
Box model?	Content, padding, border, margin
Flex vs Grid?	One-dimensional vs two-dimensional layout
Responsive design?	Layout adapts via fluid units and media queries

[TIME] FINAL 15-MINUTE REVISION

Minute 0-3 — Java + OOP JDK/JRE/JVM · `==` vs `equals` + `hashCode` contract · String immutability (security, pooling, map keys) · List/Set/Map + complexities · interface vs abstract class · four pillars, each with a *RevertPay* example (encapsulation = validated transitions, abstraction = service interface, inheritance = `JpaRepository`, polymorphism = interface implementations).

Minute 3-6 — Spring + REST + JPA Bean, IoC, DI, constructor injection · auto-configuration + embedded server · CLIENT→CONTROLLER→SERVICE→REPOSITORY→DB · annotations you claim · REST = resources + verbs + stateless + JSON · 200/201/400/401/403/404/409/500 · JPA spec vs Hibernate impl vs Spring Data repositories · `@ManyToOne` owns the FK · LAZY vs EAGER · `@Transactional` = all-or-nothing.

Minute 6-8 — PostgreSQL + SQL `NUMERIC` for money · PK auto-indexed, FK **not** auto-indexed · indexes speed reads, slow writes · ACID · transaction grouping state + ledger + audit · WHERE vs HAVING · INNER vs LEFT JOIN · the balance and ledger-imbalance queries.

Minute 8-10 — JWT + Security Authentication (who) vs authorization (what) · `header.payload.signature` · **signed, NOT encrypted** · server verifies signature + exp per request · `Authorization: Bearer` · BCrypt salted + slow · 401 vs 403 · roles **plus** ownership checks · CSRF disabled because the token is explicit, not browser-attached · logout = client deletes the token, with the honest limitation.

Minute 10-12 — React + JavaScript Component · props vs state · never mutate state · `useEffect` deps (empty = once, `[x]` = on change, none = every render) · keys and why index keys break reordering · `useReducer` for related transitions · `map/filter/reduce` · `async/await` + check `res.ok` · **AlgoViz: frames → cursor → render; pause = stop scheduling the next cursor step.**

Minute 12-14 — Projects *RevertPay*: escrow = neutral hold · `created→funded→shipped→delivered→released` (+dispute/review, expiry) · ledger with derived balance · one transaction per transition · `@Version` for concurrent release · JWT + roles + ownership. EDU2JOB: React → **Spring (auth, validate)** → Flask (encode with saved encoders, `predict_proba`, calibrate, top-5) → Spring stores history → React. Browser never calls Flask. AlgoViz: recorded frames, stable ids, reducer cursor, `delay = 1000/speed`, bounded input, honest about memory cost.

Minute 14-15 — The openers Self-introduction (\$00) · strongest project (*RevertPay* 1-minute) · why hire me · **then stop thinking and say them slowly.** Speed is not confidence. Slowness is.

SECTION 16

16 — MOCK INTERVIEW (RUN THIS WITH ME IN CHAT)

How this works: when you're ready, type "**START MOCK**" in the chat and I will run the interview as the interviewer — one question at a time, waiting for your answer before giving feedback.

If you'd rather self-test right now, this file contains the full question bank with the scoring rubric, so you can run it alone.

RULES I WILL ENFORCE (read these first)

1. **One question at a time.** I do not give you the answer before you attempt it.
2. **You must answer in your own words.** If you paste a definition from this pack, I'll mark it as weak — interviews reward natural speech.
3. **After each answer I score 0-5** using the rubric below, tell you what was missing, give you the corrected version, then ask one follow-up.
4. **Difficulty escalates:** warm-up → core technical → project depth → "why" → "what if" → curveball.
5. **If you can't answer, say "I don't know"** — that's a legitimate move and I'll score it fairly rather than punishing honesty. Bluffing gets a 0 and a note that it damaged credibility.

Scoring rubric

Score	Meaning	What it looks like
0	Wrong	Factually incorrect, or you bluffed and it was caught
1	Very weak	Vague, no structure, doesn't use the right terms
2	Partial	Right idea but missing the mechanism, or a key term is wrong
3	Acceptable	Correct and understandable; thin on detail or examples
4	Strong	Correct, structured, includes the <i>why</i> , connects to your project
5	Excellent	Correct, concise, names the trade-off unprompted, and volunteers the follow-up before being asked

Target: average 3.5+ across the technical questions. A 3 with honest confidence is fine. A 5 that sounds rehearsed is not worth as much as a natural 4.

ROUND 1 — WARM-UP (they decide in 3 minutes whether they like you)

1. Tell me about yourself.

2. Walk me through your resume in 2 minutes.
3. Which project are you most proud of, and why?
4. Why do you want to work at the target company?
5. What do you know about what we do? *(the target company works in the subscription/media-tech space — subscription management and media distribution for publishers. Answer honestly: "I know you build subscription management and media distribution products for publishers; my interest is that it's state-and-entitlement heavy, which is close to the billing and state-machine problems I've been building." Then ask a question back.)*
6. Are you available full-time? When do you graduate?
7. Do you have any questions for us?

Question 7 is graded, and most candidates fail it. Always have three ready:

- "What does the first six months look like for someone in this role?"
 - "What part of your stack do new engineers find hardest to learn?"
 - "How does the team handle code review and testing for a state-heavy product like subscriptions?"
- NOTE: Don't ask about salary or leave in the first round.

ROUND 2 — JAVA + OOP (they will spend 5-10 minutes here)

1. What is OOP, and why do we use it?
2. Explain the four pillars with an example of your own.
3. Difference between `==` and `equals()`.
4. Why is String immutable, and how does that make it a good HashMap key?
5. If two objects are equal, what must be true about their hash codes?
6. ArrayList vs LinkedList — which do you actually use?
7. Explain HashMap internals at a high level.
8. Interface vs abstract class — pick one for a payment method and justify it.
9. Checked vs unchecked exceptions.
10. What does `static` mean, and why can't a static method access instance fields?
11. What is `final` and is a final list immutable?
12. Explain polymorphism with a method you've written.
13. Where does OOP appear in your Spring project? **Follow-ups:** "Is that compile-time or runtime?" · "What's the complexity?" · "What if the class doesn't override hashCode?"

ROUND 3 — SPRING BOOT + REST + JPA (the core of the job)

1. What is Spring, and what problem does it solve?
2. What's the difference between Spring and Spring Boot?
3. What is a bean, and what is the default scope?
4. Explain dependency injection and why you use constructor injection.
5. Walk me through what happens when a request hits your controller.
6. Why do we split controller / service / repository?
7. What does `@Transactional` do, and which exceptions roll back by default?
8. What is a DTO and why not expose entities directly?
9. What is REST? What makes it stateless?

10. GET vs POST vs PUT vs PATCH.
11. You're designing an endpoint to release escrow funds — which method and why?
12. Status codes for: created, validation error, expired token, wrong role, illegal state change, unexpected bug.
13. What is JPA? And Hibernate? And Spring Data JPA?
14. What is ORM and what does it cost you?
15. @ManyToMany vs @OneToMany — which side holds the foreign key?
16. Lazy vs eager loading, and what is LazyInitializationException?
17. What is the N+1 problem and how do you fix it?
18. What is dirty checking?
19. Two threads update the same row at once — what happens in your project?
20. What is optimistic locking? **Follow-ups:** "Show me." · "Why not JDBC?" · "What's the cost of that approach?"

ROUND 4 — POSTGRESQL + SQL

1. Primary key vs foreign key vs unique constraint.
2. Why NUMERIC and not FLOAT for money?
3. What is an index, and why does it slow down writes?
4. Are foreign keys automatically indexed in Postgres? (*No — index them.*)
5. Write a query: count transactions per status, only where the count exceeds 5.
6. Write a query: users who have never made a transaction.
7. INNER vs LEFT JOIN — give an example from your schema.
8. What's ACID, and which part matters most in RevertPay?
9. How would you detect that money doesn't balance in your ledger?
10. How would you find the slowest query in production? (*EXPLAIN ANALYZE, indexes, then the query itself.*)

ROUND 5 — SECURITY

1. Authentication vs authorization.
2. How does JWT work? Walk me through login and a subsequent request.
3. **Is a JWT encrypted?**
4. Where do you store the token, and what's the risk?
5. What if the token is expired?
6. What if the user is authenticated but not authorized? What status code?
7. Roles vs ownership — why isn't a role check enough?
8. Why BCrypt and not SHA-256?
9. What does the salt do?
10. Why is CSRF disabled in your config, and is that safe?
11. How do you log out?
12. How would you implement "only the owner can view their prediction history"?

ROUND 6 — REACT + JAVASCRIPT

1. What is React and what problem does it solve?
2. What is a component? What is JSX?

3. Props vs state.
4. Why can't you mutate state directly?
5. Explain `useEffect` and its dependency array.
6. Why does an effect need a cleanup function? Give a real example.
7. When would you use `useReducer` over `useState`?
8. What are keys and why does the index make a bad key?
9. `map` vs `filter` vs `reduce`.
10. `let` vs `const` vs `var`.
11. Explain `async/await` and the event loop.
12. Why must you check `res.ok` after `fetch`?
13. What does React Router give you?
14. How does your React app authenticate to your API?
15. What happens in the UI when the API returns an error?

ROUND 7 — PROJECT DEEP DIVE (the highest-weighted round)

RevertPay

1. What is escrow and why does it exist?
2. Walk me through the full lifecycle, including disputes.
3. What stops a client from setting any status it wants?
4. Two simultaneous release requests — what happens?
5. Why a ledger instead of a balance column?
6. Draw your database schema.
7. Walk me through the release API end to end, including SQL.
8. What was the hardest part?
9. What's the biggest weakness of this project?
10. How do you know it works? (*NOTE: honest answer required*)
11. If you had two more weeks, what would you build?
12. What would break first if this went to production?

EDU2JOB

13. Summarise the project in a minute.
14. Walk me through a prediction request end to end.
15. Why Python and Java? Why not one?
16. What data enters the model, and what comes out?
17. Why Random Forest and not logistic regression or a neural network?
18. What preprocessing is needed and why?
19. What happens if the encoder sees a category it's never seen?
20. What does prediction history store, and why?
21. What if Flask is down?
22. What's missing from this project?

AlgoViz

23. Why build it?
24. Why record frames instead of animating execution?
25. What exactly is in a frame?
26. What happens when I click pause?

27. How does speed work?
28. What if I click pause twice?
29. What's the cost of your design, and how did you handle it?
30. Why does a value need a stable id?
31. Why React Router? Why Tailwind? Why Vite?
32. What would you improve?

ROUND 8 — THE ATTACK ROUND (expect to lose points here — that's the point)

1. Your resume says five technologies you've used once. Which do you actually know?
2. You claim 500+ LeetCode problems. What's your strongest topic, and prove it.
3. Why is your CGPA 8.51 and not 9+?
4. There's no live demo of any project. How do I know they work?
5. What did you actually write versus what came from a tutorial?
6. Your resume says Spring Security — walk me through the filter chain.
7. What's the most complex bug you've ever fixed?
8. Tell me about a time you were wrong about something technical.
9. Why should we hire a fresher instead of someone with two years' experience?
10. What would your teammate say is your biggest weakness?
11. Have you used Docker? Kubernetes? Kafka? Redis? (*Not hands-on — say so.*)
12. Which of your three projects is the weakest, and why?

The 3 questions most likely to end the interview badly, with the answer shape:

- **"How do I know it works if there's no demo?"** → "You can read the repository — entities, service, security config, and the tests that exist. I'd rather show you the code and be precise about what's verified than pretend there's a deployment."
- **"What did you write vs copy?"** → "I can explain every design decision in RevertPay — the state machine, the transaction boundary, the ownership checks — and I can tell you which parts came from documentation. My honest answer is that the architecture is mine and the API details came from the Spring docs."
- **"Why you over someone experienced?"** → "Because I'm cheap relative to output, I'm comfortable being corrected, and I've already built the same kind of thing you'd give me — state machines, transactional flows, and a UI that reflects them. The proof is that I can explain my decisions, not just my code."

WHAT A GOOD ANSWER SOUNDS LIKE (compare these)

Question: "What is JWT?"

[NO] **Score 1 — vague:**

"It's a token used for authentication. It's secure and encrypted."

[NO] **Score 2 — partially right, one wrong term:**

"JWT has three parts and it's encrypted so nobody can read it. The server gives it to the client after login."

[YES] Score 3 — acceptable:

"JWT is a token with three parts: header, payload and signature. After login the server signs it and sends it to the client, and the client sends it back in the Authorization header. The server checks the signature and the expiry."

[YES][YES] Score 4 — strong:

"JWT is a signed token with three Base64URL parts: a header saying which algorithm, a payload with claims like the user id, the role and the expiry, and a signature computed over the first two with the server's secret. The payload isn't encrypted — anyone can decode it — but nobody can change it without breaking the signature, which is the whole point. On each request my filter reads the bearer header, verifies the signature and expiry, and sets the authentication in the security context so the authorization rules can run. I use it because my API is stateless, so there's no session to store. The trade-off is that revocation is hard — the token stays valid until it expires, which is why expiry should be short and why a refresh-token flow is the next step."

[YES][YES][YES] Score 5 — add the unprompted trade-off:

The 4 answer, plus: "And since the payload is readable, I never put anything sensitive in it — the role and user id are fine, an email is borderline, a password or a token would never go in there."

Notice what earned the extra points: naming the mechanism, correcting the common myth (encrypted), connecting it to your own code, and volunteering the limitation before being asked. **Do that on every technical answer and you'll out-score candidates with more experience.**

HOW TO USE THE RESULTS

After the mock, list every question you scored **≤2** on. Those are your only study targets — ignore everything else. Then:

1. Re-read only the relevant file section.
2. Answer the question out loud **three times**, in your own words.
3. Move to the next weak one.

If you find more than 8 weak areas with less than an hour left, **prioritise in this order:**

1. RevertPay story (all three lengths)
2. JWT + authentication/authorization
3. Spring Boot layers + [@Transactional](#)
4. JPA relationships
5. React props/state/hooks
6. Everything else

THE NIGHT-BEFORE CHECKLIST (5 minutes, do it last)

- [] I can say my self-introduction without notes.
- [] I can say the RevertPay 30-second and 1-minute versions without notes.
- [] I can draw CLIENT → CONTROLLER → SERVICE → REPOSITORY → DB.
- [] I can draw my five tables with their relationships.
- [] I can explain login → token → protected request.
- [] I know: JWT is **signed, not encrypted**.
- [] I know my honest answer for tests, accuracy and deployment.

- [] I have three questions to ask them.
- [] I have the "I don't know, but here's how I'd find out" sentence ready.
- [] I know I'm not going to bluff — and that's why they'll trust me.

Answer these now out loud, then type START MOCK in chat and I'll begin.

1. Tell me about yourself. *(90 seconds max)*
2. What is escrow, and walk me through RevertPay's lifecycle.
3. Why did you use Spring Boot?
4. What is REST?
5. What is JPA, and what is Hibernate?
6. How does JWT work, and is it encrypted?
7. What is React, and what is the difference between props and state?
8. What is Random Forest, and why not one decision tree?
9. What happens in AlgoViz when I click pause?
10. What's the weakest part of your strongest project?